

B

C

# Bachelorarbeit

Einfluss der Netzwerktopologie auf  
Reinforcement-Learning-basierte  
Verteidigungsagenten in simulierten  
Cyberangriffsszenarien

S

# Bachelorarbeit

## Einfluss der Netzwerktopologie auf Reinforcement-Learning-basierte Verteidigungsagenten in simulierten Cyberangriffsszenarien

vorgelegt von

**Mika Oelhoff**

zur Erlangung des akademischen Grades

**Bachelor of Science**

im Studiengang Wirtschaftsinformatik

an der Hochschule Konstanz Technik, Wirtschaft und Gestaltung,  
Fakultät Informatik

Matrikelnummer: 304572

Abgabedatum: 31. August 2026

Erstgutachter/in: **Prof. Dr. Bohnet**

Zweitgutachter/in: **Prof. Dr. Langweg**

# Kurzfassung

Reinforcement Learning wird zunehmend für autonome Cyberabwehr erprobt; welche Rolle die Struktur des verteidigten Netzes dabei spielt, ist jedoch wenig untersucht. Diese Arbeit misst den Einfluss der Netzwerktopologie auf Verteidigungsagenten, die mit Proximal Policy Optimization in der Simulationsumgebung CyberBattleSim trainiert werden, erweitert um einen lernenden Verteidiger. Auf vier Topologien mit identischem Inventar – vollvermascht, Hub-and-Spoke, DMZ und mikrosegmentiert – treten je sechs eingefrorene Angreifer an: fünf Spezialisten und ein über alle Topologien gestuft trainierter Generalist. Die Datengrundlage bilden 120 Trainingsläufe über fünf Zufallsstartwerte sowie eine Evaluation mit 9000 Episoden gegen drei Vergleichsstufen. Ein Einfluss der Topologie auf die Lerngeschwindigkeit ließ sich nicht nachweisen; auf das erreichte Ergebnis ist er hingegen deutlich: Mit zunehmender Segmentierung sinkt der Anteil der Episoden, in denen der Angreifer das wertvollste System erreicht – im mikrosegmentierten Netz bis auf null, während er im vollvermaschten Netz nahezu ungebremsst bleibt, weil Sperren dort ausweichbar sind; erst Engpässe machen sie wirksam. Der auf der verteidigten Topologie trainierte Angreifer erweist sich durchweg als härtester Gegner. Methodisch zeigt die Arbeit, dass der Reward allein Verteidigungsleistung nicht abbildet und das übernommene Reward-Design Prävention unsichtbar ließ. Für die Praxis stützt das die Empfehlung konsequenter Segmentierung: Sie begrenzt, was ein Angreifer erreichen kann, und macht aktive Verteidigung erst wirksam.

# Abstract

Reinforcement learning is increasingly explored for autonomous cyber defense, yet the role played by the structure of the defended network has received little attention. This thesis measures the influence of network topology on defender agents trained with Proximal Policy Optimization in the CyberBattleSim environment, extended with a learning defender. Four topologies with identical inventory – fully meshed, hub-and-spoke, DMZ, and micro-segmented – are each defended against six frozen attackers: five specialists and one generalist trained in stages across all topologies. The data comprise 120 training runs over five random seeds and an evaluation of 9,000 episodes against three baseline tiers. No influence of topology on learning speed could be demonstrated; its influence on the achieved outcome, however, is pronounced: With increasing segmentation, the share of episodes in which the attacker reaches the most valuable system declines – down to zero in the micro-segmented network, while in the fully meshed network the attacker remains largely unchecked because blocks can simply be evaded; only choke points make them effective. The attacker trained on the defended topology consistently proves to be the hardest opponent. Methodologically, the thesis shows that reward alone does not capture defensive performance and that the inherited reward design left prevention invisible. For practice, this supports a recommendation of rigorous segmentation: it limits what an attacker can reach and is what makes active defense effective in the first place.

# Inhaltsverzeichnis

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>Kurzfassung</b>                                                   | <b>i</b>  |
| <b>Abstract</b>                                                      | <b>ii</b> |
| <b>Abkürzungsverzeichnis</b>                                         | <b>vi</b> |
| <b>1 Einleitung</b>                                                  | <b>1</b>  |
| 1.1 Motivation und Problemstellung . . . . .                         | 1         |
| 1.2 Zielsetzung und Forschungsfrage . . . . .                        | 2         |
| 1.3 Aufbau der Arbeit . . . . .                                      | 3         |
| <b>2 Grundlagen</b>                                                  | <b>4</b>  |
| 2.1 Reinforcement Learning . . . . .                                 | 4         |
| 2.1.1 Einordnung in das maschinelle Lernen . . . . .                 | 4         |
| 2.1.2 Das Grundprinzip des Reinforcement Learnings . . . . .         | 5         |
| 2.1.3 Verzögerte Belohnung . . . . .                                 | 6         |
| 2.1.4 Der Markov-Entscheidungsprozess . . . . .                      | 7         |
| 2.1.5 Policy, Return und Diskontierung . . . . .                     | 8         |
| 2.1.6 Wert- und Vorteilsfunktion . . . . .                           | 9         |
| 2.1.7 Actor und Critic . . . . .                                     | 10        |
| 2.1.8 Exploration und Exploitation . . . . .                         | 11        |
| 2.2 Deep Reinforcement Learning . . . . .                            | 11        |
| 2.2.1 Von der Tabelle zur Funktionsapproximation . . . . .           | 11        |
| 2.2.2 Die Actor-Critic-Architektur . . . . .                         | 12        |
| 2.2.3 Aktualisierung der Policy . . . . .                            | 12        |
| 2.2.4 Proximal Policy Optimization . . . . .                         | 14        |
| 2.2.5 Typische Schwierigkeiten . . . . .                             | 16        |
| 2.3 Grundbegriffe der Netzwerksicherheit . . . . .                   | 17        |
| 2.3.1 Angreifer, Verteidiger und der Ablauf eines Angriffs . . . . . | 17        |
| 2.3.2 Netzwerktopologie als Sicherheitsarchitektur . . . . .         | 19        |
| 2.3.3 Segmentierungsmuster . . . . .                                 | 20        |
| 2.4 Stand der Forschung . . . . .                                    | 21        |
| 2.4.1 Simulationsumgebungen für Netzwerkangriffe . . . . .           | 21        |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| 2.4.2    | Lernende Agenten auf beiden Seiten . . . . .                  | 23        |
| 2.4.3    | MARLon . . . . .                                              | 23        |
| <b>3</b> | <b>Versuchsaufbau</b>                                         | <b>24</b> |
| 3.1      | Von MARLon zum eigenen Aufbau . . . . .                       | 24        |
| 3.1.1    | Der geerbte Ausgangspunkt. . . . .                            | 25        |
| 3.1.2    | Eigene Vorarbeiten und der Referenzlauf . . . . .             | 26        |
| 3.1.3    | Drei Fehler in der Umsetzung . . . . .                        | 27        |
| 3.1.4    | Konsequenz für diese Arbeit . . . . .                         | 29        |
| 3.2      | Wahl der Simulationsumgebung und des Lernverfahrens . . . . . | 29        |
| 3.2.1    | Die Simulationsumgebung. . . . .                              | 29        |
| 3.2.2    | Das Lernverfahren . . . . .                                   | 30        |
| 3.3      | Umgebung und Kontrolle der Variablen . . . . .                | 31        |
| 3.3.1    | Die Topologien als Zehn-Knoten-Modell. . . . .                | 32        |
| 3.4      | Trainingsphasen . . . . .                                     | 38        |
| 3.4.1    | Phase 1: Angreifer-Vortraining . . . . .                      | 38        |
| 3.4.2    | Phase 2: Verteidiger-Matrix . . . . .                         | 38        |
| 3.5      | Aktionsraum und Invalid-Action-Masking. . . . .               | 39        |
| 3.5.1    | Der Aktionsraum des Angreifers . . . . .                      | 39        |
| 3.5.2    | Der Aktionsraum des Verteidigers . . . . .                    | 42        |
| 3.5.3    | Mehrstufige Strategien im Aktionsraum . . . . .               | 44        |
| 3.6      | Reward-Struktur und Randbedingungen . . . . .                 | 45        |
| 3.6.1    | Abwehrbonus: ein Zustandsreward für Prävention. . . . .       | 49        |
| 3.6.2    | Verfügbarkeit und SLA-Schwelle . . . . .                      | 50        |
| 3.7      | Konvergenzkriterium . . . . .                                 | 51        |
| 3.8      | Erhobene Metriken . . . . .                                   | 53        |
| 3.9      | Wirksamkeitsevaluation. . . . .                               | 54        |
| 3.9.1    | Aufbau . . . . .                                              | 54        |
| 3.9.2    | Auswertungsgrößen . . . . .                                   | 55        |
| <b>4</b> | <b>Ergebnisse und Interpretation</b>                          | <b>57</b> |
| 4.1      | Lernkurven je Topologie . . . . .                             | 58        |
| 4.2      | Crown-Jewel-Quote im Trainingsverlauf . . . . .               | 67        |
| 4.3      | Gehaltene Knoten im Trainingsverlauf . . . . .                | 69        |
| 4.4      | Konvergenz . . . . .                                          | 70        |
| 4.5      | Aktionsverhalten des Verteidigers. . . . .                    | 73        |
| 4.6      | Wirksamkeit der trainierten Verteidiger . . . . .             | 75        |
| 4.7      | Einfluss der Angreiferherkunft . . . . .                      | 79        |
| 4.8      | Einordnung des Reward-Designs . . . . .                       | 80        |

|          |                                          |            |
|----------|------------------------------------------|------------|
| 4.9      | Limitationen . . . . .                   | 81         |
| <b>5</b> | <b>Fazit und Ausblick</b>                | <b>84</b>  |
| 5.1      | Fazit . . . . .                          | 84         |
| 5.2      | Ausblick . . . . .                       | 86         |
|          | <b>Literaturverzeichnis</b>              | <b>88</b>  |
| <b>A</b> | <b>Anhang</b>                            | <b>91</b>  |
| A.1      | Quellcode und Daten . . . . .            | 91         |
| A.2      | Hyperparameter . . . . .                 | 92         |
| A.3      | Crown-Jewel-Quote je Topologie . . . . . | 94         |
| A.4      | Gehaltene Knoten je Topologie . . . . .  | 96         |
| A.5      | SLA-Brüche je Topologie . . . . .        | 99         |
| A.6      | Abwehrbonus je Topologie . . . . .       | 101        |
| A.7      | Konvergenzmarken je Topologie. . . . .   | 104        |
|          | <b>Eidesstattliche Erklärung</b>         | <b>107</b> |

# Abkürzungsverzeichnis

|             |                                                |
|-------------|------------------------------------------------|
| <b>CBS</b>  | CyberBattleSim                                 |
| <b>DMZ</b>  | Demilitarized Zone                             |
| <b>KI</b>   | Künstliche Intelligenz                         |
| <b>MARL</b> | Multi-Agent Reinforcement Learning             |
| <b>MDP</b>  | Markov Decision Process                        |
| <b>NIST</b> | National Institute of Standards and Technology |
| <b>PPO</b>  | Proximal Policy Optimization                   |
| <b>RL</b>   | Reinforcement Learning                         |
| <b>SB3</b>  | Stable-Baselines3                              |
| <b>SLA</b>  | Service Level Agreement                        |
| <b>A2C</b>  | Advantage Actor-Critic                         |

# Abbildungsverzeichnis

|      |                                                 |     |
|------|-------------------------------------------------|-----|
| 2.1  | Trainingsschleife eines Actor-Critic-Verfahrens | 14  |
| 3.1  | Topologie <i>flat</i>                           | 34  |
| 3.2  | Topologie <i>dmz</i>                            | 35  |
| 3.3  | Topologie <i>hub_and_spoke</i>                  | 36  |
| 3.4  | Topologie <i>micro_segmented</i>                | 37  |
| 3.5  | Topologie <i>chain</i>                          | 37  |
| 4.1  | Lernkurven auf <i>flat</i>                      | 60  |
| 4.2  | Lernkurven auf <i>hub_and_spoke</i>             | 61  |
| 4.3  | Reward und SLA-Brüche je Seed                   | 62  |
| 4.4  | Lernkurven auf <i>dmz</i>                       | 64  |
| 4.5  | Reward und Crown-Jewel-Quote in <i>dmz</i>      | 65  |
| 4.6  | Lernkurven auf <i>micro_segmented</i>           | 66  |
| 4.7  | Crown-Jewel-Quote im Trainingsverlauf           | 68  |
| 4.8  | Gehaltene Knoten im Trainingsverlauf            | 69  |
| 4.9  | Reimages und Sperren im Trainingsverlauf        | 73  |
| 4.10 | Abwehrbonus im Trainingsverlauf                 | 74  |
| A.1  | Crown-Jewel-Quote auf <i>flat</i>               | 94  |
| A.2  | Crown-Jewel-Quote auf <i>hub_and_spoke</i>      | 95  |
| A.3  | Crown-Jewel-Quote auf <i>dmz</i>                | 95  |
| A.4  | Crown-Jewel-Quote auf <i>micro_segmented</i>    | 96  |
| A.5  | Gehaltene Knoten auf <i>flat</i>                | 97  |
| A.6  | Gehaltene Knoten auf <i>hub_and_spoke</i>       | 97  |
| A.7  | Gehaltene Knoten auf <i>dmz</i>                 | 98  |
| A.8  | Gehaltene Knoten auf <i>micro_segmented</i>     | 98  |
| A.9  | SLA-Brüche auf <i>flat</i>                      | 99  |
| A.10 | SLA-Brüche auf <i>hub_and_spoke</i>             | 100 |
| A.11 | SLA-Brüche auf <i>dmz</i>                       | 100 |
| A.12 | SLA-Brüche auf <i>micro_segmented</i>           | 101 |
| A.13 | Abwehrbonus auf <i>flat</i>                     | 102 |
| A.14 | Abwehrbonus auf <i>hub_and_spoke</i>            | 102 |

---

|                                                            |     |
|------------------------------------------------------------|-----|
| A.15 Abwehrbonus auf <i>dmz</i> . . . . .                  | 103 |
| A.16 Abwehrbonus auf <i>micro_segmented</i> . . . . .      | 103 |
| A.17 Konvergenzmarken auf <i>flat</i> . . . . .            | 104 |
| A.18 Konvergenzmarken auf <i>hub_and_spoke</i> . . . . .   | 105 |
| A.19 Konvergenzmarken auf <i>dmz</i> . . . . .             | 105 |
| A.20 Konvergenzmarken auf <i>micro_segmented</i> . . . . . | 106 |

# Tabellenverzeichnis

|     |                                                       |    |
|-----|-------------------------------------------------------|----|
| 3.1 | Aufbau des Aktionsraums . . . . .                     | 40 |
| 3.2 | Aktionsraum des Verteidigers . . . . .                | 43 |
| 3.3 | Reward des Angreifers . . . . .                       | 45 |
| 3.4 | Reward des Verteidigers . . . . .                     | 46 |
| 4.1 | Angreifer-Reward als Start- und Endniveau . . . . .   | 59 |
| 4.2 | Verteidiger-Reward als Start- und Endniveau . . . . . | 67 |
| 4.3 | Konvergenzepisoden je Matchup . . . . .               | 71 |
| 4.4 | Restanteil in der Evaluation . . . . .                | 76 |
| 4.5 | Crown-Jewel-Quote in der Evaluation . . . . .         | 77 |
| 4.6 | Gehaltene Knoten in der Evaluation . . . . .          | 78 |
| 4.7 | Aufwand der Verteidigung . . . . .                    | 79 |
| A.1 | Hyperparameter des Trainings . . . . .                | 93 |

# 1

## Einleitung

Dieses Kapitel führt in das Thema ein. Es beschreibt zunächst, wie sich der Einsatz künstlicher Intelligenz auf beiden Seiten eines Angriffs auswirkt und welche Frage daraus für die Gestaltung von Netzen folgt, leitet daraus Ziel und Forschungsfrage ab und beschreibt abschließend den Aufbau der Arbeit.

### 1.1. Motivation und Problemstellung

Künstliche Intelligenz (KI) ist in der Cybersicherheit auf beiden Seiten angekommen. Angreifer setzen generative Modelle ein, um Phishing zu skalieren, Schadcode schneller zu entwickeln und Social Engineering überzeugender zu machen; die KI-gestützte Angriffsaktivität nahm binnen eines Jahres um 89 Prozent zu, und vom ersten Zugriff bis zum Übergriff auf weitere Systeme vergehen im Durchschnitt weniger als dreißig Minuten [1]. Verteidiger nutzen KI ihrerseits, um Telemetrie auszuwerten, Anomalien zu erkennen und Reaktionszeiten zu verkürzen [2]. An den Grundlagen der Angriffe ändert das wenig, denn ausgenutzt werden weiterhin ungepatchte Lücken, gültige Zugangsdaten und Fehlkonfigurationen. Verändert haben sich Geschwindigkeit und Anpassungsfähigkeit. Angreifer, die ihre Taktik in Echtzeit an veränderte Bedingungen anpassen, setzen Verteidigungen unter Druck, die auf festen Regeln und Signaturen beruhen [2].

Damit stellt sich die Frage, ob nicht auch die Verteidigung selbst lernen sollte. Reinforce-

ment Learning (RL) ist dafür ein naheliegender Kandidat: Dabei lernt ein eigenständig handelndes Programm, ein sogenannter *Agent*, sein Verhalten aus den Folgen seiner Handlungen statt aus vorgegebenen Regeln und muss deshalb nicht auf jede Angriffsvariante vorab vorbereitet werden [3]. Mit Simulationsumgebungen wie Microsofts CyberBattleSim (CBS) [4] oder CybORG [5] stehen dafür Trainingsplätze bereit, und Erweiterungen wie MARLON [6] stellen dem lernenden Angreifer einen lernenden Verteidiger gegenüber.

Der Blick dieser Forschung richtet sich bislang vor allem auf die Agenten: auf Algorithmen, Beobachtungsräume und Belohnungsfunktionen. Die Umgebung, in der gelernt wird, bleibt dabei meist eine feste Randbedingung, das gilt insbesondere für die Struktur des verteidigten Netzes. In der Praxis gilt jedoch gerade diese Struktur als tragender Teil der Verteidigung: Segmentierung bis hin zur Mikrosegmentierung ist ein Grundpfeiler moderner Sicherheitsarchitekturen [7]. Ob sich dieser Praxisgrundsatz im Verhalten lernender Verteidiger niederschlägt, ob ein Agent in einem segmentierten Netz also schneller lernt und wirksamer verteidigt, ist eine offene Frage. Eine entsprechende Abhängigkeit besteht auf der Gegenseite. Auch der Angreifer wird auf einer bestimmten Topologie trainiert. Zu prüfen ist daher, ob seine Wirksamkeit davon abhängt, ob die angegriffene Topologie mit seiner Trainingstopologie übereinstimmt.

Diese Fragen lassen sich nur in einem Aufbau beantworten, in dem allein die Topologie variiert, während Inventar, Knotenwerte und Trainingsbudget gleich bleiben, und in dem die Wirksamkeit an Größen gemessen wird, die tatsächlich Verteidigungsleistung abbilden. Beides erwies sich als eigenständige Aufgabe: Der übernommene Aufbau aus MARLON musste dafür an mehreren Stellen umgebaut werden (Kapitel 3).

## 1.2. Zielsetzung und Forschungsfrage

Diese Arbeit untersucht, wie sich die Netzwerktopologie auf Verteidigungsagenten auswirkt, die mit RL trainiert werden. Die Fragestellung gliedert sich in drei Teilfragen: wie schnell ein solcher Agent ein stabiles Verhalten erreicht, wie wirksam er nach abgeschlossenem Training den Angriff eindämmt, und welchen Einfluss darauf die Topologie hat, auf der der Angreifer trainiert wurde. Die zentrale Forschungsfrage lautet:

*Wie beeinflusst die Topologie des verteidigten Netzes das Training und die Wirksamkeit eines mit Reinforcement Learning trainierten Verteidigungsagenten, und welchen Anteil an diesem Einfluss hat die Topologie, auf der der Angreifer trainiert wurde?*

Der zur Beantwortung entworfene Versuchsaufbau ist Gegenstand von Kapitel 3.

### 1.3. Aufbau der Arbeit

Kapitel 2 legt die Grundlagen. Beschrieben werden das RL und das eingesetzte Verfahren Proximal Policy Optimization (PPO), die Grundbegriffe der Netzwerksicherheit samt den Prinzipien der Netzsegmentierung sowie der Stand der Forschung zu Simulationsumgebungen für Netzwerkangriffe, aus dem die Wahl von CBS mit der MARL-on-Erweiterung hervorgeht.

Kapitel 3 beschreibt den Versuchsaufbau. Dazu gehören die vier verglichenen Topologien, der Umbau von Aktionsraum und Belohnungsfunktion gegenüber dem übernommenen Rahmenwerk, die beiden Trainingsphasen, die erhobenen Zielgrößen und das Konvergenzkriterium sowie der Ablauf der abschließenden Evaluation.

Kapitel 4 stellt die Ergebnisse dar und ordnet sie unmittelbar ein. Auf ein eigenes Diskussionskapitel wird bewusst verzichtet, damit Befund, Deutung und die zugehörigen Tabellen und Abbildungen an derselben Stelle stehen. Ein abschließender Abschnitt benennt die Grenzen der Aussagekraft.

Kapitel 5 beantwortet die Forschungsfrage entlang der drei Teilfragen, leitet daraus eine Empfehlung für die Praxis ab und benennt Ansatzpunkte für Anschlussarbeiten. Der Anhang enthält die Hyperparameter sowie die vollständigen Verläufe aller 24 Kombinationen, die Kapitel 4 nur ausschnittsweise wiedergibt.

# 2

## Grundlagen

Dieses Kapitel führt die Begriffe und Verfahren ein, die für die Arbeit vorausgesetzt werden. Es reicht vom Reinforcement Learning über die Grundbegriffe der Netzwerksicherheit bis zu den Arbeiten, an die die Untersuchung anschließt. Wie diese Grundlagen für die eigenen Versuche verwendet werden, behandelt Kapitel 3.

Als Beispielszenario dient die Abwehr von Angriffen in einem Unternehmensnetz. Es begleitet die Darstellung von den allgemeinen Lernverfahren bis zu den sicherheitstechnischen Begriffen und verbindet so beide Hälften des Kapitels.

### 2.1. Reinforcement Learning

Dieser Abschnitt entwickelt das RL schrittweise von seiner Einordnung im maschinellen Lernen bis zu den Grundlagen, die der weitere Text voraussetzt.

#### 2.1.1. Einordnung in das maschinelle Lernen

Maschinelles Lernen umfasst Verfahren, die eine Aufgabe nicht über eine ausformulierte Regel lösen, sondern ihr Verhalten aus Erfahrung ableiten. Welche Art von Rückmeldung diese Erfahrung liefert, trennt die Teilgebiete. Neben dem überwachten und dem unüberwachten Lernen führen Sutton und Barto entlang dieses Kriteriums das RL als dritte Form an [8, Abschn. 1.1]. Für diese dritte Form ist im deutschen Sprachgebrauch

keine Bezeichnung gebräuchlich. Sie wird deshalb durchgehend englisch benannt.

Beim *überwachten Lernen* ist die Rückmeldung *instruktiv*. Ein Klassifikator erhält Mitschnitte des Netzwerkverkehrs, die vorab als gutartig oder böartig eingestuft wurden. Weil zu jeder Verbindung die korrekte Einstufung bekannt ist, lässt sich der Fehler des Modells unmittelbar berechnen. Beim *unüberwachten Lernen* entfällt jede Bewertung. Derselbe Klassifikator erhält dieselben Mitschnitte ohne Einstufung und fasst Verbindungen mit ähnlichem Muster zu Gruppen zusammen, ohne zu wissen, ob eine dieser Gruppen einen Angriff darstellt.

Beim RL ist die Rückmeldung *evaluativ*. Hier lässt sich das Beispiel des Klassifikators nicht mehr fortführen, denn dieser ordnet vorliegende Daten ein, ohne in das Geschehen einzugreifen. An seine Stelle tritt ein Agent, der handelt und für jede Handlung eine Bewertung erhält, aber keinen Hinweis darauf, welche Handlung besser gewesen wäre. Die Rückmeldung bewertet also, ohne anzuleiten [8, S. 25]. Im vorliegenden Beispiel entspricht das einem Verteidiger, der auf die Lage im Netz mit einer Gegenmaßnahme reagiert und allein daran gemessen wird, wie sich diese auswirkt.

Ein zweiter Unterschied betrifft nicht die Rückmeldung, sondern die Herkunft der Daten. In den ersten beiden Fällen liegen die Mitschnitte vor und sind vom Modell unabhängig; man könnte dasselbe Modell zweimal auf denselben Daten trainieren. Im RL entsteht die Erfahrung dagegen erst durch das Handeln des Agenten: Seine Maßnahmen bestimmen, welche Situationen er anschließend überhaupt zu sehen bekommt. Sichert ein Verteidiger einen Netzbereich nicht ab, so kann er auch nicht die Zustände beobachten, welche aus dieser Maßnahme hervorgegangen wären.

In beiden zuletzt genannten Fällen sind die Mitschnitte nicht eingestuft, was RL als Sonderfall des unüberwachten Lernens erscheinen lässt. Sutton und Barto widersprechen dem ausdrücklich [8, Abschn. 1.1]: Maßgeblich ist nicht die fehlende Einstufung, sondern das Ziel. Unüberwachtes Lernen sucht verborgene Struktur, RL maximiert ein Reward-Signal.

### 2.1.2. Das Grundprinzip des Reinforcement Learnings

Ein RL-Aufbau besteht aus zwei Teilen, dem *Agenten* und der *Umgebung*, die in einer festen Schleife miteinander interagieren. Zu jedem diskreten Zeitschritt  $t$  beobachtet der Agent den *Zustand*  $s_t$  der Umgebung, wählt daraufhin eine *Aktion*  $a_t$  und erhält von der Umgebung zwei Daten zurück: den neuen Zustand  $s_{t+1}$  und ein Reward-Signal  $R_{t+1}$ , also eine einzelne Zahl, die diesen Schritt bewertet.

Wie lange diese Schleife läuft, hängt von der Aufgabe ab. Sutton und Barto unterscheiden *fortlaufende* Aufgaben, die kein natürliches Ende kennen, von *episodischen*, die einen Endzustand besitzen [8, Abschn. 3.3]; welche Folgen das hat, zeigt Abschnitt 2.1.4. Die vorliegende Arbeit betrachtet ausschließlich den zweiten Fall. Ein Durchlauf vom Anfangszustand bis zum Endzustand heißt *Episode*; ein Training umfasst eine Vielzahl solcher Episoden. Die Umgebung wird dabei zu Beginn jeder Episode zurückgesetzt, die gelernten Parameter des Agenten bleiben jedoch erhalten.

Was der Agent dabei nicht erfährt, ist ebenso wesentlich. Ob eine andere Wahl zu einem höheren Reward geführt hätte, muss er durch wiederholtes Ausprobieren erschließen. Sein Ziel ist außerdem nicht der einzelne Reward, sondern deren Summe über die gesamte Episode. Das schließt den Fall ein, dass eine Aktion mit geringem unmittelbarem Reward erst die Voraussetzung für spätere, höhere Rewards schafft.

Dahinter steht eine Annahme, die Sutton und Barto als *Reward-Hypothese* benennen. Jedes Ziel lasse sich als Maximierung des Erwartungswerts einer einzelnen skalaren Größe auffassen, der Summe der erhaltenen Rewards [8, Abschn. 3.2]. Ob eine konkrete Reward-Funktion das beabsichtigte Ziel tatsächlich abbildet, ist damit nicht sichergestellt; Abschnitt 2.2.5 greift dies wieder auf.

Vier Bestandteile setzen das Verfahren zusammen. Die *Policy* legt das Verhalten des Agenten fest, indem sie jedem Zustand die dort zu wählende Aktion zuordnet. Das *Reward-Signal* gibt das Ziel vor und bewertet jeden Schritt einzeln. Die *Wertfunktion* schätzt demgegenüber die Summe der Rewards, die von einem Zustand aus noch zu erwarten sind, und blickt damit über den einzelnen Schritt hinaus. Hinzu kommt gegebenenfalls ein *Modell* der Umgebung, das vorhersagt, wie diese auf eine Aktion reagieren wird [8, Abschn. 1.3]. Die folgenden Abschnitte führen diese Bestandteile der Reihe nach ein.

### 2.1.3. Verzögerte Belohnung

Eine zentrale Herausforderung des Verfahrens besteht darin, dass ein Reward und die für ihn ursächliche Aktion zeitlich weit auseinanderliegen können. Eine Aktion kann unmittelbar keinen Reward erbringen und dennoch die Voraussetzung für einen Erfolg schaffen, der erst viele Schritte später eintritt. Der Reward fällt am Ende dieser Kette an, ohne dass erkennbar wäre, welchem der vorangegangenen Schritte er zuzurechnen ist.

Für das vorliegende Beispiel: Übernimmt ein Angreifer einen unscheinbaren Rechner, bringt ihm das kaum Punkte. Findet er dort jedoch Zugangsdaten, die ihm zwanzig Schritte später den Zugriff auf ein wertvolles System eröffnen, war genau dieser frühe

Schritt ausschlaggebend.

Minsky benannte diese Schwierigkeit bereits 1961 als *Credit-Assignment-Problem* [9]. Es ist der Hauptgrund dafür, dass RL sehr viele Durchläufe benötigt: Die Bewertung eines frühen Schrittes lässt sich erst festigen, nachdem sein Nutzen über viele Episoden hinweg wiederholt sichtbar geworden ist. Sämtliche im Folgenden eingeführten Größen sind Werkzeuge, die dieses Problem adressieren.

#### 2.1.4. Der Markov-Entscheidungsprozess

Um das Zusammenspiel aus Abschnitt 2.1.2 präzise zu beschreiben, wird es als Markov Decision Process (MDP) formalisiert. Hinter dem Begriff verbirgt sich eine Sammlung von fünf Angaben, die jede RL-Aufgabe ohnehin festlegen muss:

1. die Menge aller möglichen Zustände  $\mathcal{S}$ ,
2. die Menge aller Aktionen  $\mathcal{A}$ ,
3. die Übergangsfunktion  $P(s' | s, a)$ , also die Wahrscheinlichkeit, nach Aktion  $a$  im Zustand  $s$  im Zustand  $s'$  zu landen,
4. die Reward-Funktion  $R(s, a)$ , welche die unmittelbare Bewertung liefert,
5. den Diskontfaktor  $\gamma$  mit  $0 \leq \gamma \leq 1$ , der festlegt, wie stark künftige Rewards gegenüber sofortigen zählen.

Benannt ist das Modell nach dem Mathematiker Andrei Markov, auf dessen Arbeiten zu stochastischen Prozessen die zugrunde liegende Annahme zurückgeht: Der Zustand enthält bereits alle Information, die für die Zukunft von Bedeutung ist. Wie der Agent dorthin gelangt ist, spielt keine Rolle. Beim Schach trifft das zu, denn für den nächsten Zug zählt allein die aktuelle Stellung, nicht die Zugfolge, die zu ihr geführt hat. Die Annahme vereinfacht das Problem erheblich, weil der Agent keine Vorgeschichte mitführen muss [8, Abschn. 3.1].

Beide Randwerte des Diskontfaktors sind zulässig, führen aber zu Sonderfällen. Bei  $\gamma = 0$  zählt ausschließlich der unmittelbare Reward; der Agent handelt vollständig kurz-sichtig und kann verzögerte Belohnungen aus Abschnitt 2.1.3 grundsätzlich nicht mehr erkennen. Bei  $\gamma = 1$  zählt jeder künftige Reward mit vollem Gewicht. Zulässig ist das nur bei episodischen Aufgaben im Sinne von Abschnitt 2.1.2, bei denen jeder Durchlauf sicher endet. Eine fortlaufende Aufgabe liefert dagegen unendlich viele Rewards, deren Summe ohne Abwertung unbeschränkt anwachsen kann; das Ziel des Agenten wäre dann nicht mehr definiert. Ein Wert knapp darunter genügt bereits: Die Beiträge nehmen

dann geometrisch ab, und die Summe konvergiert.

Die fünf Angaben beschreiben die Aufgabe. Ob der Agent die Übergangsfunktion  $P$  und die Reward-Funktion  $R$  seinerseits kennt, ist damit noch nicht gesagt, und davon hängt ab, wie er lernen kann [8, Abschn. 1.3]. Beim Schach kennt er beide, denn die Regeln bestimmen, welche Stellung auf einen Zug folgt und ob sie gewonnen ist; er kann Züge daher durchspielen, ohne sie auszuführen. Solche Verfahren heißen *modellbasiert*. Ein Verteidiger im Unternehmensnetz hat diese Möglichkeit nicht: Sperrt er eine Verbindung, weiß er weder, ob der Angreifer sie überhaupt nutzen wollte, noch ob dieser auf einen anderen Weg ausweicht. Er erfährt erst hinterher, was eingetreten ist, und Verfahren, die allein damit arbeiten, heißen *modellfrei*.

Die Agenten dieser Arbeit lernen modellfrei, und eine Alternative besteht nicht: Eine Übergangsfunktion für ein Netz unter Angriff aufzustellen setzte voraus, den Ausgang jedes möglichen Angriffsschritts vorab zu kennen. Was sich nicht vorausrechnen lässt, muss ausprobiert werden; daraus folgt der hohe Bedarf an Erfahrung, den Abschnitt 2.2.5 aufgreift.

### 2.1.5. Policy, Return und Diskontierung

Das Verhalten des Agenten beschreibt die *Policy*  $\pi(a | s)$ . Sie ist eine bedingte Wahrscheinlichkeit:  $\pi(a | s)$  gibt an, mit welcher Wahrscheinlichkeit der Agent die Aktion  $a$  ausführt, wenn er sich im Zustand  $s$  befindet. Für jeden festen Zustand ergibt sie damit eine Wahrscheinlichkeitsverteilung über alle Aktionen, deren Werte sich zu eins summieren. Die Policy ist genau das, was im Training verändert wird.

Ihr Ziel ist nicht der größtmögliche unmittelbare Reward. Maßgeblich ist der *Return*  $G_t$  zum Zeitpunkt  $t$ , also die Summe aller noch folgenden Rewards  $R_{t+k+1}$ , wobei spätere Beiträge mit dem Diskontfaktor  $\gamma$  abgewertet werden:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \quad (2.1)$$

Der Laufindex  $k$  zählt dabei die künftigen Schritte, sodass der Faktor  $\gamma^k$  einen Reward umso stärker abwertet, je weiter er in der Zukunft liegt. Der um eins verschobene Index folgt der Zählweise aus Abschnitt 2.1.2: Ein Reward wird nach dem Zeitpunkt benannt, zu dem er *eintrifft*, und er trifft stets einen Schritt nach der Aktion ein, die ihn ausgelöst hat. Für  $k = 0$  steht dort deshalb  $R_{t+1}$ , also der Reward für die zum Zeitpunkt  $t$  gewählte Aktion.

### 2.1.6. Wert- und Vorteilsfunktion

Um Zustände und Aktionen vergleichbar zu machen, dienen zwei Größen. Die *Aktionswertfunktion*  $Q^\pi(s, a)$  gibt den erwarteten Return an, wenn der Agent im Zustand  $s$  zunächst die Aktion  $a$  ausführt und erst danach wieder der Policy  $\pi$  folgt. Die *Zustandswertfunktion*  $V^\pi(s)$  beantwortet dieselbe Frage ohne Festlegung auf eine bestimmte Aktion: Sie ist der Erwartungswert von  $Q^\pi(s, a)$  über die Aktionen, die die Policy im Zustand  $s$  wählt. Bei endlich vielen Aktionen ist das die mit den Auswahlwahrscheinlichkeiten gewichtete Summe:

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot | s)}[Q^\pi(s, a)] = \sum_{a \in \mathcal{A}} \pi(a | s) Q^\pi(s, a). \quad (2.2)$$

Dabei bezeichnet  $\mathbb{E}$  den Erwartungswert, und die Schreibweise  $a \sim \pi(\cdot | s)$  besagt, dass die Aktion  $a$  gemäß der Policy im Zustand  $s$  gezogen wird. Auch  $V^\pi$  setzt Handeln voraus, der Unterschied zu  $Q^\pi$  liegt allein darin, dass über die Aktionen gemittelt wird, statt eine einzelne festzulegen. Der Zustandswert steht damit nicht für eine bestimmte Aktion, etwa die wahrscheinlichste, sondern für den Erwartungswert dessen, was die Policy in  $s$  tun würde.

Aus der Differenz beider ergibt sich die *Vorteilsfunktion*, benannt nach dem englischen *advantage* und nicht zu verwechseln mit der Aktionsmenge  $\mathcal{A}$ :

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s). \quad (2.3)$$

Sie beantwortet die für das Training entscheidende Frage: War diese Aktion besser als das, was die Policy in dieser Lage üblicherweise tut? Ein positiver Wert bedeutet besser als der eigene Erwartungswert, ein negativer schlechter. Lernverfahren verschieben die Policy in Richtung der Aktionen mit positivem Vorteil; wie das im Einzelnen geschieht, behandelt Abschnitt 2.2.3.

Ein Zahlenbeispiel verdeutlicht das Zusammenspiel. Stünden dem Agenten in einer Lage drei Aktionen mit den Aktionswerten 80, 10 und 0 offen und wählte die Policy sie mit 70, 20 und 10 Prozent, ergäbe sich nach Gleichung (2.2) ein Zustandswert von  $0,7 \cdot 80 + 0,2 \cdot 10 + 0,1 \cdot 0 = 58$ . Die erste Aktion hätte damit den Vorteil +22, die zweite −48. Steigt der Anteil der ersten im Lauf des Trainings auf 95 Prozent, wächst der Zustandswert auf 76,4, und ihr Vorteil schrumpft auf +3,6.

Die Bezugsgröße wächst also mit der Policy mit. Sobald der Agent überwiegend das Beste tut, kann nichts mehr deutlich besser als sein eigener Erwartungswert sein, und die Anpassungen kommen zum Erliegen. Formal folgt das aus Gleichung (2.2): Unter

der Policy gemittelt ergibt der Vorteil stets null.

Der Vorteil misst nur den Abstand zum eigenen Erwartungswert. Ob eine Lage insgesamt günstig oder ungünstig ist, fällt damit heraus; verglichen wird ausschließlich innerhalb einer Lage. Unempfindlich gegen die Höhe der Rewards ist das Verfahren deshalb aber nicht: Werden alle Rewards mit zehn multipliziert, wächst der Vorteil um denselben Faktor und mit ihm die Größe der Anpassung. Heraus fällt das Niveau einer einzelnen Lage, nicht der Maßstab der Reward-Funktion.

$V^\pi$  und  $Q^\pi$  sind dem Agenten allerdings nicht bekannt. Wären sie es, ließe sich stets die Aktion mit dem höchsten Aktionswert wählen, und ein Lernproblem bestünde nicht. Beide müssen deshalb geschätzt werden, und wie das geschieht, behandelt Abschnitt 2.2.

### 2.1.7. Actor und Critic

Aus den bisher eingeführten Größen ergibt sich eine Aufgabenteilung. Ein Agent muss zweierlei leisten: Er muss eine Aktion wählen, und er muss einschätzen können, wie gut seine Lage ist. Das Erste leistet die Policy aus Abschnitt 2.1.5, das Zweite die Zustandswertfunktion aus Abschnitt 2.1.6. Beide sind bislang nur beschrieben, nicht verortet.

Eine verbreitete Familie von Verfahren weist die beiden Aufgaben deshalb getrennten Bestandteilen zu, und das erklärt zugleich deren Namen [8, Abschn. 13.5]. Der *Actor* hält die Policy und wählt die Aktionen; er ist der handelnde Teil. Der *Critic* schätzt den Zustandswert und bewertet im Nachhinein, ob das Ergebnis besser oder schlechter ausfiel als erwartet. Diese Differenz ist der Vorteil aus Gleichung (2.3). Er steuert die Anpassung des Actors, während der Critic zugleich an den beobachteten Rewards nachjustiert wird. Beide verbessern sich also im selben Durchlauf; zu Beginn ist der Critic allerdings ungenau, weshalb frühe Vorteilsschätzungen mit Vorsicht zu betrachten sind.

Diese Zweiteilung ist nicht zwingend. *Wertbasierte* Verfahren kommen ohne eigenen Actor aus und leiten ihr Verhalten unmittelbar aus geschätzten Aktionswerten ab; rein *policybasierte* Verfahren verzichten umgekehrt auf den Critic und arbeiten allein mit den beobachteten Returns [8, Abschn. 13.3–13.5]. *Actor-Critic-Verfahren* verbinden beides. Der in dieser Arbeit eingesetzte Algorithmus zählt zu dieser Klasse.

### 2.1.8. Exploration und Exploitation

Ein Agent, der stets die bislang beste bekannte Aktion wählt, entdeckt nichts Besseres mehr. Einer, der nur ausprobiert, nutzt sein Wissen nicht. Zwischen diesen beiden Polen muss abgewogen werden, und die Abwägung ist nicht einmalig, sondern begleitet das gesamte Training. Sutton und Barto heben diesen Konflikt als eine Schwierigkeit hervor, die im überwachten und unüberwachten Lernen gar nicht erst auftritt [8, Abschn. 1.1].

Verfahren mit *stochastischer* Policy lösen das implizit. Weil die Policy nach Abschnitt 2.1.5 eine Wahrscheinlichkeitsverteilung ausgibt und keine einzelne Aktion, bleibt ein Rest an Exploration erhalten, solange diese Verteilung nicht auf eine einzige Aktion zusammenfällt. Der Agent zieht seine Aktion in jedem Schritt neu aus der Verteilung und weicht damit gelegentlich von seiner derzeit besten Einschätzung ab.

## 2.2. Deep Reinforcement Learning

Der vorangegangene Abschnitt beschreibt das RL unabhängig davon, wie Policy und Wertfunktion dargestellt werden. Dieser Abschnitt führt tiefe neuronale Netze als solche Darstellung ein, beschreibt die daraus entstehende Actor-Critic-Architektur und das in dieser Arbeit eingesetzte Verfahren PPO und schließt mit den Schwierigkeiten, die dieses Vorgehen mit sich bringt.

### 2.2.1. Von der Tabelle zur Funktionsapproximation

Wie Policy und Wertfunktionen dargestellt werden, blieb bisher offen. Für kleine Aufgaben genügt eine Tabelle mit einem Eintrag je Zustand. Die Einträge stehen dabei unabhängig nebeneinander: Was der Agent in einem Zustand gelernt hat, steht in genau einer Zeile und sagt über keine andere etwas aus. Bei großen Zustandsräumen wird das zum Problem, denn dort erreicht der Agent jeden einzelnen Zustand nur selten oder überhaupt nicht. Er müsste jeden davon eigens erleben, um ihn beurteilen zu können.

Die *Funktionsapproximation* ersetzt die Tabelle durch eine Funktion, die den Wert aus den Merkmalen eines Zustands *berechnet*, statt ihn nachzuschlagen. Sie besitzt einstellbare Parameter, die im Training angepasst werden; bei neuronalen Netzen heißen diese *Gewichte* und werden zusammenfassend mit  $\theta$  bezeichnet. Weil ähnliche Zustände ähnliche Merkmale aufweisen, liefert die Funktion für sie auch ähnliche Ergebnisse. Gelerntes überträgt sich damit auf Zustände, die der Agent nie besucht hat.

Werden für diese Approximation tiefe neuronale Netze eingesetzt, spricht man von *Deep Reinforcement Learning*. Der Ansatz verbindet die Entscheidungslogik des RL mit der Fähigkeit neuronaler Netze, aus hochdimensionalen Beobachtungen brauchbare Merkmale zu gewinnen [8, Teil II]. Auf diese Weise lernten Agenten bereits allein aus Bildpunkten und Punktestand, Videospiele auf menschlichem Niveau zu spielen [10, S. 529].

### 2.2.2. Die Actor-Critic-Architektur

Actor und Critic aus Abschnitt 2.1.7 sind bislang Rollen, keine konkreten Gebilde: Es steht fest, was sie leisten sollen, nicht aber, wodurch. Die Funktionsapproximation beantwortet das. Beide Rollen werden durch je ein neuronales Netz besetzt, und diese Aufteilung heißt *Actor-Critic-Architektur* [8, Abschn. 13.5].

Der *Actor* bildet den beobachteten Zustand auf eine Wahrscheinlichkeitsverteilung über die Aktionen ab und stellt damit die Policy dar; seine Gewichte werden mit  $\theta$  bezeichnet, weshalb die Policy im Folgenden als  $\pi_\theta$  geschrieben wird. Der *Critic* bildet denselben Zustand auf eine einzelne Zahl ab, nämlich auf seine Schätzung des Zustandswerts  $V^\pi(s)$ . Beide sehen also dieselbe Eingabe und unterscheiden sich allein in dem, was sie ausgeben.

Damit ist zugleich gesagt, welcher Art die Agenten dieser Arbeit sind: Angreifer wie Verteidiger sind Deep-RL-Agenten nach genau diesem Muster, jeder mit einem Actor- und einem Critic-Netz.

### 2.2.3. Aktualisierung der Policy

Bislang wurde gesagt, dass die Policy im Training verändert wird, aber nicht, wie. Dieser Abschnitt schließt die Lücke. Er beschreibt zunächst das Prinzip, dann die Schätzung des Vorteils und schließlich den Ablauf, in dem beides zusammenwirkt.

**Das Prinzip: ein Schritt entlang des Gradienten.** Die Policy ist nach Abschnitt 2.2.1 durch die Gewichte  $\theta$  des Actors bestimmt. Sie zu verändern heißt also, diese Zahlen zu verändern. Gesucht ist dafür eine Zielgröße  $L(\theta)$ , die genau dann groß ist, wenn die Policy gut ist. Der *Gradient*  $\nabla_\theta L(\theta)$  gibt für jedes einzelne Gewicht an, in welche Richtung und wie stark es diese Zielgröße beeinflusst. Ein Lernschritt verschiebt die Gewichte ein Stück in diese Richtung:

$$\theta \leftarrow \theta + \alpha \nabla_\theta L(\theta). \quad (2.4)$$

Der Faktor  $\alpha$  heißt *Lernrate* und legt fest, wie weit ein einzelner Schritt reicht. Er ist klein gewählt, sodass sich die Policy über viele kleine Schritte verändert statt über wenige große.

Damit bleibt die Frage, welche Zielgröße einzusetzen ist. Das *Policy-Gradienten-Theorem* gibt darauf die Antwort [8, Abschn. 13.2–13.5]; in der für Actor-Critic-Verfahren gebräuchlichen Schätzform lautet der Gradient [11, Gl. (1)]

$$\nabla_{\theta} L(\theta) = \hat{\mathbb{E}}_t \left[ \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]. \quad (2.5)$$

Darin ist  $\hat{A}_t$  der geschätzte Vorteil der tatsächlich gespielten Aktion. Der wahre Vorteil aus Gleichung (2.3) ist unbekannt, weil  $Q^{\pi}$  und  $V^{\pi}$  es sind; geschätzt wird er aus den tatsächlich erhaltenen Rewards und den Vorhersagen des Critic [12].

Für jede beobachtete Kombination aus Zustand und Aktion wird ermittelt, wie die Gewichte zu verändern wären, damit genau diese Aktion in genau diesem Zustand wahrscheinlicher wird, und dieser Beitrag wird mit dem Vorteil gewichtet. War der Vorteil positiv, verschiebt der Schritt die Gewichte so, dass die Aktion künftig wahrscheinlicher wird; war er negativ, kehrt sich das Vorzeichen um und sie wird unwahrscheinlicher. Genau darin besteht das Lernen.

Das Symbol  $\hat{\mathbb{E}}_t$  bezeichnet dabei einen Erwartungswert, der nicht exakt berechnet, sondern als Mittelwert über die gesammelten Beobachtungen geschätzt wird. Gemittelt wird über alle Paare aus Zustand und Aktion, die der Agent in den vorangegangenen Zeitschritten tatsächlich durchlaufen hat. Der Unterschied zum Erwartungswert  $\mathbb{E}$  aus Gleichung (2.2) ist also, dass dort über eine bekannte Verteilung summiert wird, während hier eine Stichprobe an ihre Stelle tritt; das Dach kennzeichnet die Schätzung.

**Der Ablauf.** Beides zusammen ergibt eine Schleife, die sich über das gesamte Training wiederholt und in Abbildung 2.1 dargestellt ist. Der Agent handelt zunächst eine feste Zahl von Schritten nach seiner aktuellen Policy und speichert dabei Zustände, Aktionen, Rewards und Critic-Schätzungen; dieser Vorrat heißt *Rollout*. Anschließend werden daraus die Vorteile bestimmt. Das setzt voraus, dass bereits feststeht, wie es weiterging, und ist deshalb erst nach dem Sammeln möglich. Erst dann folgen die Gewichts Anpassungen: Der Vorrat wird in kleinere Teile, sogenannte *Minibatches*, zerlegt und mehrfach durchlaufen, wobei jeder Teil einen Schritt nach Gleichung (2.4) auslöst. Ein vollständiger Durchgang durch den Vorrat heißt *Epoche*. Zugleich wird der Critic daraufhin angepasst, dass seine Schätzungen den tatsächlich beobachteten Returns näher kommen. Danach wird der Vorrat verworfen und die Schleife beginnt von vorn, denn er stammt nun von einer veralteten Policy.



Abbildung 2.1: Trainingsschleife eines Actor-Critic-Verfahrens. Gehandelt wird in Schritt 1, gelernt erst in Schritt 3; zwischen zwei Lernphasen bleibt die Policy unverändert.

Zwei Eigenschaften dieses Ablaufs sind später von Bedeutung. Erstens ist die Policy zwischen zwei Lernphasen konstant, der Agent lernt also nicht nach jedem einzelnen Schritt. Zweitens wird derselbe Vorrat mehrfach verwendet, was die Zahl der benötigten Umgebungsschritte senkt, zugleich aber die Frage aufwirft, wie weit sich die Policy dabei von derjenigen entfernen darf, mit der der Vorrat gesammelt wurde. Genau daran setzt das im folgenden Abschnitt beschriebene Verfahren an.

### 2.2.4. Proximal Policy Optimization

Advantage Actor-Critic (A2C) und PPO sind zwei verbreitete Verfahren, die beide der Actor-Critic-Architektur folgen und regelmäßig miteinander verglichen werden [13]. Sie unterscheiden sich darin, wie stark ein einzelner Lernschritt die Policy verändern darf.

A2C sammelt wenige Schritte Erfahrung und aktualisiert die Policy daraufhin unmittelbar [8, Abschn. 13.5]; die Schrittweite ist dabei nicht nach oben begrenzt. Fällt ein Vorteil einmal ungewöhnlich groß aus, kann eine einzelne Aktualisierung die Policy weit verschieben. Eine Aktion, die zufällig einmal sehr gut ausging, wird dann sofort zur Gewohnheit, auch wenn sie es nicht verdient hat. Schulman et al. benennen genau das als Schwäche des unbeschränkten Vorgehens, das zu „destructively large policy updates“ führen könne [11, Abschn. 2.1]. PPO setzt dort an und deckelt die Verschiebung.

Gemessen wird sie an einer einzigen Größe: Um welchen Faktor ändert sich die Wahrscheinlichkeit der Aktion, die tatsächlich gespielt wurde? Hielt die alte Policy sie mit 20 Prozent für richtig und die neue mit 24 Prozent, so beträgt dieser Faktor  $24/20 = 1,2$ .

Formal ist das [11, Abschn. 3]

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{alt}}}(a_t | s_t)}. \quad (2.6)$$

Dabei steht  $\theta$  für die Gewichte des Actor-Netzes, also für das, was beim Training verändert wird, und  $\theta_{\text{alt}}$  für ihren Stand vor der Aktualisierung. Im Zähler steht die neue Wahrscheinlichkeit der Aktion  $a_t$  im Zustand  $s_t$ , im Nenner die alte. Ein Wert von eins bedeutet keine Änderung, ein größerer Wert eine Erhöhung, ein kleinerer eine Verringerung.

Zu beachten ist, dass  $r_t(\theta)$  keine einzelne Zahl je Lernschritt ist. Der Index  $t$  steht für einen bestimmten Zeitschritt, und das Verhältnis wird für jedes im Rollout beobachtete Paar  $(s_t, a_t)$  gesondert gebildet. Erst die Mittelung über all diese Paare ergibt die Größe, die das Training maximiert.

Um diesen Faktor legt PPO ein festes Band. Erlaubt ist nur der Bereich zwischen  $1 - \varepsilon$  und  $1 + \varepsilon$ , bei dem von Schulman et al. vorgeschlagenen Wert  $\varepsilon = 0,2$  also zwischen 0,8 und 1,2, was einer Änderung von höchstens zwanzig Prozent entspricht [11, Abschn. 3]. Verlässt der Faktor dieses Band, wird er für die Berechnung auf den Randwert zurückgesetzt. Genau das leistet die Funktion `clip` in der Zielfunktion [11, Gl. (7)]

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right], \quad (2.7)$$

die anstelle der Grundform aus Gleichung (2.5) in Gleichung (2.4) eingesetzt wird. Sie enthält drei Bestandteile. Der geschätzte Vorteil  $\hat{A}_t$  gibt an, ob die Aktion besser oder schlechter ausfiel als erwartet. Der Faktor  $r_t(\theta)$  davor sorgt dafür, dass eine Erhöhung der Wahrscheinlichkeit bei positivem Vorteil den Ausdruck wachsen lässt: Gute Aktionen wahrscheinlicher zu machen, lohnt sich. Das  $\hat{\mathbb{E}}_t$  ist der bereits eingeführte geschätzte Erwartungswert, hier also der Mittelwert über alle Paare aus Zustand und Aktion des Rollouts.

Der Nutzen der Deckelung zeigt sich am Zahlenbeispiel. Eine Aktion sei mit  $\hat{A}_t = 5$  deutlich besser ausgefallen als erwartet, und die Aktualisierung wolle ihre Wahrscheinlichkeit von 20 auf 40 Prozent anheben, also  $r_t(\theta) = 2$ . Der linke Term im Minimum ergäbe dann  $2 \cdot 5 = 10$ , der rechte dagegen nur  $1,2 \cdot 5 = 6$ , weil der Faktor dort auf 1,2 gestutzt wird. Das Minimum wählt den kleineren Wert, hier also 6. Das ist genau der Wert, den auch eine Anhebung auf lediglich 24 Prozent erbracht hätte. Über das Band hinaus ist somit nichts mehr zu gewinnen, und der Anreiz zum großen Sprung entfällt. Bei negativem Vorteil greift die Begrenzung spiegelbildlich auf der Gegenseite und verhindert, dass die Wahrscheinlichkeit einer Aktion in einem Zug fast auf null gedrückt wird.

Damit ist zugleich die am Ende von Abschnitt 2.2.3 offene Frage beantwortet: Der Vorrat darf mehrfach verwendet werden, weil die Deckelung verhindert, dass sich die Policy dabei zu weit von derjenigen entfernt, mit der er gesammelt wurde.

### 2.2.5. Typische Schwierigkeiten

Fünf Eigenschaften des Ansatzes erschweren seinen Einsatz und werden im weiteren Verlauf der Arbeit wiederholt eine Rolle spielen.

**Hoher Bedarf an Erfahrung.** Aus dem Credit-Assignment-Problem aus Abschnitt 2.1.3 folgt unmittelbar, dass ein Agent sehr viele Schritte benötigt, bevor sich eine Bewertung festigt. Wo überwachte Verfahren aus einem einzelnen eingestuftem Beispiel unmittelbar lernen, muss ein RL-Agent denselben Zusammenhang über viele Episoden hinweg wiederholt beobachten. Ein Training in der Realität scheidet dadurch für viele Aufgaben aus.

**Der Reward bestimmt das Verhalten vollständig.** Ein Agent maximiert die Zielgröße, die ihm vorgegeben wird, und nicht die Absicht dahinter. Belohnt eine Reward-Funktion versehentlich einen Vorgang, der sich beliebig wiederholen lässt, ohne dass sich an der Lage etwas ändert, so lernt der Agent genau diese Wiederholung. Sein Verhalten ist dann formal korrekt und inhaltlich wertlos.

Damit ist die Reward-Hypothese aus Abschnitt 2.1.2 nicht widerlegt, aber ihre praktische Grenze benannt: Sie behauptet, dass sich ein Ziel als skalare Größe ausdrücken *lässt*, nicht, dass eine beliebige solche Größe das gemeinte Ziel auch trifft. Ob beides zusammenfällt, entscheidet der Entwurf der Reward-Funktion. Er ist deshalb kein Nebenaspekt, sondern bestimmt, was überhaupt gemessen wird.

**Mehrere lernende Agenten verletzen die Modellannahme.** Der MDP aus Abschnitt 2.1.4 setzt eine konstante Übergangsfunktion voraus. Handeln zwei Agenten im selben System und lernen beide gleichzeitig, so ist das nicht mehr gegeben: Aus Sicht des einen verändert sich die Umgebung schon dadurch, dass der andere seine Policy anpasst. Formal liegt dann kein MDP mehr vor, sondern ein *Markov-Spiel*, und die Zielgröße verschiebt sich fortlaufend. Aufgaben dieser Art sind Gegenstand des Multi-Agent Reinforcement Learning (MARL), das hier nicht weiter verfolgt wird: Der Aufbau dieser Arbeit vermeidet die Schwierigkeit, indem zu jedem Zeitpunkt nur einer der beiden Agenten lernt (Abschnitt 3.4).

**Unvollständige Beobachtbarkeit.** Die Markov-Annahme verlangt, dass der beobachtete Zustand alle für die Zukunft wesentliche Information enthält. In sicherheitsnahen Anwendungen ist das selten erfüllt: Ein Angreifer kennt nur die Teile des Netzes, die er bereits entdeckt hat. Ein Verteidiger sieht umgekehrt zwar den Zustand der eigenen Systeme und damit auch, welche davon kompromittiert sind, nicht aber den Wissensstand des Angreifers, also welche Knoten dieser bereits entdeckt und welche Zugangsdaten er erbeutet hat. Beide beobachten den Zustand somit nur teilweise; formal liegt dann ein *teilweise beobachtbarer* Entscheidungsprozess vor.

In der Praxis wird die Abweichung überwiegend hingenommen und weiterhin ein MDP unterstellt. Gangupantulu et al. benennen den Grund: Die teilweise beobachtbare Modellierung sei zwar realistischer, habe sich aber für große Netze nicht als skalierbar erwiesen und verlange, zahlreiche Wahrscheinlichkeitsverteilungen vorab zu modellieren [14, Abschn. II.C]. Die Verfahren werden also angewendet, ohne dass ihre Voraussetzungen streng erfüllt sind, und das ist eine bewusste Abwägung zwischen Wirklichkeitstreue und Handhabbarkeit.

**Stochastik der Ergebnisse.** Schließlich verläuft das Training selbst nicht deterministisch. Die Initialisierung der Netze, die Ziehung der Aktionen aus der Policy und die Umgebung selbst enthalten Zufall. Zwei Läufe desselben Aufbaus können deshalb unterschiedlich ausfallen, weshalb ein einzelner Lauf eine Stichprobe darstellt und kein Ergebnis.

## 2.3. Grundbegriffe der Netzwerksicherheit

Dieser Abschnitt wechselt vom Lernverfahren zum Gegenstand. Er beschreibt die Rollen von Angreifer und Verteidiger sowie den Ablauf eines Angriffs, legt fest, was in dieser Arbeit unter einer Topologie verstanden wird, und stellt die Segmentierungsmuster vor, aus denen die verglichenen Topologien hervorgehen.

### 2.3.1. Angreifer, Verteidiger und der Ablauf eines Angriffs

Ein Netz besteht in der hier verwendeten Abstraktion aus *Knoten*. Ein Knoten steht für einen einzelnen Rechner, also einen *Host*, der *Dienste* anbietet. Jeder Dienst lauscht auf einem *Port* und verlangt zur Anmeldung *Zugangsdaten*. Welche Verbindungen zwischen zwei Knoten überhaupt zustande kommen dürfen, regeln *Firewall-Regeln*: Eine Freigabe erlaubt Verkehr auf einem bestimmten Port, eine Sperre unterbindet ihn.

Der *Angreifer* verfolgt das Ziel, Kontrolle über möglichst wertvolle Systeme zu erlangen. Für diese wertvollsten Systeme hat sich der Begriff *Crown Jewels* eingebürgert. Gemeint sind diejenigen IT-Systeme, auf deren bestimmungsgemäßen Betrieb eine Organisation angewiesen ist; wie kritisch ein System ist, bemisst sich dabei an der Funktion, die es erfüllt, und an den Daten, die es vorhält [14, Abschn. II.A].

Entscheidend an dieser Definition ist, dass sie nicht an der Netzstruktur ansetzt, sondern an der Aufgabe der Organisation. Ermittelt werden Crown Jewels in einem eigenen Verfahren, der *Crown Jewels Analysis*, die die Geschäftsaufgabe mit den vorhandenen IT-Werten abgleicht und jedem System einen Relevanzwert zuweist; erst dadurch treten die betreffenden Systeme als die wichtigsten und wertvollsten hervor [14, Abschn. II.A]. Derselbe Rechner kann somit in einer Organisation ein Crown Jewel sein und in einer anderen ohne besondere Bedeutung.

Welches System diese Rolle in einem Netz einnimmt, ist damit keine Frage der Technik, sondern eine Festlegung, die vorab getroffen werden muss.

Üblicherweise wird nicht der Weg in das Netz hinein modelliert, sondern der Weg darin. Das *Assumed-Breach-Prinzip* setzt voraus, dass ein erster Host bereits kompromittiert ist, und fragt, was der Angreifer von dort aus erreicht. Die Annahme ist realistisch, weil sich der erste Zugang, etwa über eine Phishing-Nachricht, kaum zuverlässig verhindern lässt, und sie verschiebt den Blick auf das, was danach geschieht.

Dieses Weiterarbeiten von einem übernommenen Host zum nächsten heißt *laterale Bewegung* und ist als eigene Taktik definiert [15]. Sie setzt zweierlei voraus: einen Netzpfad zum Ziel, den die Firewall zulassen muss, und gültige Zugangsdaten für das dort laufende System. Beide Voraussetzungen sind voneinander unabhängig und müssen gemeinsam erfüllt sein. Ein Angreifer, der ein Passwort kennt, den betreffenden Rechner aber nicht erreicht, kommt nicht weiter. Umgekehrt gilt dasselbe.

Ein Host, der einem Angreifer als Zwischenstation für solche Schritte dient, heißt *Pivot Point*, ein Knoten, den jeder Weg zu einem Ziel passieren muss, *Chokepoint* [14, Abschn. II.A]. Ein Pivot Point ist nicht zwingend selbst wertvoll; wird sein Risiko allein an seiner eigenen Funktion bemessen, bleibt der Weg, den er eröffnet, unentdeckt.

Eine besondere Rolle nimmt dabei der *Domain-Controller* ein, also der Verzeichnisdienst, an dem sich die Systeme einer Windows-Domäne anmelden. Seine Domänendatenbank enthält die Passwort-Hashes sämtlicher Domänenkonten, und deren Auslesen ist als eigene Angriffstechnik dokumentiert [16]. Ein Hash ist dabei noch kein Passwort: Er lässt sich offline durch systematisches Ausprobieren einem Passwort zuordnen oder, je nach Anmeldeverfahren, unmittelbar an dessen Stelle vorlegen. Wer den Domain-Controller

übernimmt, verfügt damit über die Anmeldeinformationen des gesamten Netzes.

Der *Verteidiger* verfolgt das Gegenziel und verfügt dafür über Maßnahmen, die sich in zwei Gruppen einteilen lassen. *Wiederherstellende* Maßnahmen machen eine bereits eingetretene Kompromittierung rückgängig; das *Neuaufsetzen* eines Hosts, auch *Reimaging* genannt, entzieht dem Angreifer den Zugang zu diesem System. *Präventive* Maßnahmen verhindern einen Zugriff, bevor er stattfindet, etwa indem der eingehende Verkehr zu einem gefährdeten System gesperrt wird.

Weder wiederherstellende noch präventive Maßnahmen sind kostenlos, und darin liegt der eigentliche Zielkonflikt der Verteidigung. Ein neu aufgesetzter Host steht während des Vorgangs nicht zur Verfügung, ein gesperrter Dienst ist auch für berechtigte Nutzer nicht mehr erreichbar. Wie viel Ausfall hinnehmbar ist, wird als *Verfügbarkeit* gemessen und üblicherweise in einem Service Level Agreement (SLA) festgeschrieben. Ein Verteidiger, der das Netz vollständig abschaltet, hat zwar jeden Angriff unterbunden, aber seine Aufgabe verfehlt.

### 2.3.2. Netzwerktopologie als Sicherheitsarchitektur

Der Begriff Topologie wird uneinheitlich verwendet und bedarf daher einer Festlegung. In dieser Arbeit bezeichnet er die Sicherheitsarchitektur eines Netzes, also den Verlauf der Vertrauensgrenzen und die Frage, welche Systeme miteinander kommunizieren dürfen. Gemeint ist ausdrücklich nicht die physische Verkabelung: Graphentheoretische Muster wie Bus, Ring oder Stern beschreiben die Verdrahtung von Geräten und liegen auf einer anderen Abstraktionsebene.

Der Grund für diese Festlegung liegt darin, dass die Verkabelung nichts darüber aussagt, wer worauf zugreifen darf. Zwei Systeme am selben Switch können durch Zugriffsregeln vollständig voneinander getrennt sein, während zwei Systeme an entgegengesetzten Enden eines Gebäudes uneingeschränkt miteinander kommunizieren. Für einen Angreifer, der sich lateral bewegt, ist allein maßgeblich, welche Zugriffe zugelassen sind.

Die Zero-Trust-Architektur des National Institute of Standards and Technology (NIST) verlagert den Maßstab in dieselbe Richtung, von der Lage eines Systems auf die für es geltenden Regeln. Ihr zweiter Grundsatz lautet, dass die Position im Netz für sich genommen kein Vertrauen begründet; die zugehörigen Annahmen halten fest, dass das gesamte private Unternehmensnetz nicht als implizite Vertrauenszone gelten darf [7, Abschn. 2.1–2.2]. Über einen Zugriff entscheidet danach nicht die Zugehörigkeit zu einem Netzabschnitt, sondern eine ausdrückliche Regel. Damit ist die Ebene benannt, auf der diese Arbeit den Topologiebegriff ansiedelt: die der zugelassenen Kommunikati-

onsbeziehungen.

Das Mittel dazu ist die *Segmentierung*. Sie unterbindet nicht benötigte Kommunikationspfade und erschwert damit genau die laterale Bewegung aus Abschnitt 2.3.1: Ein Angreifer, der einen Host übernommen hat, erreicht von dort aus weniger Systeme, als er es in einem unsegmentierten Netz täte. Genau darin unterscheiden sich die im folgenden Abschnitt beschriebenen Muster.

### 2.3.3. Segmentierungsmuster

Die gebräuchlichen Muster lassen sich als Spektrum auffassen, das vom unsegmentierten Netz bis zur feingranularen Trennung einzelner Systeme reicht. Vier davon werden in dieser Arbeit verglichen; ihre konkrete Umsetzung beschreibt Abschnitt 3.3.1.

**Flach.** Ein flaches Netz kennt keine interne Segmentierung. Alle Hosts liegen in derselben Vertrauenszone, jeder erreicht jeden. Das ist der Ausgangszustand, den die Zero-Trust-Literatur als Risiko benennt: Ein einziger kompromittierter Host genügt, um sich frei im gesamten Netz zu bewegen [7].

**DMZ.** Die Demilitarized Zone (DMZ) führt eine grobe Zonentrennung ein. Von außen erreichbare Dienste stehen in einer entmilitarisierten Zone, die per Firewall vom internen Netz getrennt ist [17]. Der Perimeter ist damit befestigt, das Innere bleibt aber vergleichsweise offen. Für einen Angreifer bedeutet das eine Hürde beim Übertritt in das Intranet, danach jedoch vergleichsweise freie Bewegung.

**Hub-and-Spoke.** Das Hub-and-Spoke-Modell fällt aus der Reihe, weil es nicht mit Zonen arbeitet, sondern sämtliche Zugriffe über einen einzigen Knoten führt. Als Referenzarchitektur ist es dadurch gekennzeichnet, dass die Verbindungen nicht transitiv sind: Jeder Spoke ist mit dem Hub verbunden, kein Spoke mit einem anderen. Die Arbeitslasten der Spokes sind damit voneinander getrennt, während die zentrale Zugriffskontrolle im Hub liegt [18]. Der Hub ist damit ein Chokepoint im Sinne von Abschnitt 2.3.1, und zwar ein baulich erzwungener: Er entsteht nicht dadurch, dass ein Angreifer diesen Weg wählt, sondern dadurch, dass kein anderer existiert.

Der Begriff selbst stammt aus der Praxis der Netzarchitektur und ist in keinem Standard normiert, das zugrunde liegende Prinzip ist es sehr wohl: NIST verlangt für eine Zero-Trust-Architektur, dass Ressourcen ohne Passieren eines Kontrollpunkts nicht erreichbar sind [7, Abschn. 3.4.1]. Hub-and-Spoke ist die bauliche Umsetzung dieses Gedankens: ein einziger Punkt, an dem jeder Weg vorbeiführt.

Der Preis des Musters ist eine ausgeprägte Abhängigkeit, denn die Trennung der Spokes ruht vollständig auf diesem einen Punkt. Geht die Kontrolle über ihn verloren, entfällt sie ersatzlos. Realisiert wird der Hub deshalb üblicherweise nicht durch ein Endsystem, sondern durch ein eigens dafür vorgesehenes und entsprechend gehärtetes Gerät.

**Micro-Segmentierung.** Micro-Segmentierung treibt die Trennung auf die Ebene einzelner Systeme. Erlaubt ist nur, was explizit nötig ist, und jede Verbindung wird verifiziert. Das entspricht dem Zero-Trust-Prinzip [7]. Über die Erreichbarkeit hinaus betrifft der Grundsatz auch die Zugangsdaten: Zugriffe sollen je Sitzung geprüft werden, statt dauerhaft gültige Berechtigungen auf den Systemen vorzuhalten [7, Abschn. 2.1]. Beides zusammen verkürzt die Wege eines Angreifers nicht, sondern verlängert sie, weil er für jeden Schritt sowohl einen zugelassenen Pfad als auch die passenden Zugangsdaten benötigt.

## 2.4. Stand der Forschung

Dieser Abschnitt ordnet die Arbeit in die vorhandene Literatur ein. Er stellt die verfügbaren Simulationsumgebungen für Netzwerkangriffe vor, prüft, inwieweit sie einen lernenden Verteidiger vorsehen, und beschreibt mit MARL on die Erweiterung, auf der der eigene Aufbau beruht.

### 2.4.1. Simulationsumgebungen für Netzwerkangriffe

Ein RL-Agent lernt aus Erfahrung und benötigt dafür nach Abschnitt 2.2.5 sehr viele Episoden. In einem produktiven Firmennetz ist das weder praktikabel noch verantwortbar. Das Training findet deshalb in einer eigens gebauten Umgebung statt, die die für laterale Bewegung wesentlichen Mechanismen nachbildet, ohne ein reales Netz in allen Details zu kopieren. Einen Überblick über RL-Verfahren in der Cybersicherheit insgesamt geben Nguyen und Reddi [3].

Solche Umgebungen unterscheiden sich vor allem entlang einer Achse. Eine *Simulation* bildet das Netz als abstraktes Modell im Programm nach; ein Schritt kostet dann nur die Rechenzeit für eine Zustandsänderung. Eine *Emulation* betreibt stattdessen echte Systeme, etwa virtuelle Maschinen mit realer Netzanbindung, sodass ein Schritt tatsächlich ausgeführt wird. Die Emulation ist wirklichkeitsnäher, die Simulation um Größenordnungen schneller.

**CyberBattleSim.** CBS [4] ist eine Simulation von Microsoft. Sie bildet ein Netzwerk als gerichteten Graphen ab; jeder Knoten steht für einen Host mit Diensten, Schwachstellen und einer Firewall-Konfiguration. Der Angreifer beginnt nach dem Assumed-Breach-Prinzip auf einem bereits kompromittierten Einstiegsknoten. Eine Aktion ist etwa das Ausnutzen einer lokalen oder entfernten Schwachstelle oder der Verbindungsaufbau zu einem weiteren Knoten mit zuvor erbeuteten Zugangsdaten. Das Netz wird unmittelbar im Programmcode als Graph definiert, sodass sich Knoten, Zugangsdaten und Firewall-Regeln gezielt setzen lassen. Bereitgestellt wird die Umgebung über die *Gym-Schnittstelle* [4], die festlegt, wie eine Umgebung ihren Zustand zurückgibt und eine Aktion entgegennimmt. Auf derselben Schnittstelle setzt auch Stable-Baselines3 (SB3) auf [19], sodass sich beide ohne Anpassung verbinden lassen.

**CybORG.** CybORG [5] verfolgt einen zweigleisigen Ansatz und stellt sowohl einen Simulations- als auch einen Emulationsmodus bereit, wobei letzterer über eine Cloud-Infrastruktur betrieben wird. Das dokumentierte Szenario umfasst drei Hosts in zwei Subnetzen. Die Umgebung ist zugleich der Ausgangspunkt der CAGE-Wettbewerbe und damit vor allem auf vergleichbare, vorgegebene Aufgabenstellungen ausgelegt.

**CyGIL.** CyGIL [20] ist eine Emulation, die Aktionen auf tatsächlich betriebenen virtuellen Maschinen ausführt. Die dokumentierten Szenarien umfassen vier beziehungsweise neun Hosts. Der Preis der Wirklichkeitstreue ist die Laufzeit: Nach eigener Messung dauert eine Episode etwa 30 Sekunden im kleineren und 20 bis 30 Minuten im größeren Szenario, das Training eines einzelnen Agenten entsprechend Stunden bis Tage. Eine spätere Ausbaustufe ergänzt mit CyGIL-S einen Simulator, der aus Daten des Emulators erzeugt wird und dessen Betrieb damit voraussetzt [21].

**Angriffsgraphen als Gegenentwurf.** Ein zweiter Zweig der Literatur verzichtet ganz auf eine ausführbare Nachbildung des Netzes und fasst stattdessen einen vorab erzeugten *Angriffsgraphen* unmittelbar als MDP auf, dessen Übergangswahrscheinlichkeiten aus Bewertungskennzahlen bekannter Schwachstellen stammen [14, Abschn. II.C]. Der Ansatz skaliert auf Graphen mit mehreren tausend Knoten, setzt aber voraus, dass sich der Verlauf eines Angriffs vorab vollständig niederschreiben lässt, und kennt keinen zweiten, gleichzeitig handelnden Agenten.

Welche dieser Umgebungen für die vorliegende Arbeit gewählt wurde und aus welchen Gründen, behandelt Abschnitt 3.2.

### 2.4.2. Lernende Agenten auf beiden Seiten

Ein Merkmal zieht sich durch die genannten Arbeiten: Trainiert wird üblicherweise der Angreifer. Der Verteidiger ist, sofern überhaupt vorhanden, eine feste regelbasierte Komponente.

CybORG erprobt in seiner Ursprungsfassung einen einzigen angreifenden Agenten und führt die für einen lernenden Verteidiger nötigen Aktionen, Sensoren und Aktuatoren als ersten Punkt der künftigen Arbeiten auf [5]. CyGIL ist ausdrücklich nur für das Training von Angreifern erprobt [20]; auch die spätere Ausbaustufe sieht zwar begrifflich einen verteidigenden Agenten vor, trainiert in sämtlichen Experimenten aber weiterhin allein den angreifenden [21]. Dieselbe Einschränkung gilt für CBS selbst, wie die Autoren von CybORG in ihrem Vergleich feststellen: Ein fester Verteidiger sei möglich, dessen Training jedoch nicht vorgesehen [5].

Für eine Frage nach der *Lerneffizienz eines Verteidigers* ist das die entscheidende Lücke, denn sie lässt sich ohne einen zweiten lernenden Agenten nicht stellen. Zugleich führt ein solcher Aufbau in die in Abschnitt 2.2.5 genannte Schwierigkeit: Lernen beide Seiten gleichzeitig, liegt kein MDP mehr vor.

### 2.4.3. MARLon

MARLon [6] schließt genau diese Lücke für CBS. Es erweitert die Umgebung um einen zweiten, ebenfalls lernenden Agenten für die Verteidigung; beide handeln abwechselnd im selben Netz. Bereitgestellt werden dafür Implementierungen von A2C und PPO aus der Bibliothek SB3 [19], jeweils für Angreifer wie Verteidiger.

Der Verteidiger erhält dabei eine an den Angreifer gekoppelte Bewertung, und sein Aktionsraum umfasst Maßnahmen wie das Neuaufsetzen eines Knotens sowie das Sperren und Freigeben von Verkehr. Die genaue Ausgestaltung dieser Kopplung, die sich für die vorliegende Arbeit als folgenreich erwiesen hat, behandelt Abschnitt 3.1.1.

MARLon ist damit die Grundlage der vorliegenden Arbeit. Welche Anpassungen daran nötig waren, beschreibt Kapitel 3.

# 3

## Versuchsaufbau

Untersucht wird, ob und wie stark die Netzwerktopologie beeinflusst und wie schnell und wie gut ein Verteidigungsagent lernt. Die Topologie ist die unabhängige Variable, jedoch nicht der einzige variierte Faktor: Der Verteidiger trainiert stets gegen einen bestimmten eingefrorenen Angreifer. Ob und in welchem Umfang dessen Herkunft die Aufgabe beeinflusst, ist offen und selbst Teil der Untersuchung. Beide Größen werden deshalb systematisch gekreuzt, sodass sich ihre Beiträge trennen lassen (Abschnitt 3.4).

Alles Übrige muss konstant gehalten werden, sonst lassen sich beobachtete Unterschiede nicht mehr zuordnen. Bevor dieser Aufbau im Einzelnen beschrieben wird, ist jedoch ein Umweg nötig: Der hier verwendete Reward und Aktionsraum sind nicht die von MARLon übernommenen, sondern eine eigene Überarbeitung. Abschnitt 3.1 begründet, warum diese Überarbeitung nötig wurde.

### 3.1. Von MARLon zum eigenen Aufbau

Dieser Abschnitt beschreibt, wie aus dem übernommenen Rahmenwerk der in dieser Arbeit verwendete Aufbau wurde. Er benennt zunächst den geerbten Ausgangspunkt, schildert dann einen vollständigen Durchgang durch die Versuchsmatrix und die drei Fehler, die dieser sichtbar machte, und begründet daraus die anschließende Überarbeitung.

### 3.1.1. Der geerbte Ausgangspunkt

MARLlon [6] stellt die Trainingsumgebung für beide Agenten bereit, wie in Abschnitt 2.4.3 beschrieben. Für den Verteidiger legt die Originalarbeit dabei eine bestimmte Reward-Struktur fest: Sein Reward ergibt sich durch Negation des jeweils letzten Angreifer-Rewards, ergänzt um eine große Strafe bei Unterschreiten der Netzverfügbarkeit. Die Autoren begründen diese Wahl damit, dass der Verteidiger den Fortschritt des Angreifers minimieren solle, und halten als unmittelbare Folge fest, dass sein maximal erreichbarer Reward damit bei null liegt und mit jedem Fortschritt des Angreifers weiter sinkt [6]. Prävention ist in dieser Formel nicht vorgesehen, und einen Anreiz zur Rückeroberung enthält sie ebenso wenig: CBS schreibt den Knotenwert nur bei der ersten Übernahme gut, sodass ein späterer Besitzwechsel desselben Knotens den Reward beider Seiten unverändert lässt. Gefördert wird damit gerade nicht das Verhalten, auf das es in dieser Arbeit ankommt: den Angreifer am Vordringen zu hindern und bereits verlorene Systeme zurückzugewinnen.

Für den Aktionsraum des Verteidigers sieht MARLlon fünf Aktionsarten vor:

1. einen Knoten neu aufsetzen,
2. Verkehr sperren,
3. Verkehr freigeben,
4. einen Dienst stoppen,
5. einen Dienst wieder starten.

Port und Richtung sind dabei zusätzliche Parameter, die der Agent selbst wählt. Für den Angreifer identifiziert die Originalarbeit den Umgang mit ungültigen Aktionen als eigenes Problem. Gültig ist eine Aktion, wenn der Angreifer sie im aktuellen Zustand überhaupt versuchen kann: Der Ausgangsknoten muss übernommen, das Ziel entdeckt und die eingesetzten Zugangsdaten müssen bereits erbeutet sein. Ob der Versuch gelingt, entscheidet sich unabhängig davon. Ein Verbindungsversuch auf einen Port, auf dem das Ziel keinen Dienst anbietet, ist eine gültige Aktion, die lediglich folgenlos bleibt. Eine Bestrafung ungültiger Aktionen würde dazu führen, dass der Agent riskante, aber notwendige Aktionen ganz meidet; die Autoren entscheiden sich stattdessen für Reward 0 bei ungültigen Aktionen, kombiniert mit einer zufälligen Ersatzwahl. Diese Entscheidung ist im Code der Umgebung selbst verankert, nicht nur im Trainingsablauf.

### 3.1.2. Eigene Vorarbeiten und der Referenzlauf

Dass der Verteidiger damit weder für verhinderte noch für rückgängig gemachte Übernahmen etwas erhält, war der Anlass, seinen Reward in mehreren Schritten zu überarbeiten. Ein zwischenzeitlicher Entwurf war ereignisbasiert und vergütete jedes Neuaufsetzen eines kompromittierten Knotens mit einem festen Betrag. Damit belohnte er einen beliebig wiederholbaren Vorgang, und zwar einseitig: CBS schreibt einen Knotenwert nur bei der ersten Übernahme gut, sodass dem Angreifer eine Rückerobierung nichts einbrachte, während der Verteidiger für jedes Neuaufsetzen desselben Knotens erneut vergütet wurde. Ein einzelner Knoten wechselte dadurch fortwährend zwischen kompromittiert und neu aufgesetzt, ohne dass sich die Lage im Netz über einen solchen Zyklus hinweg verändert hätte, und der Verteidiger wurde für jede Runde erneut belohnt. Das ist genau der in Abschnitt 2.2.5 beschriebene Fall, in dem ein Agent formal korrekt maximiert und inhaltlich nichts erreicht. Daraufhin wurde zur Kopplung aus Abschnitt 3.1.1 zurückgekehrt, die MARL<sub>on</sub> ursprünglich vorsieht.

Über diese Kopplung hinaus umfasste der Aufbau zu diesem Zeitpunkt vier eigene Ergänzungen:

1. vier Vergleichstopologien anstelle des einen von MARL<sub>on</sub> vorgesehenen festen Netzes (Abschnitt 3.3.1),
2. ein Invalid-Action-Masking für den Angreifer nach Huang und Ontañón [22], das MARL<sub>ons</sub> Zufallersatzverfahren ablöst (Abschnitt 3.5),
3. ein Haltebonus je gehaltenem Knoten und Zeitschritt (Abschnitt 3.6),
4. ein Abzug in Höhe des Knotenwerts, wenn ein gehaltener Knoten neu aufgesetzt wird (Abschnitt 3.6).

Die zweite Ergänzung verdient eine Erläuterung: Statt eine ungültige Aktion zu raten und auf eine zufällige gültige auszuweichen, wählt der Angreifer ausschließlich aus der im jeweiligen Zustand gültigen Menge. Wirkungslose Aktionen entfielen dadurch praktisch vollständig, und der Angreifer erreichte das Crown Jewel zuverlässig, statt wie zuvor nur in Ausnahmefällen. Erst das machte ihn als Gegner brauchbar, denn die Lerneffizienz eines Verteidigers lässt sich nur gegen einen Angreifer messen, der überhaupt etwas ausrichtet.

Von der übernommenen Reward-Struktur blieben damit zwei Bestandteile unberührt, und gerade sie bestimmen die erreichbare Obergrenze: die Kopplung des Verteidiger-Rewards an den negierten Angreifer-Reward und die Begrenzung des Angreifer-Schrittwerts nach unten auf null. Letztere ist in CBS selbst verankert und wurde bewusst beibehalten;

Abschnitt 3.6 legt dar, weshalb. Die eigenen Ergänzungen verschieben den Angreifer-Reward, heben diese Obergrenze aber nicht auf.

Auf diesem Stand entstand der im Folgenden als *Referenzlauf* bezeichnete Durchgang durch die vollständige Versuchsmatrix. Er ist der Vergleichsmaßstab dieser Arbeit: Sein Ergebnis begründet die anschließend vorgenommenen Änderungen am Aufbau, und an ihm wird deren Wirkung gemessen. Die Schwächen, die die folgenden Abschnitte beschreiben, wurden erst durch ihn sichtbar und waren zum Zeitpunkt seiner Durchführung nicht bekannt. Darin unterscheidet er sich von den übrigen Zwischenständen des Aufbaus, die nur dort zur Sprache kommen, wo sie eine Entwurfsentscheidung erklären. Die Matrix kreuzt die vier Verteidigungstopologien mit sechs Angreifern: fünf davon wurden auf jeweils genau einer Topologie trainiert, ein sechster nacheinander auf allen. Wegen der in Abschnitt 2.2.5 genannten Stochastik wird jede der 24 Kombinationen fünfmal wiederholt, jeweils mit einem anderen Startwert für die Zufallsgeneratoren, einem sogenannten *Seed*. Das ergibt 120 Trainingsläufe.

Als Budget waren dabei je 500.000 Schritte vorgesehen, ausgeschöpft wurde es aber nicht durchgängig: Das Konvergenzkriterium aus Abschnitt 3.7 diente im Referenzlauf noch als Abbruchbedingung und beendete das Training, sobald es erfüllt war. Für die später berichteten Läufe wurde davon abgerückt; Abschnitt 3.7 begründet das. Die vier Ergänzungen sind auch im überarbeiteten Aufbau enthalten und werden in den folgenden Abschnitten beschrieben; was erst nach dem Referenzlauf hinzukam, ist Gegenstand von Abschnitt 3.1.4.

Das Ergebnis bestätigte, was Abschnitt 3.1.1 bereits aus der Formel ableitet: In allen 8642 protokollierten Trainingsepisoden war der Verteidiger-Reward negativ, der höchste beobachtete Einzelwert betrug  $-14,0$ . Das ist keine Fehlfunktion, sondern die Obergrenze null, empirisch bestätigt. Für eine Metrik, die ursprünglich vorgesehen war, hat das unmittelbare Folgen: Break-even, definiert als die Episode, ab der der geglättete Reward nicht mehr negativ ist, kann unter dieser Kopplung strukturell nie eintreten und entfällt deshalb vollständig (Abschnitt 3.8).

### 3.1.3. Drei Fehler in der Umsetzung

Über die reine Reward-Formel hinaus fanden sich drei Stellen, an denen die Sperre als Mittel der Prävention beeinträchtigt war. Sämtliche drei wurden erst nach dem Referenzlauf behoben und waren während seiner gesamten Dauer wirksam.

Der erste betrifft die Liste der sperrbaren Ports und ist der eigenen Arbeit zuzurechnen. MARL<sub>on</sub> hält sie fest verdrahtet im Code, passend zu der Umgebung, für die

das Rahmenwerk entworfen wurde; dort ist sie zutreffend. Mit den in Abschnitt 3.3.1 beschriebenen Topologien kamen jedoch andere Dienste ins Netz, und die Liste wurde dabei nicht mitgeführt. Von den sechs tatsächlich vorkommenden Ports fehlten ihr zwei, während sie zwei weitere enthielt, die in keiner der Topologien auftreten. Der Datenbank-, der Backup- und der Dateiserver, darunter das Crown Jewel, ließen sich dadurch nie ansprechen. Für die späteren Läufe wird die Liste nicht mehr fest vorgegeben, sondern aus der Umgebung abgeleitet und folgt damit jeder Topologie.

Die beiden folgenden Punkte stammen dagegen aus dem unveränderten MARLlon-Code. Sie blieben im Referenzlauf ohne Folgen und wurden erst durch die spätere Überarbeitung zum Hindernis; der Vollständigkeit halber gehören sie dennoch hierher.

Die Methode zum Sperren von Verkehr entfernte lediglich eine vorhandene Freigabe-Regel, statt eine ausdrückliche Sperr-Regel zu setzen. Unterbunden war der Verkehr danach tatsächlich, weil CBS einen Port ohne Regel als gesperrt behandelt. Ein so gesperrter Port sah anschließend jedoch genauso aus wie einer, für den nie eine Freigabe bestand, und das ist in den hier verwendeten Topologien der bauliche Normalzustand: Sie arbeiten mit Freigabelisten, sodass die meisten Ports ohnehin keine Regel tragen. Ein Eingriff des Verteidigers war damit nicht mehr von der Ausgangslage zu unterscheiden. Für den Angreifer änderte das nichts, denn er stieß in beiden Fällen auf eine geschlossene Tür. Sobald jedoch der Reward eine bewusst gesetzte Sperre erkennen und belohnen soll, wie in Abschnitt 3.6 beschrieben, ist die Unterscheidung unentbehrlich. Die zugehörige Methode zum Freigeben von Verkehr war zudem fehlerhaft: Beide Zweige der Fallunterscheidung schrieben in die eingehende Regelliste, sodass sich eine ausgehende Sperre nicht wieder aufheben ließ.

Schließlich zählte die Verfügbarkeitsberechnung nur, ob ein Dienst läuft, nicht aber, ob sein Port gesperrt ist. Sperren kostete damit keine Verfügbarkeit. Im Referenzlauf fiel auch das nicht ins Gewicht, weil eine Sperre dort ohnehin nichts einbrachte. Wird sie dagegen belohnt, entsteht daraus ein Fehlanreiz: Der Verteidiger könnte das Netz vollständig abriegeln, ohne dafür je das SLA zu verletzen.

Für den Referenzlauf bleibt damit vor allem der erste Punkt. Er allein erklärt noch nicht, warum der Verteidiger ausschließlich das Neuaufsetzen lernte, denn die entscheidende Ursache liegt in der Reward-Formel: Ein verhinderte Zugriff bringt dem Angreifer null ein und dem Verteidiger über die Kopplung ebenfalls null. Die fehlerhafte Portliste kam hinzu und schloss die wertvollsten Systeme von vornherein aus. Und die einzige verbliebene präventive Handlung wurde sogar bestraft: Das Stoppen eines Dienstes auf einem nicht kompromittierten Knoten kostete zehn Punkte, und da sich nicht vorhersehen lässt, welchen Knoten der Angreifer als Nächstes angreift, traf diese Strafe den Verteidiger

regelmäßig. Von den präventiven Mitteln blieb ihm keines, das sich gelohnt hätte.

#### 3.1.4. Konsequenz für diese Arbeit

Aus diesem Befund folgt nicht, dass sich mit der Fragestellung nach dem Einfluss der Topologie nichts anfangen ließe. Der Referenzlauf bleibt verwertbar, allerdings für eine engere Aussage, als ursprünglich beabsichtigt: Er beschreibt einen Verteidiger, der ausschließlich reaktiv handeln kann, also Wiederherstellung statt Prävention. Reimaging ist selbst eine Form der Abwehr, denn es entzieht dem Angreifer den Zugang und macht dessen Gewinn rückgängig, auch wenn es keinen Angriff verhindert.

Für die in dieser Arbeit berichteten Ergebnisse wurde der Aufbau aus zwei Gründen grundlegend überarbeitet: Erstens lassen sich die drei in Abschnitt 3.1.3 genannten Punkte als Implementierungsfehler und nicht als Entwurfsentscheidungen einordnen; sie zu beheben ist unabhängig von der Forschungsfrage geboten. Zweitens lässt erst ein Verteidiger, der Prävention tatsächlich ausführen und dafür belohnt werden kann, die Frage nach dem Einfluss der Topologie auf einen *präventiven* Verteidiger überhaupt zu; unter dem Referenzlauf wäre jede Topologie mit derselben Unmöglichkeit konfrontiert gewesen. Die folgenden Abschnitte beschreiben den überarbeiteten Aufbau, wie er den in Kapitel 4 berichteten Läufen zugrunde liegt. Wo eine Änderung das Ergebnis unmittelbar beeinflusst, wird auf den Referenzlauf als Vergleichspunkt zurückverwiesen.

## 3.2. Wahl der Simulationsumgebung und des Lernverfahrens

Zwei Entscheidungen liegen dem Versuch zugrunde, bevor sich Fragen seiner Ausgestaltung stellen: in welcher Simulationsumgebung er stattfindet und mit welchem Lernverfahren die Agenten trainiert werden. Dieser Abschnitt begründet beide.

### 3.2.1. Die Simulationsumgebung

Von den in Abschnitt 2.4.1 vorgestellten Umgebungen fiel die Wahl auf CBS in Verbindung mit MARLon. Maßgeblich dafür sind drei Anforderungen, die sich unmittelbar aus der Fragestellung ergeben: eine frei definierbare Netzwerktopologie, ein lernender Agent auf beiden Seiten und ein Durchsatz, der eine Versuchsmatrix dieses Umfangs erlaubt. Die beiden Alternativen scheitern jeweils an mehreren dieser Anforderungen.

Zu beachten ist dabei, dass alle drei Umgebungen fortlaufend weiterentwickelt werden. Der folgende Vergleich bezieht sich auf den Stand, den die jeweiligen Veröffentlichungen dokumentieren; spätere Erweiterungen sind, soweit sie belegt sind, angeführt.

Die erste Anforderung ist die entscheidende, weil sie den Untersuchungsgegenstand selbst betrifft. Dass CBS das Netz unmittelbar im Programmcode als Graph definiert (Abschnitt 2.4.1), ermöglicht erst den in Abschnitt 3.3 beschriebenen Aufbau, in dem sich die vier Topologien allein in ihrer Segmentierung unterscheiden, während Knotenzahl und Knotenwerte konstant bleiben. Die dokumentierten Szenarien der beiden anderen Umgebungen sind demgegenüber klein und auf andere Zwecke zugeschnitten. Eine Topologie zu verändern bedeutet dort, die zugrunde liegende Infrastruktur umzubauen.

Die zweite Anforderung betrifft die Gegenseite. Der Verteidiger soll sich gegen ein Angriffsverhalten behaupten, das seinerseits aus Erfahrung entstanden ist und nicht aus einer festen Regel. Wie Abschnitt 2.4.2 zeigt, ist der lernende Verteidiger in beiden Alternativen jedoch nicht umgesetzt. Genau diese Lücke schließt MARLon, indem es für beide Rollen RL-Agenten bereitstellt.

Die dritte Anforderung ist eine praktische. Die in dieser Arbeit ausgewerteten 120 Trainingsläufe umfassen zusammen rund 60 Millionen Simulationsschritte. Rechnet man die für CyGIL dokumentierten Episodendauern (Abschnitt 2.4.1) auf diesen Umfang hoch, liegt das Ergebnis jenseits des für eine Abschlussarbeit verfügbaren Zeitrahmens; der dort ebenfalls genannte Simulator CyGIL-S nimmt diese Hürde nicht, weil er den Betrieb des Emulators voraussetzt. CybORG bietet zwar einen reinen Simulationsmodus, betreibt seinen Emulator jedoch über eine Cloud-Infrastruktur [5], was neben dem Zeit- auch einen Kostenaufwand nach sich zieht.

Mit der Wahl wird zugleich eine Einschränkung übernommen: Als reine Simulation bildet CBS keine echten Systeme ab, sodass die Übertragbarkeit der Ergebnisse auf reale Netze offenbleibt. Die höhere Wirklichkeitstreue der emulationsbasierten Ansätze wäre hier gegen die Zahl der durchführbaren Läufe einzutauschen gewesen. Da die Fragestellung auf den *Vergleich* von Topologien unter sonst gleichen Bedingungen zielt und nicht auf absolute Aussagen über konkrete Netze, wiegt die Zahl der Läufe schwerer. Abschnitt 4.9 greift diesen Punkt erneut auf.

### 3.2.2. Das Lernverfahren

MARLon stellt mit A2C und PPO zwei Verfahren aus SB3 bereit (Abschnitt 2.4.3). Für beide existieren vollständige, parallel gepflegte Implementierungen für Angreifer wie Verteidiger, sodass die Wahl offensteht. Sie fiel auf PPO.

Die in Abschnitt 2.2.4 beschriebene Deckelung erzeugt gleichmäßigere Lernkurven, da einzelne Ausreißer den bisherigen Lernfortschritt nicht in einem Schritt verwerfen können. Betroffen ist davon nicht allein die Darstellung, sondern die Messbarkeit: Zu den erhobenen Größen gehört die Lerneffizienz und damit der Zeitpunkt, zu dem ein Lauf als konvergiert gilt, und das Konvergenzkriterium in Abschnitt 3.7 entscheidet anhand der Streuung und des Trends der Episoden-Rewards innerhalb eines Fensters. Ein Verfahren, dessen Kurve wiederholt springt, würde dieses Kriterium unabhängig von der Topologie später auslösen und die Vergleichbarkeit zwischen den Topologien beeinträchtigen. Die stabilere Kurve ist somit Voraussetzung dafür, dass die gemessenen Unterschiede der Topologie zugerechnet werden können.

Für den Diskontfaktor aus Abschnitt 2.1.4 gilt in dieser Arbeit  $\gamma = 0,99$ ; ein Reward verliert pro Zeitschritt somit ein Prozent an Gewicht. Der Wert wurde nicht eigens gewählt, sondern entspricht dem Standardwert von SB3 [19] und dem in der PPO-Literatur gebräuchlichen Ansatz [11]. Die übrigen Hyperparameter sind in Abschnitt A.2 aufgeführt.

### 3.3. Umgebung und Kontrolle der Variablen

Jede Topologie besteht aus denselben zehn Knoten mit denselben Werten. Der Einstiegsknoten `WebServer` trägt den Wert 0, die Arbeitsplatzrechner je 10, der Mail- und der Applikationsserver je 15, der Dateiserver 20, der Backup-Server 25, der Domain Controller 30 und der Datenbankserver als Crown Jewel 100. Die Summe dieser Knotenwerte beträgt in jeder Topologie exakt 235 Punkte.

Das Crown Jewel im Sinne von Abschnitt 2.3.1 ist hier der Datenbankserver. Bei ihm laufen die Daten zusammen, auf denen der Geschäftsbetrieb beruht, und er vereint sie zugleich an einer einzigen Stelle; gerade diese Bündelung macht ihn zum lohnenden Ziel, denn ein einzelner erfolgreicher Zugriff genügt, um den gesamten Datenbestand zu entwenden. Alle übrigen Systeme sind für einen Angreifer demgegenüber vor allem als Zwischenstationen von Interesse. Angenommen wird damit ein Angriff, der auf das Entwenden von Daten zielt, die *Exfiltration*; bei einem anderen Angreiferziel fiele die Wahl möglicherweise anders aus.

Dass jedem System überhaupt ein einzelner Zahlenwert zugeordnet wird, ist keine Eigenheit dieser Simulation, sondern folgt dem Vorgehen der in Abschnitt 2.3.1 beschriebenen Crown Jewels Analysis: Dort ist die Zuweisung eines Relevanzwerts je IT-System der Kern des Verfahrens, und aus diesen Werten ergibt sich anschließend, welche

Systeme als besonders schützenswert gelten [14, Abschn. II.A]. Die hier vergebenen Werte bilden diese Rangfolge nach: Die Produktionsdatenbank trägt mit 100 Punkten ein Vielfaches jedes anderen Systems, der Verzeichnisdienst folgt mit 30 Punkten als wichtigster Wegbereiter, und die Arbeitsplatzrechner bilden mit je 10 Punkten das untere Ende. Der Einstiegsknoten trägt den Wert 0, weil sein Besitz dem Angreifer nach dem Assumed-Breach-Prinzip von Beginn an zusteht und deshalb keinen Fortschritt darstellt.

Diese Gleichheit ist beabsichtigt: Da Knotenzahl und Knotenwerte identisch bleiben, unterscheiden sich die Topologien ausschließlich darin, welche Knoten miteinander kommunizieren dürfen und wo die Zugangsdaten liegen. Ein Unterschied im Lernverhalten lässt sich somit auf die Struktur zurückführen und nicht auf ein lukrativeres Ziel.

Die Knotenwerte sind allerdings nicht der einzige Beitrag zum Reward. Abschnitt 3.6 führt die vollständige Bewertung beider Agenten auf und zeigt, dass die erreichbaren Reward-Niveaus zwischen den Topologien erheblich auseinanderliegen.

Als Verteidigungsumgebung dienen die vier in Abschnitt 2.3.3 qualitativ beschriebenen Muster, umgesetzt als *flat*, *hub\_and\_spoke*, *dmz* und *micro\_segmented*. Hinzu kommt *chain*: Diese Topologie dient ausschließlich dem Angreifertraining und wird nicht verteidigt.

### 3.3.1. Die Topologien als Zehn-Knoten-Modell

Angestrebt ist ein Netz, das ein Leser als Unternehmensnetz wiedererkennt: ein von außen erreichbarer Webserver, dahinter Anwendungs-, Mail- und Dateidienste, ein Verzeichnisdienst und eine Produktionsdatenbank als wertvollstes System. Die Rollen und ihre Beziehungen sind so gewählt, dass die Wege zwischen ihnen fachlich begründbar bleiben und nicht bloß eine abstrakte Graphenstruktur abbilden.

Diesem Anspruch steht die Vergleichbarkeit gegenüber, und wo beide in Konflikt geraten, hat die Vergleichbarkeit Vorrang. Weil dasselbe Inventar aus zehn Knoten in allen Topologien gelten muss, lassen sich keine Komponenten ergänzen, die nur in einer davon vorkämen. Am deutlichsten zeigt sich das beim Hub-and-Spoke-Modell: Wie in Abschnitt 2.3.3 beschrieben, wäre der Hub in einem realen Netz ein eigens für diese Aufgabe vorgesehenes und entsprechend gehärtetes Gerät, das ein Angreifer nicht als Sprungbrett nutzt. Ein solcher Knotentyp existiert im Modell nicht, weshalb die Rolle einem der vorhandenen Systeme zufällt, nämlich dem Applikationsserver; die Wahl begründet die Beschreibung der Topologie weiter unten. Diese Vereinfachung ist bewusst in Kauf genommen; ihre Folgen für die Aussagekraft greift Abschnitt 4.9 auf.

Zum Verständnis der folgenden Abbildungen sind zwei Kantenarten zu unterscheiden, weil das Modell beides bewusst trennt. Eine durchgezogene Kante bedeutet *Erreichbarkeit*: Die Firewall lässt Verkehr zum Zielknoten zu. Eine gestrichelte Kante bedeutet *Besitz von Zugangsdaten*: Auf dem Ausgangsknoten sind die Anmeldedaten des Zielknotens hinterlegt.

Beide Kantenarten sind dabei gerichtet und beschreiben ausschließlich, wohin sich ein Angreifer bewegen kann. Sie bilden nicht den gesamten Verkehr eines Netzes ab. In einem realen Unternehmensnetz greift ein Arbeitsplatzrechner selbstverständlich auf Server zu; solche Verbindungen sind hier nur dort verzeichnet, wo sie einem Angreifer einen Weg eröffnen. Ein Knoten ohne eingehende Kante ist deshalb nicht vom Netz abgeschnitten, sondern für den Angreifer schlicht kein mögliches nächstes Ziel.

Beide Bedingungen müssen erfüllt sein, damit ein Knoten übernommen werden kann, sie müssen aber nicht von derselben Stelle herrühren. Erbeutete Zugangsdaten bleiben dem Angreifer erhalten und lassen sich auch von einem anderen Knoten aus einsetzen, sofern von dort ein Pfad zum Ziel führt. Erreichbarkeit und Berechtigung können also aus zwei verschiedenen Quellen stammen. Welche Verbindungsversuche der Angreifer darüber hinaus überhaupt unternehmen kann, schränkt Abschnitt 3.5.1.1 weiter ein.

Eine Besonderheit gilt für den Domain-Controller: Er hält in jeder Topologie die Zugangsdaten sämtlicher Knoten und bildet damit die in Abschnitt 2.3.1 beschriebene Rolle eines Active-Directory-Servers nach. Wer ihn übernimmt, verfügt über die Zugangsdaten zum gesamten Netz; ob er die betreffenden Systeme auch erreicht, entscheidet weiterhin die Topologie. Die acht zugehörigen Kanten sind in den Abbildungen aus Gründen der Lesbarkeit durch eine Beschriftung ersetzt.

—→ Erreichbarkeit    - - - -> Zugangsdaten    Einstieg    Crown Jewel

**flat.** Ein flaches Netz kennt keine interne Segmentierung; jeder Knoten erreicht jeden anderen (Abbildung 3.1). Die fehlende Segmentierung betrifft dabei ausschließlich die Erreichbarkeit. Die Zugangsdaten sind auch hier nicht beliebig verteilt, sondern folgen der tatsächlichen Nutzung der Systeme. Alle drei Arbeitsplätze binden dieselbe Dateiablage ein, denn genau dafür existiert ein Dateiserver. Der Webserver spricht als Darstellungsschicht nur mit seinem Anwendungs-Backend, und dieses hält seinerseits einen fest konfigurierten Datenbankzugang sowie die Anmeldedaten für den Verzeichnisdienst. Mail- und Dateiserver melden sich ebenfalls am Verzeichnisdienst an, wie es für Dienstknoten üblich ist.

Bewusst nicht abgebildet ist der Fall, dass sich ein privilegiertes Konto an einem Endgerät anmeldet und dort verwertbare Anmeldedaten hinterlässt. Kein Arbeitsplatz hält

Zugangsdaten des Domain-Controllers. Die Modellannahme lautet, dass Administration von einem gesonderten Gerät außerhalb dieses Inventars erfolgt. Der Weg zum Verzeichnisdienst führt daher ausschließlich über die Dienstkonten der Server.

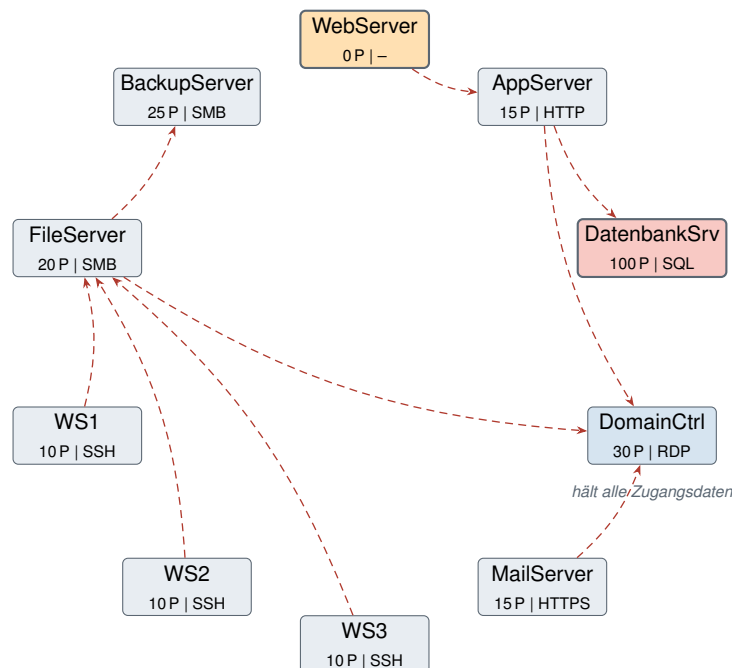


Abbildung 3.1: Topologie *flat*. Die Erreichbarkeit ist vollständig, jeder Knoten erreicht jeden anderen; die 90 gerichteten Kanten sind nicht eingezeichnet. Dargestellt sind daher nur die Zugangsdaten.

**dmz.** Die Zuordnung folgt den drei Zonen (Abbildung 3.2). In der entmilitarisierten Zone liegen der Webserver sowie die beiden Dienste, die er benötigt: der Mailserver für ausgehende Benachrichtigungen und der Applikationsserver als Backend. Beide melden sich am Domain-Controller an und überschreiten damit als einzige die Zonengrenze ins Intranet. Der Applikationsserver hält zusätzlich die Zugangsdaten der Datenbank, wie es einer fest konfigurierten Datenbankverbindung entspricht. Im Intranet selbst greift keine weitere Trennung mehr: Der Domain-Controller erreicht alle Endsysteme, und der Dateiserver sichert auf den Backup-Server. Genau dieses Gefälle ist gemeint, wenn vom befestigten Perimeter bei weichem Inneren die Rede ist.

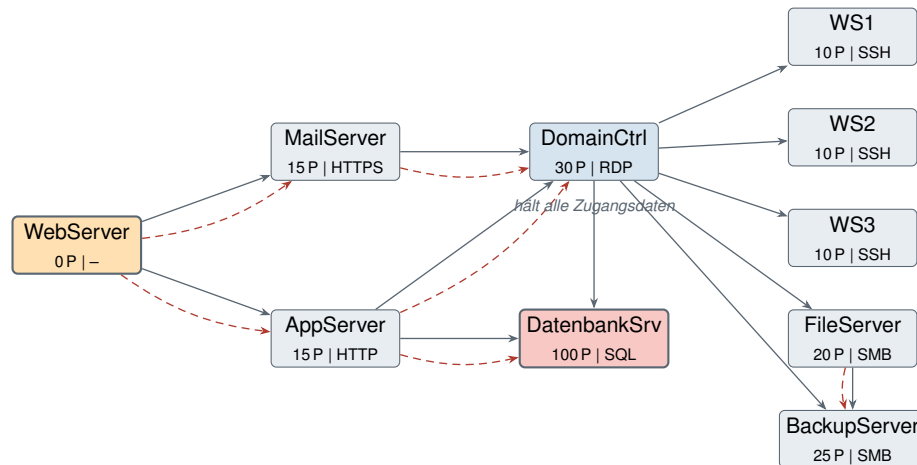


Abbildung 3.2: Topologie *dmz*. Der Einstiegsknoten erreicht nur die beiden Dienste der entmilitarisierten Zone; erst über diese führt der Weg ins Intranet.

**hub\_and\_spoke.** Die Rolle des Hubs übernimmt der Applikationsserver. Nach dem Domain-Controller unterhält er die meisten fachlich begründeten Verbindungen über Bereichsgrenzen hinweg: Er liest aus der Datenbank, meldet sich am Verzeichnisdienst an und greift auf die Dateiablage zu. Zu den übrigen Spokes besteht keine solche fachliche Beziehung; die Arbeitsplätze etwa nutzen die Anwendung über den Webserver und nicht unmittelbar. Das Muster verlangt eine solche Beziehung aber auch nicht: Die Kanten zwischen Hub und Spokes folgen aus der Transitrolle selbst, die sämtlichen Verkehr über diesen einen Punkt führt, und fallen bei jeder Wahl des Hubs gleichermaßen an.

Dass nicht der Domain-Controller zum Hub wurde, obwohl er noch mehr Bereiche verbindet, ist eine bewusste Entscheidung. Dass der Hub überhaupt ein übernehmbares System sein muss, ist bereits das oben beschriebene Zugeständnis an das feste Inventar. Der Domain-Controller hält darüber hinaus die Zugangsdaten sämtlicher Knoten, und beides zusammen machte seine Übernahme zum Verlust des gesamten Netzes: Der Angreifer erhielte mit einem einzigen Knoten Reichweite und Zugriff auf alles, und für sechs der neun übrigen Knoten verkürzte sich der Angriffsweg um einen Schritt, weil ihre Zugangsdaten dann bereits am Engpass lägen. Die Trennung von Reichweite und Zugriff, die dieses Muster gerade auszeichnet, wäre aufgehoben.

Allein der Hub besitzt ausgehende Firewall-Regeln, alle übrigen Knoten haben keine (Abbildung 3.3).

Für den Angreifer bedeutet das eine harte Einschränkung. Von einem übernommenen Endsystem aus kommt er nicht weiter, denn ein Spoke erreicht keinen anderen Spoke. Jeder Schritt setzt den Hub voraus. Zugleich verschafft ihm der Hub aber nur Reichweite,

nicht Zugriff: Die Zugangsdaten der Arbeitsplätze sowie des Mail-, Datei- und Backup-Servers liegen ausschließlich beim Domain-Controller. Der Angreifer muss also erst diesen übernehmen, um die Reichweite des Hubs überhaupt nutzen zu können.

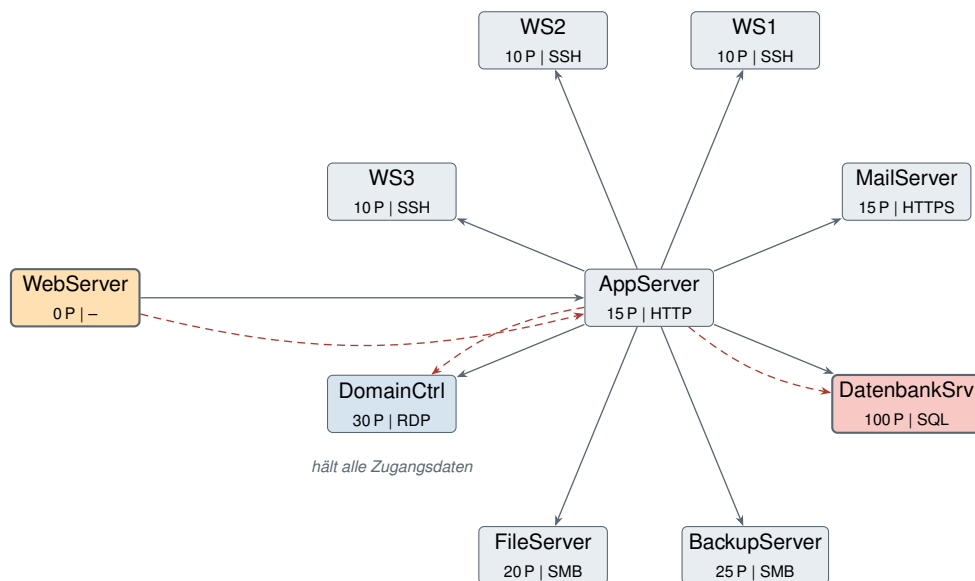


Abbildung 3.3: Topologie *hub\_and\_spoke*. Der Applikationsserver ist als Hub der einzige Übergang zwischen dem Einstiegsknoten und allen übrigen Systemen.

**micro\_segmented.** Für den Einstiegsknoten bleibt genau eine Verbindung übrig: die zum Applikationsserver, dessen Backend er als Webserver benötigt. Der Applikationsserver seinerseits darf die Datenbank und den Domain-Controller erreichen, weil er Anwendungsdaten liest und Anmeldungen prüfen muss. Zu den Arbeitsplätzen, zum Mail-, Datei- und Backup-Server besteht dagegen keine fachliche Notwendigkeit, folglich existiert auch keine entsprechende Freigabe. Diese sechs Knoten sind damit nicht vom Netz getrennt, sie nehmen ihre Arbeit weiterhin auf und greifen ihrerseits auf Dienste zu; nach dem oben Gesagten ist solcher Verkehr hier nur nicht verzeichnet, weil er dem Angreifer keinen Weg eröffnet. Vom Einstiegsknoten aus führt zu ihnen jedoch kein Pfad, sodass für den Angreifer höchstens 145 der insgesamt 235 Punkte erreichbar bleiben (Abbildung 3.4).

Ein zweiter Unterschied betrifft die Zugangsdaten und wiegt für den Angriffsweg schwerer. In den übrigen Topologien hinterlegt der Applikationsserver die Zugangsdaten der Datenbank dauerhaft bei sich, wie es bei einer fest konfigurierten Datenbankverbindung üblich ist. Hier speichert er sie nicht; sie liegen ausschließlich beim Domain-Controller. Der Angreifer erreicht die Datenbank zwar schon vom Applikationsserver aus, kann sie aber erst übernehmen, nachdem er den Domain-Controller kompromittiert hat. Aus zwei Sprüngen werden drei.

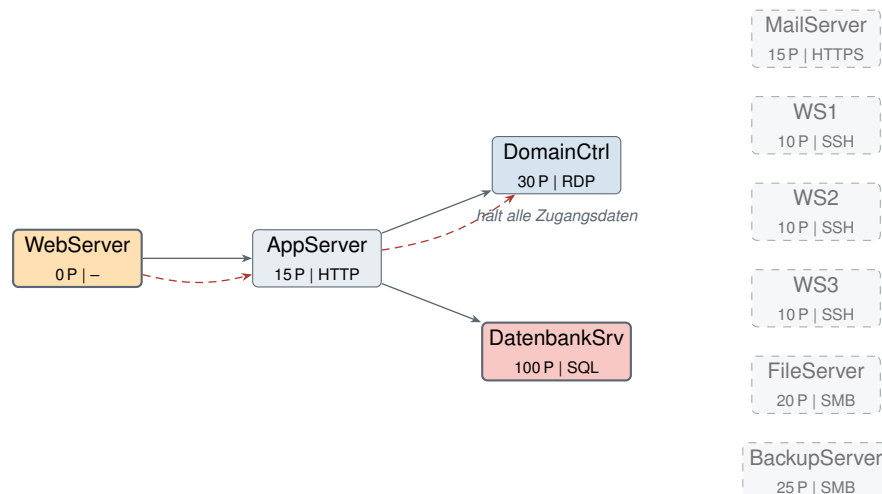


Abbildung 3.4: Topologie *micro\_segmented*. Blass dargestellte Knoten sind vom Einstiegsknoten aus über keinen Pfad erreichbar. Anders als in den übrigen Topologien speichert der Applikationsserver die Zugangsdaten der Datenbank nicht zwischen; sie liegen ausschließlich beim Domain-Controller.

**chain.** Die Kette ist als strikte Linie ohne jede Verzweigung ausgelegt (Abbildung 3.5) und damit die einfachste denkbare Anordnung: Wer sie lösen will, muss nur den Nachbarn entdecken, dessen Zugangsdaten auslesen und ihn übernehmen, und das neun Mal hintereinander. Ein darauf trainierter Angreifer beherrscht folglich genau dieses Grundmuster und nichts darüber hinaus, insbesondere keine Entscheidung zwischen mehreren Wegen.

Damit dient die Kette als Kontrollbedingung für die Frage, ob dieses Grundmuster allein bereits genügt. Behauptet sich ein solcher Angreifer auch in den vier verzweigten Topologien, spricht das dafür, dass diese sich für den Verteidiger weniger unterscheiden, als ihre Struktur nahelegt. Scheitert er dort, lässt sich umgekehrt annehmen, dass die Verzweigungen eine andere Fähigkeit verlangen. Ein Beweis ist beides nicht, da nur eine einzelne Angreiferherkunft betrachtet wird; als Anhaltspunkt taugt der Vergleich gleichwohl. Voraussetzung dafür ist, dass auch der Kette dasselbe Inventar zugrunde liegt: dieselben zehn Knoten, dieselben Werte, dieselben Zugangsports. Verändert ist allein die Anordnung.

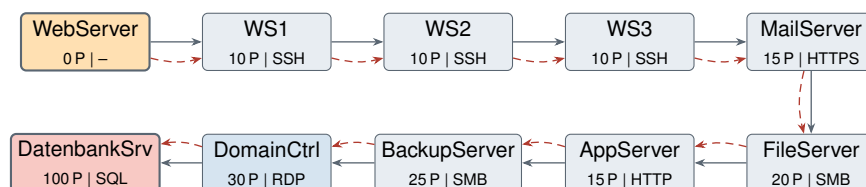


Abbildung 3.5: Topologie *chain*, verwendet für das Angreifertraining. Jeder Knoten gibt Erreichbarkeit und Zugangsdaten ausschließlich für seinen Nachfolger preis; von dieser Topologie führen neun Sprünge bis zum Crown Jewel.

## 3.4. Trainingsphasen

Der Versuch gliedert sich in zwei Phasen. In der ersten wird ausschließlich der Angreifer trainiert, in der zweiten ausschließlich der Verteidiger.

### 3.4.1. Phase 1: Angreifer-Vortraining

Ein Verteidiger lässt sich nur sinnvoll bewerten, wenn ihm ein kompetenter Gegner gegenübersteht. Deshalb werden zunächst Angreifer ohne jede Verteidigung trainiert, jeweils 500.000 Zeitschritte lang.

Naheliegender wäre, den Angreifer schon hier gegen einen Verteidiger antreten zu lassen, etwa einen zufällig handelnden, um ihn auf spätere Gegenwehr vorzubereiten. Darauf wird bewusst verzichtet, und zwar aus demselben Grund, aus dem in Phase 2 der Angreifer eingefroren wird: Was den Angreifer formt, soll allein die Topologie sein. Ein mittrainierter Verteidiger, ob zufällig oder lernend, wirkt als zweite Einflussgröße auf die Herkunft jedes Angreifers ein, und die beobachteten Unterschiede zwischen den Angreifern ließen sich nicht mehr eindeutig der Struktur zuschreiben. Ein zufälliger Verteidiger wäre zudem selbst willkürlich gesetzt: Seine Aktionen bevorzugten keine Topologie planvoll, verzerrten das Angreifertraining aber dennoch in einer nicht kontrollierten Weise. Der Preis dieser Reinheit ist, dass die Angreifer keine Erfahrung mit Gegenwehr sammeln; Abschnitt 4.9 greift das auf.

Fünf *Spezialisten* entstehen dabei, einer je Topologie. Jeder lernt zwar nur in einer einzigen Umgebung, tritt in Phase 2 jedoch in allen vier Verteidigungstopologien als Gegner an. Ein Angreifer trifft dort also in aller Regel auf eine Struktur, die er im Training nicht gesehen hat. Genau diese Kreuzung erlaubt es, den Einfluss der verteidigten Topologie von dem der Angreiferherkunft zu trennen. Ergänzend wird ein *Generalist* trainiert, im Folgenden Super-Angreifer genannt: Ein einzelnes Modell durchläuft nacheinander alle fünf Topologien – in der Reihenfolge *chain*, *flat*, *hub\_and\_spoke*, *dmz*, *micro\_segmented* –, wobei die Gewichte von Stufe zu Stufe weitergereicht werden. Damit lässt sich prüfen, ob ein über mehrere Strukturen trainierter Angreifer den Spezialisten überlegen ist.

### 3.4.2. Phase 2: Verteidiger-Matrix

In der zweiten Phase wird für jede Kombination aus Verteidigungstopologie und Angreifer ein eigener Verteidiger trainiert. Bei vier Topologien und sechs Angreifern ergeben sich

24 Matchups. Der Angreifer ist dabei *eingefroren*: Sein Modell wird geladen, handelt weiter, erhält aber keine Gewichtsadjustments mehr.

Das Einfrieren hält den Gegner über den gesamten Lauf konstant und macht die beobachtete Lernkurve dem Verteidiger zurechenbar. Lernten beide Agenten gleichzeitig, verschöbe sich das Ziel fortlaufend, und ein Unterschied zwischen zwei Topologien ließe sich nicht mehr von einem Unterschied im Gegnerverhalten trennen. Diese Entscheidung lässt sich auch am MARLON-Aufbau selbst festmachen: Dort führt gleichzeitiges Training beider Agenten dazu, dass der Angreifer keinen brauchbaren Angriffsweg mehr findet, weil wiederholte Rücksetzungen durch einen anfangs instabilen Verteidiger ihn immer wieder auf den Ausgangszustand zurückwerfen, bevor er über den Anfang der Episode hinauskommt [6]. Erkauft wird die Zurechenbarkeit damit, dass der Verteidiger sich auf genau ein unveränderliches Gegnerverhalten einspielen kann. Eben deshalb wird nicht ein einzelner Angreifer eingesetzt, sondern gegen jede Topologie alle sechs: Jeder Verteidiger sieht zwar nur einen Gegner, über die Matrix hinweg werden aber sechs verschiedene Angriffsmuster geprüft, davon fünf, die nicht auf der verteidigten Topologie entstanden sind. Vollständig aufheben lässt sich die Einschränkung dadurch nicht; Abschnitt 4.9 greift sie auf.

Jedes Matchup wird fünfmal wiederholt, und verglichen wird nicht der einzelne Lauf, sondern der über die fünf Wiederholungen gemittelte Verlauf. Insgesamt ergeben sich damit 120 Trainingsläufe zu je 500.000 Zeitschritten.

### 3.5. Aktionsraum und Invalid-Action-Masking

Was ein Agent lernen kann, ist durch die Menge der ihm zur Verfügung stehenden Aktionen begrenzt. Dieser Abschnitt beschreibt diese Menge für beide Seiten samt der Maskierung, die im jeweiligen Zustand ungültige Aktionen ausschließt, und prüft abschließend, welche mehrstufigen Strategien darin überhaupt angelegt sind.

#### 3.5.1. Der Aktionsraum des Angreifers

In jedem Zeitschritt wählt der Angreifer genau eine Aktion aus. Eine Aktion ist dabei keine grobe Absicht wie „greife den Dateiserver an“, sondern eine vollständig ausformulierte Anweisung: Sie legt fest, was getan wird, von welchem Knoten aus und gegen welches Ziel. CBS kennt dafür drei Arten.

Eine *lokale Aktion* durchsucht einen bereits übernommenen Knoten von innen, etwa nach dort gespeicherten Zugangsdaten. Eine *entfernte Aktion* richtet sich von einem übernommenen Knoten aus gegen ein anderes System und späht dieses aus. Eine *Verbindung* meldet sich mit erbeuteten Zugangsdaten an einem Zielknoten an und übernimmt ihn. Nur die dritte Art erweitert den Besitz; die ersten beiden beschaffen die Voraussetzungen dafür.

Jede Kombination aus Aktionsart und Parametern bildet einen eigenen Eintrag. Die Größe des Aktionsraums ergibt sich damit als Produkt der jeweiligen Auswahlmöglichkeiten (Tabelle 3.1).

Tabelle 3.1: Aufbau des Aktionsraums des Angreifers bei zehn Knoten.

| Art              | Parameter                                                             | Anzahl |
|------------------|-----------------------------------------------------------------------|--------|
| Lokale Aktion    | 10 Knoten $\times$ 2 Schwachstellen                                   | 20     |
| Entfernte Aktion | 10 Quellen $\times$ 10 Ziele $\times$ 1 Schwachstelle                 | 100    |
| Verbindung       | 10 Quellen $\times$ 10 Ziele $\times$ 6 Ports $\times$ 9 Zugangsdaten | 5400   |
| Gesamt           |                                                                       | 5520   |

Die 5400 Verbindungen wirken auf den ersten Blick redundant, denn ein Knoten besitzt in der Modellierung genau einen Zugangsport und genau einen Satz Zugangsdaten; mit der Wahl des Ziels stünden beide inhaltlich also bereits fest. CBS nimmt diese Einschränkung jedoch nicht vor: Angeboten werden für jede Quelle-Ziel-Kombination sämtliche 54 Kombinationen aus Port und zwischengespeicherter Zugangsdatenkarte, mit dem ausdrücklichen Hinweis im Quellcode der Umgebung, dass Erfolg damit nicht garantiert sei. Der Angreifer muss also selbst lernen, welche einzelne davon zum jeweiligen Knoten passt; sie sind dem Aktionsraum nicht vorweg entzogen.

Technisch sind diese 5520 Einträge als flaches `Discrete` umgesetzt, also als eine einzige durchnummerierte Liste, in der jede vollständige Aktion einen eigenen Index besitzt. MARLon verwendet an dieser Stelle ein `MultiDiscrete`, bei dem der Agent je Zug eine Reihe von Zahlen ausgibt: eine für die Aktionsart, weitere für Quellknoten, Zielknoten, Port und Zugangsdaten.

Nötig wurde die Umstellung durch das im Folgenden beschriebene Masking. Dieses kennzeichnet jeden Eintrag der Liste einzeln als gültig oder ungültig. Bei einem `MultiDiscrete` müsste jede Stelle für sich gekennzeichnet werden, und damit lassen sich verknüpfte Bedingungen nicht ausdrücken: Ob ein Zielknoten in Frage kommt, hängt davon ab, welcher Quellknoten gewählt wurde, denn nur von einem übernommenen Knoten aus lässt sich überhaupt handeln. Eine Kennzeichnung je Stelle kennt diese Abhängigkeit nicht und müsste einen Zielknoten entweder immer oder nie zulassen. Erst wenn sämtliche Kombinationen als einzelne Einträge nebeneinanderstehen, lässt sich die Bedingung abbilden, weil dann genau die gültigen *Gesamtaktionen* bezeichnet werden können. Der Preis ist eine sehr lange Liste, doch da davon je Schritt nur wenige Einträge gültig sind, wählt der Angreifer tatsächlich aus einer kleinen Menge.

Das Verfahren selbst ist das *Invalid-Action-Masking* nach Huang und Ontañón [22]. Vor jeder Entscheidung wird ein boolescher Vektor übergeben, der die im aktuellen Zustand gültigen Aktionen markiert; die Policy wählt ausschließlich aus diesen. Ohne Maskierung entfällt ein Großteil der Schritte auf wirkungslose Aktionen, etwa den Angriff von einem nicht kontrollierten Knoten aus.

Technisch gibt der Actor je Aktion zunächst einen unnormierten Zahlenwert aus, ein sogenanntes *Logit*; erst ein nachgelagerter Rechenschritt überführt diese Werte in die Wahrscheinlichkeiten der Policy [22, Abschn. „Background“]. Die Maskierung setzt an dieser Stelle an: Die Logits ungültiger Aktionen werden vor der Normierung durch eine sehr große negative Zahl ersetzt, in der hier eingesetzten Implementierung  $-10^8$ , sodass ihre Wahrscheinlichkeit praktisch null wird. Die Wahrscheinlichkeitsmasse verteilt sich dadurch ausschließlich auf die gültigen Aktionen, statt über alle 5520 zu streuen. Die Markov-Eigenschaft bleibt davon unberührt, denn maskiert wird nicht der Zustand, sondern die in ihm zulässige Aktionsmenge. Zustandsabhängige Aktionsmengen sind Bestandteil der üblichen MDP-Definition. Huang und Ontañón zeigen zudem, dass der resultierende Gradient weiterhin ein gültiger Policy-Gradient ist [22].

Entscheidend ist der Zeitpunkt der Maskenberechnung. Sie erfolgt unmittelbar vor der Aktionswahl aus dem aktuellen Zustand. Eine zwischengespeicherte Maske wäre um den vorangegangenen Verteidigerzug veraltet, da dieser Knoten neu aufsetzen oder Verbindungen sperren kann. Huang und Ontañón behandeln den Fall eines einzelnen Agenten, bei dem diese Frage nicht auftritt; sobald ein zweiter Agent zwischen zwei Zügen handelt, genügt es nicht mehr, die Maske einmal je eigenem Schritt zu bestimmen.

#### Reichweite der Maskierung: Zugangsdaten allein genügen nicht

Die von CBS bereitgestellte Aktionsmaske prüft für eine Verbindung ausschließlich, ob der Angreifer die betreffende Zugangsdatenkarte besitzt, nicht aber, ob zwischen

Quelle und Ziel überhaupt ein Netzpfad besteht. Weil die Modellierung Erreichbarkeit und Zugangsdatenbesitz trennt (Abschnitt 3.3.1), würde sie damit Verbindungen entlang gar nicht vorhandener Kanten erlauben, sobald der Angreifer die passenden Zugangsdaten irgendwo erbeutet hat, und die Topologie als Einflussgröße unterlaufen.

Die native Maske wurde deshalb um eine eigene Einschränkung ergänzt. Der Angreifer führt seither mit, welche Kante er von welchem Knoten aus tatsächlich aufgedeckt hat; aufgedeckt wird eine Kante ausschließlich durch die auf den Quellknoten gerichtete lokale Aktion zur Nachbarschaftserkundung, nicht durch das Erbeuten von Zugangsdaten. Die von CBS gelieferte Maske wird für Verbindungen und entfernte Aktionen mit dieser Liste geschnitten und bildet damit auch die Netzstruktur ab.

Anders als das Masking selbst, das bereits im Referenzlauf eingesetzt wurde, stammt diese Erweiterung aus der späteren Überarbeitung. Sie greift unmittelbar in das Verhalten des Angreifers ein, weshalb sämtliche Angreifer für die berichteten Läufe neu vortrainiert wurden.

### 3.5.2. Der Aktionsraum des Verteidigers

Der Verteidiger verfügt über drei Aktionsarten: Er kann einen Knoten *neu aufsetzen* und damit den Angreifer von diesem System vertreiben, er kann dessen eingehenden *Verkehr sperren*, und er kann eine bestehende Sperre wieder *aufheben*. Jede der drei Arten benennt ausschließlich den betroffenen Knoten; Sperren und Aufheben wirken dabei auf sämtliche eingehenden Dienste dieses Knotens zugleich, ohne dass Port oder Richtung gesondert gewählt würden.

Technisch ist das als `MultiDiscrete` über vier Dimensionen umgesetzt, `[3, 10, 10, 10]`. Der erste Wert wählt die Aktionsart, die übrigen drei benennen den Knoten für Neuaufsetzen, Sperren beziehungsweise Aufheben; nur die zur gewählten Art passende Dimension wird ausgewertet (Tabelle 3.2).

Tabelle 3.2: Aktionsraum des Verteidigers bei zehn Knoten.

| Aktionsart           | Parameter | Kombinationen | davon wirksam |
|----------------------|-----------|---------------|---------------|
| Knoten neu aufsetzen | 10 Knoten | 10            | 9             |
| Verkehr sperren      | 10 Knoten | 10            | 9             |
| Verkehr aufheben     | 10 Knoten | 10            | 9             |
| Gesamt               |           | 30            | 27            |

Der Einstiegsknoten schränkt beide Spalten der Tabelle zusätzlich ein. Er ist nicht neu aufsetzbar, wodurch für das Neuaufsetzen nur neun der zehn Knoten tatsächlich gültige Ziele sind. Eine Sperre auf ihm ist zwar technisch gültig, bringt aber nie einen Nutzen: Er trägt den Wert null und bleibt über die gesamte Episode kompromittiert, weil er nicht zurückgewonnen werden kann. Wirksam wählbar bleiben damit 27 der 30 strukturell möglichen Aktionen.

Gegenüber dem Original ist dieser Aktionsraum damit in zweifacher Hinsicht enger. MARLON sieht fünf Aktionsarten vor, neben dem Neuaufsetzen, dem Sperren und dem Aufheben auch das Stoppen und das Starten eines Dienstes, und verlangt zu jeder von ihnen zusätzlich Port und Richtung als selbst zu wählende Parameter (Abschnitt 3.1.1). Hier sind es drei Aktionsarten mit dem Knoten als einzigem Parameter. Beide Änderungen gehen auf Beobachtungen am Referenzlauf zurück.

Als erste Änderung sind Dienst stoppen und Dienst starten entfallen. Für den Angreifer ist ein gestoppter Dienst von einer gesperrten Verbindung ohnehin nicht zu unterscheiden, beide lassen den Verbindungsversuch scheitern. Da eine Sperre nach dem Umbau denselben Knoten in seiner Gesamtheit betrifft und seit der in Abschnitt 3.6.2 beschriebenen Korrektur ebenfalls Verfügbarkeit kostet, blieb kein Zweck, den nicht auch sie erfüllt.

Ausschlaggebend war damit nicht, dass eine der beiden Aktionen die schlechtere gewesen wäre, sondern dass zwei Aktionen für dieselbe Wirkung den Aktionsraum ohne Gewinn an Ausdruckskraft vergrößert hätten. Dass die Wahl auf die Sperre fiel, hat einen inhaltlichen Grund: Sie setzt am Zugang zu einem System an und lässt dessen Betrieb unberührt, während ein Dienststopp in das System selbst eingreift. Von zwei

gleich wirkenden Mitteln ist damit das mildere gewählt.

Offenzulegen ist dabei, dass diese Gleichwertigkeit nur für die Sperre in ihrer heutigen Form gilt und nie gemessen wurde. Im Referenzlauf standen dem Verteidiger zwar beide Aktionen gleichzeitig zur Verfügung, aber nicht als gleichwertige Alternativen: Die Sperre wirkte dort je Port statt je Knoten, erreichte wegen der in Abschnitt 3.1.3 beschriebenen Portliste die wertvollsten Systeme überhaupt nicht und brachte, da sie weder Verfügbarkeit kostete noch belohnt wurde, keinerlei Signal hervor. Der Dienststopp dagegen war auf sauberen Knoten mit zehn Punkten belastet. Dass der Verteidiger ihn dort ablegte, ist deshalb kein Vergleichsergebnis. Ein Lauf, in dem beide Aktionen unter den heutigen Bedingungen nebeneinander zur Wahl stünden, wurde nicht durchgeführt; die Gleichwertigkeit folgt aus der Betrachtung der Wirkungsketten, nicht aus einer Messung.

Als zweite Änderung wählt der Verteidiger für Sperren und Freigaben nicht mehr selbst, welcher Port betroffen ist. Er benennt nur den Knoten; welche Dienste dort laufen, ist als Grundwissen des Betreibers vorausgesetzt und kein Erkundungsergebnis, wie es für den Angreifer eines wäre. Dieselbe Asymmetrie gilt bereits an anderer Stelle des Modells: Der Verteidiger spricht jeden Knoten direkt über seinen Index an und darf ihn neu aufsetzen, sieht sein eigenes Netz für die Zwecke dieser Aktionen also vollständig. Die Wahl des Ports blieb die einzige Stelle, an der er über sein eigenes Netz raten musste; diese Uneinheitlichkeit ist mit der Änderung beseitigt.

### 3.5.3. Mehrstufige Strategien im Aktionsraum

Die Wirkung einer Verteidigeraktion hängt davon ab, wo sie ansetzt. Eine Sperre wirkt nur auf den *Ziel*knoten einer Verbindung. Auf einem Knoten, den der Angreifer bereits besitzt, ändert eine Sperre deshalb nichts, denn als Ausgangspunkt einer weiteren Verbindung braucht er den eigenen eingehenden Zugang nicht. Auf einem noch nicht übernommenen Knoten verhindert sie die Übernahme über diesen Weg sehr wohl.

Daraus folgt eine Strategie, die den Angreifer vollständig einschließen würde: Man drängt ihn durch wiederholtes Neuaufsetzen auf den Einstiegsknoten zurück und sperrt anschließend alle von dort aus erreichbaren Knoten. Der Angreifer besäße dann zwar weiterhin ein System, käme aber nirgendwo mehr hin. Der Aktionsraum sieht diese Möglichkeit demnach vor.

Ob ein lernendes Verfahren sie auch findet, ist eine andere Frage. Die Einschließung besteht aus vielen einzelnen Sperren, und ihr Nutzen tritt erst ein, wenn die letzte von ihnen gesetzt ist: Solange auch nur ein Weg offen bleibt, nimmt der Angreifer ihn. Bis

dahin bringt jede einzelne Sperre für sich genommen nichts. Der Verteidiger müsste eine lange Folge bestimmter Aktionen zufällig treffen, bevor sich überhaupt ein Reward zeigt, an dem PPO sie erkennen könnte. Das ist der in Abschnitt 2.1.3 beschriebene Fall der verzögerten Belohnung in ausgeprägter Form.

Erreichbar wird eine solche Strategie erst, wenn bereits die einzelne Sperre einen eigenen Reward trägt, wenn die Zwischenschritte also für sich verstärkt werden und der Agent sie nicht erst am Ende der Kette bewertet bekommt. Genau darauf zielt der in Abschnitt 3.6.1 beschriebene Abwehrbonus. Ob er dafür ausreicht, wird in Kapitel 4 untersucht.

### 3.6. Reward-Struktur und Randbedingungen

Der Reward entsteht auf zwei Ebenen: CBS bewertet jede Angreiferaktion, und der Wrapper setzt darauf die Kopplung beider Agenten sowie die in diesem Abschnitt beschriebenen Ergänzungen.

Tabelle 3.3: Reward des Angreifers je Aktion. Aufgeführt sind nur die Bestandteile, die in den hier verwendeten Topologien überhaupt auftreten können. Der untere Block stammt nicht aus CBS, sondern wird vom Wrapper ergänzt.

| Ereignis                  | Reward      | Anmerkung                                                                                              |
|---------------------------|-------------|--------------------------------------------------------------------------------------------------------|
| Knoten übernommen         | +Knotenwert | 10 bis 100, auch nach Rückeroberung                                                                    |
| Knoten neu entdeckt       | +5          | je Knoten                                                                                              |
| Zugangsdaten neu entdeckt | +3          | je Credential                                                                                          |
| Eigenschaft neu entdeckt  | +2          | je Eigenschaft eines Knotens, hier Betriebssystem und Rolle, die ein Scan aus der Ferne sichtbar macht |

Fortsetzung auf der nächsten Seite

| Ereignis                                 | Reward                           | Anmerkung                                                                          |
|------------------------------------------|----------------------------------|------------------------------------------------------------------------------------|
| Aktion erstmals erfolgreich              | +7                               | je Knoten und Schwachstelle                                                        |
| Fehlversuch, Wiederholung, Aktionskosten | 0                                | in CBS zwischen $-1$ und $-50$ , durch die unten beschriebene Kappung ohne Wirkung |
| Alle Knoten übernommen                   | +5000                            | ersetzt den Schrittwert                                                            |
| Knoten gehalten                          | $+0,005 \cdot \text{Knotenwert}$ | je Knoten und Schritt                                                              |
| Knoten durch Reimage verloren            | $-\text{Knotenwert}$             |                                                                                    |

Damit ist der Reward des Angreifers vollständig beschrieben. Der des Verteidigers folgt in Tabelle 3.4. Er entsteht überwiegend durch Spiegelung der eben aufgeführten Werte und trägt darüber hinaus einige eigene Posten.

Tabelle 3.4: Reward des Verteidigers je Schritt.

| Ereignis      | Reward                                | Anmerkung                                                                                                         |
|---------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Grundkopplung | $-\text{Angreifer-Reward}$            | jeder Schritt                                                                                                     |
| Abwehrbonus   | $\pm 0,00025 \cdot \text{Knotenwert}$ | je Knoten, der gesperrt und zugleich nicht kompromittiert ist; Vorzeichen je nach Bedrohungslage, Abschnitt 3.6.1 |

Fortsetzung auf der nächsten Seite

| Ereignis                       | Reward | Anmerkung                           |
|--------------------------------|--------|-------------------------------------|
| Ungültige Aktion               | 0      | Zug wird übersprungen               |
| SLA unterschritten             | -50    | je Schritt, ergänzt den Schrittwert |
| Angreifer vollständig entfernt | +5000  | strukturell unerreichbar            |

Mehrere Eigenschaften dieser Struktur sind für die Auswertung wesentlich und nicht offensichtlich.

Erstens erklärt sich die Nullzeile in Tabelle 3.3 aus einer Kappung: CBS kennt für Fehlversuche, Wiederholungen und Aktionskosten durchaus Abzüge, begrenzt den Schrittwert des Angreifers aber nach unten auf null, bevor er zurückgegeben wird. Ein Fehlversuch kostet ihn also nichts, er bringt lediglich nichts ein. Diese Kappung ist für sich genommen sinnvoll: Sie verhindert, dass der Verteidiger über die Grundkopplung Reward für Vorgänge gutgeschrieben bekäme, zu denen er nichts beigetragen hat, was das Lernsignal verfälschen würde. Sie schließt aber zugleich, in Verbindung mit der reinen Grundkopplung, den Kanal, über den Prävention im Reward sichtbar würde, wie Abschnitt 3.1.1 gezeigt hat. Die Begrenzung greift zudem nur auf dem Wert, den CBS selbst berechnet; der untere Tabellenblock wird erst danach aufgeschlagen. Negative Angreifer-Rewards entstehen deshalb ausschließlich durch verlorene Knoten: Setzt der Verteidiger einen übernommenen Knoten neu auf, verliert der Angreifer dessen Wert.

Zweitens folgt aus dem Abzug für verlorene Knoten eine Bedingung, die CBS von sich aus nicht erfüllt. Der Abzug ist eine eigene Ergänzung (Tabelle 3.3, unterer Block): Im Auslieferungszustand kostet ein Reimage den Angreifer nichts, er behält den einmal gutgeschriebenen Knotenwert einfach. Damit ist dort auch nichts unausgeglichen. Sobald ein Verlust aber etwas kostet, muss die Rückeroberung desselben Knotens ihn wieder einbringen, sonst ist der Kreislauf aus Verlust und Rückeroberung kein Nullsummengeschäft mehr.

Genau hier lag die Ursache des in Abschnitt 3.1.2 geschilderten Verhaltens, das dem Referenzlauf vorausging. CBS schreibt einen Knotenwert nur beim *ersten* Besitz gut und vermerkt die Übernahme dauerhaft; ein Reimage löscht diesen Vermerk nicht. Zusammen mit dem neu eingeführten Abzug erhielt der Angreifer für eine Rückeroberung also

nichts, während ihn jeder Verlust den vollen Knotenwert kostete. Über die Grundkopplung wurde daraus eine Einnahmequelle für den Verteidiger: Er konnte denselben Knoten wieder und wieder neu aufsetzen und dabei jedes Mal dessen Wert vereinnahmen, ohne dass sich an der Lage im Netz etwas änderte. In Vorversuchen trat dieses Verhalten deutlich hervor. Bei zehn vorhandenen Knoten wurden bis zu 268 Rückgewinnungen je Episode gezählt, und der Verteidiger-Reward korrelierte mit der Zahl der Reimages bis zu  $r = +0,98$ . Der Vermerk wird beim Verlust eines Knotens deshalb zurückgesetzt. Jede Rückeroberung zählt damit erneut als erste und gleicht den vorangegangenen Abzug exakt aus. Der Verteidiger behält seinen Gewinn nur dann, wenn der Angreifer den Knoten nicht zurückholt.

Drittens maß der Reward vor der hier vorgenommenen Ergänzung nur, *ob* ein Knoten kompromittiert wurde, nicht *wie lange*. Ein Angreifer, der ein System zwei Schritte lang hielt, stand damit genauso da wie einer, der es über die gesamte Episode behielt. Der Schaden eines kompromittierten Systems hängt jedoch von beidem ab: von seinem Wert und von der Verweildauer des Angreifers. Deshalb erhält der Angreifer je Schritt einen Anteil des Werts aller aktuell gehaltenen Knoten. Ein fester Betrag je Knoten würde Arbeitsplatzrechner und Produktionsdatenbank gleichsetzen; der wertanteilige Bonus tut das nicht, und der Einstiegs-knoten mit dem Wert null fällt automatisch heraus.

Über die Grundkopplung wirkt dieser Haltebonus unmittelbar auf den Verteidiger, der damit nicht nur für die Rückeroberung belohnt wird, sondern auch dafür, sie früh zu erreichen. Genau diese Kopplung bestimmt auch die Größenordnung des in Abschnitt 3.6.1 eingeführten Abwehrbonus. Der Verteidiger steht bei einem kompromittierten Knoten stets vor der Wahl, diesen neu aufzusetzen oder ihn liegen zu lassen und stattdessen einen bedrohten sauberen Knoten abzuschirmen. *Sauber* heißt dabei, dass der Angreifer den Knoten nicht besitzt; *bedroht* heißt, dass mindestens ein Knoten, der ihn laut Topologie erreichen darf, bereits kompromittiert ist, der Angreifer ihn also unmittelbar angehen kann.

Zwischen diesen beiden Wegen muss das Neuaufsetzen stets die lohnendere Maßnahme sein, denn der Verteidiger soll den Angreifer zurückdrängen und nicht neben ihm her wirtschaften. Andernfalls entstünde ein stilles Einvernehmen: Der Verteidiger ließe den Angreifer sitzen und unterhalte daneben Sperren, für die er fortlaufend bezahlt wird, während der Angreifer ungestört seinen Haltebonus bezieht. Je Schritt kostet ihn das Liegenlassen eines kompromittierten Knotens  $Q$  über die Kopplung  $0,005 \cdot \text{Wert}(Q)$ , und das Abschirmen eines bedrohten Knotens  $Z$  bringt ihm  $0,00025 \cdot \text{Wert}(Z)$ . Damit das Neuaufsetzen auch im ungünstigsten Fall überwiegt, beim billigsten kompromittierten Knoten (Wert 10) gegenüber dem wertvollsten bedrohten Ziel (Wert 100), muss die Haltebonusrate mehr als das Zehnfache der Abwehrbonusrate betragen. Mit 0,005 gegen

0,00025 liegt das Verhältnis bei 20 und damit mit Sicherheitsabstand darüber.

### 3.6.1. Abwehrbonus: ein Zustandsreward für Prävention

Der zentrale Unterschied zum in Abschnitt 3.1.1 beschriebenen Original ist ein zustands-basierter Bonus für erfolgreiche Prävention. Den Anlass liefert der Referenzlauf: Dort blieb ein verhinderter Zugriff für beide Seiten folgenlos, weil er dem Angreifer nichts nahm und dem Verteidiger über die Kopplung deshalb nichts einbrachte (Abschnitt 3.1.2). Ohne ein Signal, das Prävention sichtbar macht, lässt sich nicht untersuchen, ob eine Topologie sie begünstigt. Ein ereignisbasierter Bonus, der jeden abgewehrten Verbindungsversuch einzeln belohnt, wäre durch Wiederholung ausbeutbar: Der eingefrorene Angreifer läuft in jeder Episode erneut gegen dieselbe Sperre, und der Verteidiger erhielte bei jedem Versuch erneut eine Belohnung. Belohnt wird deshalb stattdessen der *Zustand* je Zeitschritt.

Betrachtet wird jeder gesperrte, saubere Knoten  $Z$  mit positivem Wert. Sein Beitrag bemisst sich am Anteil der Bedrohung an allen Knoten, die  $Z$  laut Topologie erreichen dürfen:

$$f = \frac{\text{kompromittierte Vorgänger von } Z}{\text{alle Vorgänger von } Z}, \quad \text{Beitrag}(Z) = 0,00025 \cdot \text{Wert}(Z) \cdot (2f - 1). \quad (3.1)$$

Ist jeder Vorgänger von  $Z$  kompromittiert ( $f = 1$ ), erhält der Verteidiger den vollen positiven Beitrag; ist keiner kompromittiert ( $f = 0$ ), den vollen negativen. Dazwischen wird linear interpoliert. Die zweite Hälfte der Formel ist der eigentliche Kern: Sie verhindert, dass der Verteidiger einfach alles sperrt und dafür bezahlt wird, denn eine Sperre ohne bestehende Bedrohung kostet ihn ebenso viel, wie eine Sperre gegen eine vollständige Bedrohung ihm einbringt.

Eine frühere, hier verworfene Fassung entschied binär, ob *irgendein* kompromittierter Knoten den Zugangsport von  $Z$  überhaupt hätte nutzen dürfen, unabhängig davon, wie viele saubere Knoten dieselbe Sperre zusätzlich vom Zugriff ausschloss. In einer Topologie mit hohem Eingangsgrad, etwa *flat* mit neun eingehenden Kanten je Knoten, zahlte diese Fassung praktisch jede beliebige Sperre positiv aus, selbst wenn sie überwiegend legitime Zugänge und nur nebenbei eine tatsächliche Bedrohung traf. Die Der Anteil  $f$  aus Gleichung (3.1) vermeidet das, weil er die Zahl der Vorgänger im Nenner führt: Je mehr Wege auf einen Knoten führen, desto größer muss der kompromittierte Teil von ihnen sein, damit sich eine Sperre auszahlt. Grundlage der Vorgängerbeziehung ist dabei die in Abschnitt 3.3.1 definierte Adjazenz.

Der Abwehrbonus bleibt gegenüber der SLA-Strafe und der Grundkopplung bewusst klein und soll das auch sein. Der eigentliche Anreiz zur Prävention entsteht über die Grundkopplung selbst: Ein veränderter Zugriff ist ein Knotenwert samt zugehörigem Haltebonus, den der Angreifer nie erhält, und dieser Effekt gilt unabhängig davon, wie präzise eine einzelne Sperre im Sinne von Gleichung (3.1) war.

Die Grundkopplung macht das Spiel bis auf zwei Ausnahmen zu einem Nullsummenspiel: Was der Angreifer gewinnt, verliert der Verteidiger und umgekehrt. Daraus folgt unmittelbar, dass ein Reimage dem Verteidiger den Wert des zurückgewonnenen Knotens einbringt, denn der Angreifer verliert ihn, und die Kopplung spiegelt das. Die beiden Ausnahmen sind der Abwehrbonus selbst und die SLA-Strafe.

### 3.6.2. Verfügbarkeit und SLA-Schwelle

Die Verfügbarkeit ist die einzige Größe, die dem Verteidiger Grenzen setzt. Ohne sie könnte er beliebig viele Knoten gleichzeitig neu aufsetzen und sperren, ohne dafür je zu zahlen. Ihre Berechnung ist deshalb im Einzelnen anzugeben. CBS führt eine Netzverfügbarkeit zwischen null und eins. Sie ist das mit den Knotengewichten gebildete Mittel über alle Knoten; im vorliegenden Aufbau tragen alle zehn Knoten das Gewicht 1. Der Beitrag eines einzelnen Knotens beträgt

$$a_i = \begin{cases} \frac{1 + \sum_{s \in S_i} w_s \cdot [s \text{ erreichbar}]}{1 + \sum_{s \in S_i} w_s} & \text{Knoten in Betrieb,} \\ 0 & \text{Knoten wird neu aufgesetzt,} \end{cases} \quad (3.2)$$

wobei  $S_i$  die Dienste des Knotens bezeichnet,  $w_s$  deren Gewicht, das hier durchgängig 1 beträgt, und ein Dienst als erreichbar gilt, wenn er läuft *und* sein Port nicht durch eine ausdrückliche Sperr-Regel des Verteidigers blockiert ist. Diese zweite Bedingung war im in Abschnitt 3.1.1 beschriebenen Original nicht vorgesehen; ohne sie wäre Sperren in Bezug auf die Verfügbarkeit vollständig kostenlos gewesen. Ein Knoten mit einem einzigen Dienst trägt somit 1,0 bei, solange der Dienst läuft und sein Port offen ist, und 0,5, wenn der Verteidiger ihn selbst gesperrt hat. Das ist zugleich der Regelfall: Jeder Knoten des Inventars trägt genau einen Dienst, allein der Einstiegsknoten trägt zwei. Er kommt als Ziel einer Sperre jedoch ohnehin nicht in Betracht. Eine Sperre trifft nur eingehende Verbindungen, und der Angreifer sitzt von Beginn an auf ihm und nutzt ihn als Ausgangspunkt; einen Abwehrbonus trägt eine Sperre auf ihm ebenfalls nicht ein, denn der Bonus wird für den gesperrten Knoten selbst berechnet, und dieser ist hier dauerhaft kompromittiert und trägt zudem den Wert 0.

Entscheidend ist der zweite Fall in Gleichung (3.2): Ein neu aufgesetzter Knoten trägt 15 Zeitschritte lang nichts bei. Die Schwelle liegt bei 0,60; sie wird unterschritten, sobald weniger als 60 Prozent der gewichteten Verfügbarkeit erhalten sind. Bei zehn gleich gewichteten Knoten bedeutet das: Vier gleichzeitig neu aufgesetzte Knoten ergeben genau 0,60 und lösen noch nicht aus, fünf ergeben 0,50 und damit einen Bruch. Eine einzelne Sperre verschiebt diese Grenze um 0,05.

Der Verteidiger gerät also nicht durch vorsichtiges Handeln in einen Bruch, sondern entweder durch ausgedehntes gleichzeitiges Neuaufsetzen oder durch Sperren in einem Umfang, der einen erheblichen Teil des Netzes gleichzeitig betrifft. Anders als beim Original ersetzt ein SLA-Bruch dabei nicht mehr den gesamten Schrittwert, sondern ergänzt ihn um  $-50$ . Solange Brüche selten blieben, machte diese Unterscheidung wenig Unterschied; seit gesperrte Ports auf die Verfügbarkeit einwirken, sind Brüche jedoch der Normalfall geworden, und eine ersetzende Strafe hätte einzelne Episoden vollständig dominiert und mit allen übrigen unvergleichbar gemacht.

Abschließend sind zwei Randbedingungen des Aufbaus festzuhalten; beide bestimmen, wie eine Episode verläuft und endet. Eine Episode umfasst höchstens 2000 Zeitschritte, wobei der Angreifer zuerst agiert. Vorzeitig endet sie ausschließlich dann, wenn der Angreifer sämtliche zehn Knoten übernommen hat. Insbesondere beendet ein SLA-Bruch die Episode nicht, obwohl CBS das vorsähe; er wird lediglich gezählt und bestraft. Damit beruhen alle Matchups auf derselben Datenbasis.

Der Einstiegsknoten gehört dem Angreifer von Beginn an und kann nicht neu aufgesetzt werden. Die formale Siegbedingung des Verteidigers, den Angreifer vollständig zu verdrängen, ist damit strukturell unerreichbar. Das ist beabsichtigt: Wäre der Einstiegsknoten neu aufsetzbar, könnte der Verteidiger die Episode im ersten Schritt beenden und dafür den Siegbonus von 5000 einstreichen. Zu lernen bliebe allein diese eine Aktion, und alles Übrige, Sperren wie Abwägen, wäre gegenstandslos. Gemessen wird folglich Eindämmung, nicht Auslöschung. Für den Angreifer bleibt der verbleibende Fußabdruck wertlos, denn der Einstiegsknoten trägt den Wert 0.

### 3.7. Konvergenzkriterium

Ziel ist festzustellen, ab welcher Episode ein Verteidiger nichts Neues mehr dazulernt. Diese Episode ist eine der drei Zielgrößen dieser Arbeit (Abschnitt 3.8): Sie misst, wie schnell ein Verteidiger ein stabiles Verhalten erreicht, und ist damit die Größe, an der sich ein Einfluss der Topologie auf die Lerneffizienz zeigen müsste. Das Kriterium dient

also der Messung, nicht der Steuerung des Trainings. Die Schwierigkeit liegt darin, ein solches Plateau von einer zufällig ruhigen Phase zu unterscheiden.

Zwei Bedingungen müssen dafür gleichzeitig erfüllt sein, jeweils über ein gleitendes Fenster von  $W = 15$  Episoden. Die erste betrifft die *Streuung*: Die Standardabweichung der letzten  $W$  Episoden-Rewards, geteilt durch die bisherige Reward-Spanne des Laufs, darf 0,05 nicht überschreiten. Die zweite betrifft den *Trend*: Der Abstand zwischen dem Mittelwert der letzten  $W$  Episoden und dem der  $W$  davor, ebenfalls auf die Spanne bezogen, darf denselben Wert nicht überschreiten. Formal gilt mit  $r_t$  als Reward der Episode  $t$  und  $\Delta$  als Spanne aller bisherigen Rewards:

$$\frac{\sigma(r_{t-W+1}, \dots, r_t)}{\Delta} \leq 0,05 \quad \text{und} \quad \frac{|\bar{r}_{t-W+1:t} - \bar{r}_{t-2W+1:t-W}|}{\Delta} \leq 0,05. \quad (3.3)$$

Beide Bedingungen müssen über zehn aufeinanderfolgende Episoden gehalten werden, damit eine einzelne ruhige Phase nicht bereits als Konvergenz gilt. Als Konvergenzepisode gilt diejenige, in der diese Strecke vollständig ist. Da zwei Fenster als Historie benötigt werden, können die Bedingungen frühestens in Episode 30 erstmals zutreffen, und die Strecke ist frühestens in Episode 39 erfüllt. Vor Episode 39 kann ein Lauf demnach nicht als konvergiert gelten.

Die Normierung auf die Spanne statt auf den Mittelwert ist bewusst gewählt. Der übliche Variationskoeffizient teilt die Standardabweichung durch den Mittelwert; da der Verteidiger-Reward jedoch um null pendeln kann, würde dieser Quotient dort unbeschränkt wachsen. Die Spanne bleibt demgegenüber stets ein belastbarer Bezugswert und erlaubt zudem den Vergleich zwischen Topologien mit sehr unterschiedlichem Reward-Niveau. Für dieses Kriterium existiert keine Literaturquelle; es ist empirisch an eigenen Läufen kalibriert.

Anders als noch im Referenzlauf wird dieses Kriterium nicht mehr verwendet, um ein Training vorzeitig abubrechen. Alle 120 Läufe erhalten dasselbe Budget von 500.000 Schritten; erst nachträglich wird das Kriterium auf die vollständige Lernkurve angewendet, um die Episode zu bestimmen, ab der ein Lauf als konvergiert gilt. Dieser Verzicht auf einen laufzeitgesteuerten Abbruch hat drei Gründe. Der erste ist die Vorgeschichte: In früheren Läufen, deren Reward-Struktur noch eine andere war, griff das Kriterium sehr früh, sodass die Läufe nach wenigen Dutzend Episoden endeten und über den weiteren Verlauf nichts mehr auszusagen war. Die übrigen beiden Gründe betreffen die Vergleichbarkeit. Nur bei gleichem Budget ist gewährleistet, dass alle 24 Matchups tatsächlich vergleichbar sind: Ein früh abgebrochener Lauf hätte strukturell weniger Gelegenheit zum Lernen gehabt als ein durchgelaufener. Und die Konvergenzepisode ist selbst die abhängige Variable dieser Arbeit. Würde das Training beim Erreichen des Kriteriums

beendet, ließe sich nicht mehr prüfen, ob die damit unterstellte Konvergenz tatsächlich gehalten hätte. Die gemessene Konvergenzepisode wäre dann nichts anderes als der Zeitpunkt, zu dem der Abbruch ausgelöst wurde, und keine aus dem weiteren Verlauf bestätigte Eigenschaft des Lernens.

Das Kriterium setzt ein unveränderliches Ziel voraus. Diese Annahme ist in Phase 2 erfüllt, weil der Angreifer eingefroren ist. Für Aufbauten, in denen beide Agenten gleichzeitig lernen, wäre es nicht zulässig.

### 3.8. Erhobene Metriken

Erhoben wird je Episode eine Reihe von Größen. Sie haben nicht denselben Rang: Drei von ihnen sind Zielgrößen, an denen die Fragestellung beantwortet wird; die übrigen dienen dazu, einen beobachteten Unterschied auf sein Zustandekommen zurückzuführen.

#### Zielgrößen

Der *Konvergenzzeitpunkt* aus Abschnitt 3.7 misst die Lerneffizienz, also wie schnell ein Verteidiger ein stabiles Verhalten erreicht, unabhängig davon, auf welchem Niveau. Eine ergänzende Metrik, die stattdessen einen Leistungsmeilenstein abbildet, nämlich die Episode, ab der der geglättete Verteidiger-Reward nicht mehr negativ ist, entfällt bewusst: Unter der Grundkopplung ist dieser Reward strukturell negativ (Abschnitt 3.6).

Die *Crown-Jewel-Quote* gibt den Anteil der Episoden an, in denen der Angreifer den Datenbankserver übernimmt. Sie misst die Wirksamkeit der Verteidigung unmittelbar und ist unabhängig von der Reward-Skala, kennt aber nur zwei Ausgänge.

Feiner auflösend ist deshalb die *Zahl der gehaltenen Knoten*, erhoben als größte Zahl gleichzeitig kompromittierter Systeme im Verlauf einer Episode. Sie unterscheidet einen Angreifer, der im Einstiegsknoten steckenbleibt, von einem, der bis kurz vor das Crown Jewel vordringt, und bildet damit auch Teilerfolge der Verteidigung ab, die die Crown-Jewel-Quote allein nicht sichtbar macht.

#### Größen zur Deutung

Der *Episoden-Reward beider Agenten* ergibt, über die Episoden aufgetragen, das Paar von Lernkurven, aus dem der Konvergenzzeitpunkt abgeleitet wird. Als Wirksamkeitsmaß taugt er nicht: Er gibt an, wie wenig der Angreifer gewinnt, nicht wie zuverlässig ein Angriff verhindert wurde. Aufschlussreich ist vor allem der Abstand zwischen beiden Kurven. Der Reward des Angreifers entsteht allein aus dem Geschehen im Netz, der des Verteidigers zusätzlich aus Abwehrbonus und SLA-Strafe. Weichen die Kurven

voneinander ab, so misst der Abstand demnach, was den Verteidiger sein eigenes Vorgehen kostet.

Die *Zahl der Schritte mit SLA-Bruch* beziffert diesen Anteil unmittelbar. Sie ist die Gegenrechnung zu jeder Verteidigungsleistung: Eine niedrige Crown-Jewel-Quote ist wenig wert, wenn sie mit einem über weite Strecken abgeriegelten Netz erkaufte wurde.

Die *Aktionsstatistik* protokolliert, welche Aktionstypen beide Agenten gewählt haben, getrennt nach gültigen und ungültigen Aktionen. Aus ihr lassen sich Verhaltensänderungen ablesen, die der Reward allein nicht sichtbar macht, etwa eine Verschiebung zwischen Sperren und Neuaufsetzen im Verlauf eines Laufs. Mitgeschrieben werden außerdem der Abwehrbonus je Episode und die Zahl der dabei abgeschirmten Knoten. Für die Höhe des Rewards ist der Bonus bedeutungslos, für die Deutung nicht: Er ist die einzige Größe, die Prävention unmittelbar ausweist, und damit die Probe darauf, ob die in Abschnitt 3.6.1 beabsichtigte Wirkung überhaupt eintritt.

Alle genannten Größen werden je Episode mitgeschrieben und beschreiben damit zunächst den Trainingsverlauf. Im folgenden Abschnitt werden dieselben Größen ein zweites Mal erhoben, dann an einer fertig trainierten Policy.

## 3.9. Wirksamkeitsevaluation

Das Training beantwortet, wie schnell ein Verteidiger stabil wird und welches Vorgehen er sich dabei aneignet. Es beantwortet nicht, wie gut der fertig trainierte Verteidiger ist. Ein Agent kann auf einem schlechten Niveau ebenso zuverlässig konvergieren wie auf einem guten. Für die Frage nach der Wirksamkeit ist deshalb ein zweiter Durchgang nötig, der von der Lerngeschwindigkeit vollständig getrennt ist.

### 3.9.1. Aufbau

In der Evaluation sind *beide* Agenten eingefroren. Es findet keine Gewichts Anpassung mehr statt; die fertigen Policies werden nur noch ausgeführt. Gemessen wird damit das Ergebnis des Trainings, nicht sein Zustandekommen.

Jeder der 120 Trainingsläufe wird in drei Stufen wiederholt, die sich ausschließlich im Verteidiger unterscheiden:

**Kein Verteidiger** Der Angreifer agiert ungehindert. Diese Stufe liefert die Obergrenze

dessen, was in der jeweiligen Topologie überhaupt zu erreichen ist, und dient als Bezugsgröße.

**Zufallsverteidiger** Ein Agent, der in jedem Schritt gleichverteilt aus dem Aktionsraum aus Tabelle 3.2 zieht. Er handelt also, ohne etwas gelernt zu haben.

**Trainierter Verteidiger** Das PPO-Modell des jeweiligen Laufs.

Der Angreifer ist in allen drei Stufen derselbe und wird aus demselben Lauf-Ordner geladen wie das Verteidigermodell. Damit unterscheiden sich die Stufen tatsächlich nur im Verteidiger.

Der Zufallsverteidiger ist dabei die eigentliche Messlatte, nicht das ungeschützte Netz. Dass ein Verteidiger besser abschneidet als gar keiner, sagt wenig; die Frage ist, ob das Gelernte über zufälliges Handeln hinausgeht.

Je Zelle werden 25 Episoden mit derselben Episodenlänge von höchstens 2000 Schritten gespielt. Bei vier Topologien, sechs Angreifern, fünf Seeds und drei Stufen ergeben sich 360 Zellen und damit 9000 Episoden.

### 3.9.2. Auswertungsgrößen

Als Hauptgröße dient der *Restanteil*: der Angreifer-Reward mit Verteidiger, geteilt durch den Angreifer-Reward ohne Verteidiger, jeweils im selben Matchup. Ein Wert von 100 Prozent bedeutet, dass der Verteidiger nichts bewirkt, ein Wert nahe null, dass der Angriff weitgehend unterbunden wird. Gebildet wird er in drei Stufen: zunächst der Median über die Episoden einer Zelle, dann der Median über die fünf Seeds, und erst danach das Verhältnis. Berechnet wird er für den trainierten und für den Zufallsverteidiger gleichermaßen, beide gegen dieselbe Stufe ohne Verteidiger. Die Differenz beider Restanteile ist der eigentliche Befund, denn sie beziffert, was das Training über zufälliges Handeln hinaus einbringt.

Auf ein Mittel über die sechs Angreifer wird bewusst verzichtet. Deren erreichbare Niveaus liegen weit auseinander, und beide naheliegenden Aggregationen verzerren in unterschiedliche Richtungen: Ein Verhältnis der Summen wird vom stärksten Angreifer bestimmt, ein Mittel der Einzelverhältnisse vom schwächsten. Die Ergebnisse werden deshalb je Matchup ausgewiesen.

Ergänzend dienen dieselben beiden Zielgrößen wie im Training, die Zahl der vom Angreifer gehaltenen Knoten und die Crown-Jewel-Quote. Der Restanteil sagt, um welchen Faktor der Ertrag des Angreifers schrumpft, aber nicht, woran das liegt; ein

halbierter Reward kann bedeuten, dass der Angreifer auf halbem Weg stehen blieb, oder dass er zwar durchkam, aber deutlich länger brauchte. Erst die beiden Zielgrößen entscheiden zwischen diesen Fällen.

# 4

## Ergebnisse und Interpretation

Grundlage dieses Kapitels ist der finale Lauf: 120 Trainingsläufe über 24 Matchups und fünf Seeds, gefolgt von 9000 Evaluationsepisoden über 360 Läufe. In allen folgenden Tabellen steht die Zeile für die *verteidigte* Topologie und die Spalte für den Angreifer, also für die Topologie, auf der dieser trainiert wurde.

Zahlen werden in diesem Kapitel nach drei festen Vorschriften gebildet, die hier einmal benannt seien, statt sie bei jeder Größe zu wiederholen.

*Verlaufswerte* beschreiben die Form der Lernkurven in den Abbildungen: je Episode das Mittel über die fünf Seeds, darüber ein zentriertes gleitendes Mittel über fünf Episoden. Über die Seeds wird bewusst gemittelt und nicht der Median genommen: Ein Einbruch, der nur einzelne Läufe betrifft, soll in der Kurve sichtbar bleiben, und gerade solche Einbrüche werden in Abschnitt 4.1 besprochen. Die Glättung dient allein der Lesbarkeit.

*Niveauwerte* beziffern, wo ein Training beginnt und wo es endet: je Seed der Median seiner ersten beziehungsweise letzten zehn Episoden, darüber das Mittel der fünf Seeds; sie heißen im Folgenden *Startniveau* und *Endniveau*. Der Median je Seed wirkt vor allem am Trainingsende: Dort kann eine einzelne Episode mit SLA-Brüchen um Größenordnungen unter dem typischen Wert liegen und würde ein Mittel allein bestimmen; am Anfang, wo alle zehn Episoden gleichermaßen extrem ausfallen, gibt erschlicht die Mitte des Fensters wieder. Das anschließende Mittel hält alle fünf Seeds im Ergebnis, sodass ein abweichender Lauf den Zellwert sichtbar verschiebt. Alle Anfangs- und Endwerte in Text und Tabellen sind solche Niveauwerte und keine abgelesenen Kurvenpunkte. Der Grund liegt am Rand der Kurven: Das zentrierte Glättungsfenster ragt

dort über die vorhandenen Episoden hinaus, bei Episode 1 stehen von fünf Episoden nur drei, und der abgelesene Wert hinge damit mehr an der gewählten Fensterbreite als an den Daten. Wo der Text einen Wert aus dem Inneren einer Kurve zitiert, ist er als Wert der Mittelkurve gekennzeichnet.

*Quoten* wie die Crown-Jewel-Quote beruhen je Episode und Seed auf einem einzelnen Ja oder Nein. Fünf Seeds erlauben je Episode damit nur sechs mögliche Quoten in Stufen von zwanzig Prozentpunkten; ein Verlauf ist so nicht ablesbar. Quoten werden deshalb gemeinsam über ein Fenster von elf Episoden und alle fünf Seeds gebildet, also aus 55 Beobachtungen je Punkt. In der Evaluation entfällt das Fenster: Die Policy ist dort fertig trainiert, es gibt keinen Verlauf mehr, und eine Quote fasst schlicht alle 125 Episoden einer Zelle zusammen.

## 4.1. Lernkurven je Topologie

Die erste Frage an einen Trainingslauf ist, ob überhaupt etwas gelernt wurde. Sie lässt sich am Verlauf des Episoden-Rewards ablesen, aber nicht an dem des Verteidigers allein.

Dessen Kurve steigt nämlich in allen 120 Läufen, und kein einziger verschlechtert sich. Das ist weniger ein Befund als eine Folge des Aufbaus. Ein untrainierter Verteidiger sperrt wahllos, unterschreitet damit die Verfügbarkeitsschwelle und kassiert die Strafe von  $-50$  über hunderte Schritte einer Episode (Abschnitt 3.6.2). Die Startwerte fallen dadurch extrem aus und sagen kaum etwas über den Angreifer; beispielsweise liegt das Startniveau von *flat* gegen den eigenen Angreifer bei  $-8381$  (Tabelle 4.2). Dass der Agent diese Strafe binnen weniger Episoden abstellt, ist die erste und drastischste Korrektur; eine Verbesserung ist von diesem Startniveau aus fast unvermeidlich.

An der zweiten Kurve entscheidet sich deshalb die eigentliche Frage: Gelingt es dem Verteidiger, den Reward des Angreifers zu senken, ihn also tatsächlich am Vordringen zu hindern? Nur diese Größe entsteht allein aus dem Geschehen im Netz und ist von Abwehrbonus und SLA-Strafe unberührt (Abschnitt 3.6).

Auch darauf lautet die Antwort ja, allerdings in sehr unterschiedlichem Ausmaß. Tabelle 4.1 stellt Anfang und Ende für alle 24 Matchups gegenüber.

Tabelle 4.1: Angreifer-Reward als Start- und Endniveau. Hervorgehoben sind die Zellen, in denen Verteidiger und Angreifer auf derselben Topologie entstanden sind.

| Verteidigt  | Chain     | Flat              | H & S           | DMZ              | Micro           | Super     |
|-------------|-----------|-------------------|-----------------|------------------|-----------------|-----------|
| Flat        | 296 → 160 | <b>1349 → 664</b> | 325 → 164       | 257 → 147        | 300 → 166       | 581 → 188 |
| Hub & Spoke | 230 → 110 | 271 → 62          | <b>758 → 75</b> | 139 → 80         | 253 → 108       | 275 → 64  |
| DMZ         | 208 → 80  | 161 → 66          | 225 → 86        | <b>901 → 150</b> | 137 → 64        | 164 → 79  |
| Micro-Seg.  | 177 → 77  | 244 → 75          | 194 → 49        | 68 → 52          | <b>655 → 20</b> | 495 → 77  |

Die hervorgehobenen Zellen sind diejenigen, in denen die verteidigte Topologie und die Trainingstopologie des Angreifers übereinstimmen. Da die Spalten in derselben Reihenfolge stehen wie die Zeilen, bilden sie in der Tabelle eine Diagonale; dieser Begriff wird im Folgenden für sie verwendet.

Die seedweise Bildung der Niveauwerte ist dabei keine Förmlichkeit. Da das Budget in Zeitschritten und nicht in Episoden vergeben ist, umfassen die Läufe unterschiedlich viele Episoden. Deutlich tritt das in drei Matchups auf: 251 bis 253 in *flat* gegen *flat*, 252 bis 259 in *hub\_and\_spoke* gegen *hub\_and\_spoke* und 261 bis 311 in *dmz* gegen *dmz*. Würde man erst über die Seeds mitteln und dann die letzten Episoden nehmen, so beruhte das Ende einer solchen Zelle auf immer weniger Läufen. Alle drei betroffenen Matchups liegen auf der Diagonalen; eine ungenauere Bildung hätte also gerade die Zellen verzerrt, an denen die Befunde dieses Kapitels hängen.

Schon die Startwerte zeigen, dass die Herkunft eines Angreifers zählt. Gegen einen noch untrainierten Verteidiger erreicht in jeder der vier Zeilen derjenige Angreifer den höchsten Reward, der auf ebendieser Topologie trainiert wurde. Der Abstand zum Zweitbesten reicht dabei vom Faktor 1,3 in *micro\_segmented* bis zum Faktor 4,0 in *dmz*, wo der Spezialist 901 Punkte erzielt und der nächstbeste Angreifer 225. Zweitbester ist in *flat* und *micro\_segmented* jeweils der Super-Angreifer, also der einzige, der ebenfalls auf dieser Topologie trainiert hat, wenn auch nicht ausschließlich.

Außerhalb der Diagonalen behält der Angreifer am Ende zwischen 16 und 76 Prozent seines anfänglichen Rewards, im Mittel 42 Prozent. Auf der Diagonalen bricht er dagegen auf 3 bis 17 Prozent ein. Der Angreifer, der die verteidigte Struktur kennt, startet also von einem weit höheren Niveau und wird dennoch am gründlichsten zurückgedrängt.

Eine Zelle folgt diesem Muster nicht. *flat* gegen *flat* startet mit 1349 ebenfalls hoch, endet

aber bei 664 und behält damit knapp die Hälfte, also ähnlich viel wie ein gewöhnliches Matchup. Dieser Fall wird das gesamte Kapitel begleiten.

Für jede verteidigte Topologie zeigt eine eigene Abbildung den Verlauf beider Agenten gegen alle sechs Angreifer. Jedes der sechs Teilbilder trägt dabei eine eigene Skala, denn die Anfangswerte der einzelnen Matchups liegen um mehr als eine Größenordnung auseinander.

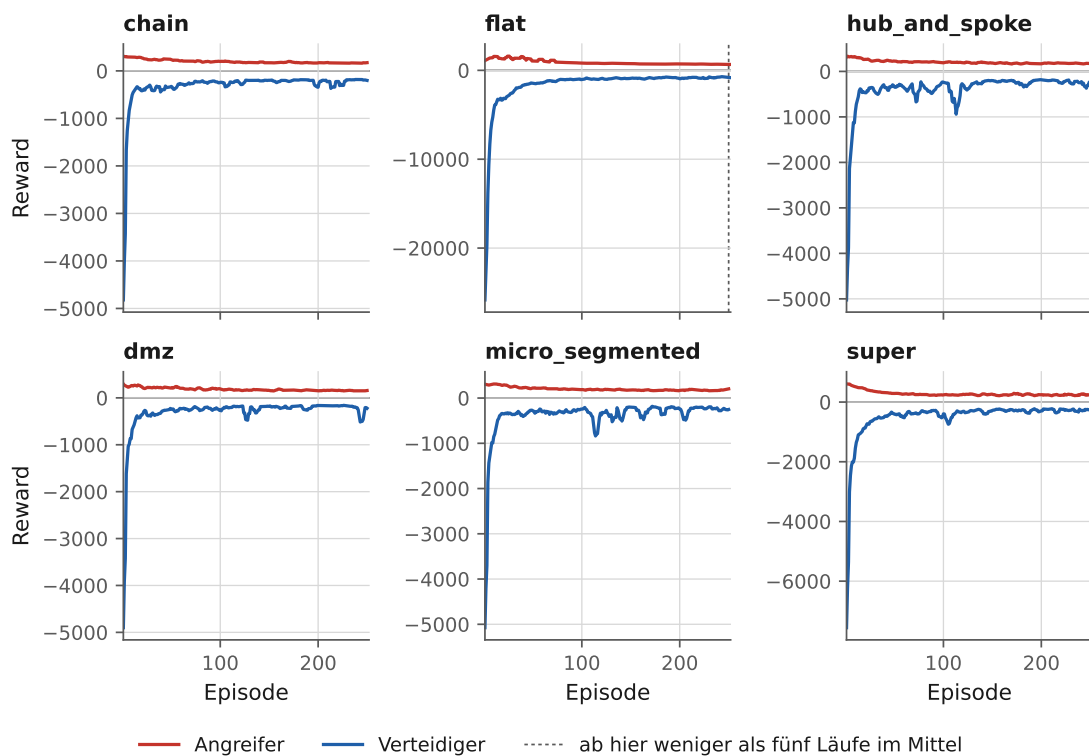


Abbildung 4.1: Verteidiger auf *flat* gegen alle sechs Angreifer, gemittelt über fünf Seeds und geglättet über das gleitende Mittel von fünf Episoden.

Die Zeile *flat* in Tabelle 4.1 zeigt den Angreifer in fünf der sechs Spalten auf einem höheren Endniveau als in jeder anderen verteidigten Topologie; einzig der DMZ-Angreifer endet auf seiner Heimattopologie mit 150 knapp über den 147. In Abbildung 4.1 kommt entsprechend keine der sechs Verteidigerkurven der Nulllinie nennenswert nahe. Am deutlichsten ist das im Teilbild des eigenen Angreifers, oben in der Mitte; Tabelle 4.2 beziffert es: Startniveau  $-8381$ , Endniveau  $-710$ , während dieselbe Topologie gegen alle übrigen Angreifer zwischen  $-168$  und  $-197$  endet.

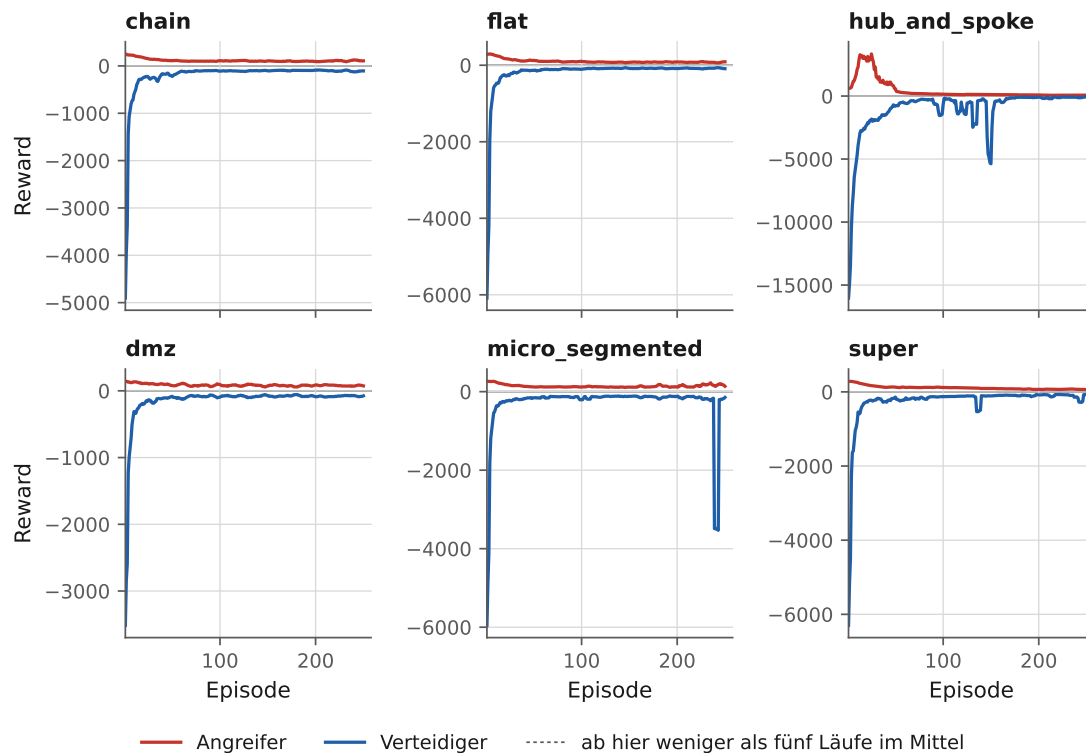


Abbildung 4.2: Verteidiger auf *hub\_and\_spoke* gegen alle sechs Angreifer.

In *hub\_and\_spoke* (Abbildung 4.2) fällt das Teilbild des eigenen Angreifers durch etwas anderes auf. Dort steigt die Mittelkurve des Angreifers zunächst bis auf über +3300 in Episode 25, also weit über das Niveau aller übrigen Teilbilder, und fällt dann über das Training fast vollständig wieder ab. Der Verteidiger beginnt im Startniveau bei -7910 und endet bei -78 (Tabelle 4.2). Der Anfang ist dabei kein Spiegelbild: Der Verteidiger verliert ein Vielfaches dessen, was der Angreifer gewinnt, weil zur Kopplung noch die SLA-Strafe hinzukommt. Erst am Ende stehen die beiden Werte einander spiegelbildlich gegenüber, und genau das zeigt an, dass keine Brüche mehr anfallen. Der auf dieser Topologie trainierte Angreifer beginnt also mit Abstand am stärksten und verliert im Verlauf 90 Prozent seines Rewards. Am Ende ist er von den übrigen fünf Angreifern dieser Zeile nicht mehr zu unterscheiden, die zwischen 62 und 110 liegen (Tabelle 4.1).

In dieser Abbildung fallen zudem die tiefen Einbrüche auf, etwa gegen Episode 150 im selben Teilbild und gegen Episode 240 gegen den Micro-Segmentation-Angreifer. Beide gehen auf einzelne Seeds zurück und haben einen unterschiedlichen Umfang. Der erste ist eine Phase: In zwei der fünf Läufe häufen sich zwischen Episode 110 und 165 die SLA-Brüche, in einem über 21 und im anderen über 16 Episoden. Der zweite ist eine einzige Episode, in der die Verfügbarkeit über 1668 der rund 2000 Schritte unter der

Schwelle lag.

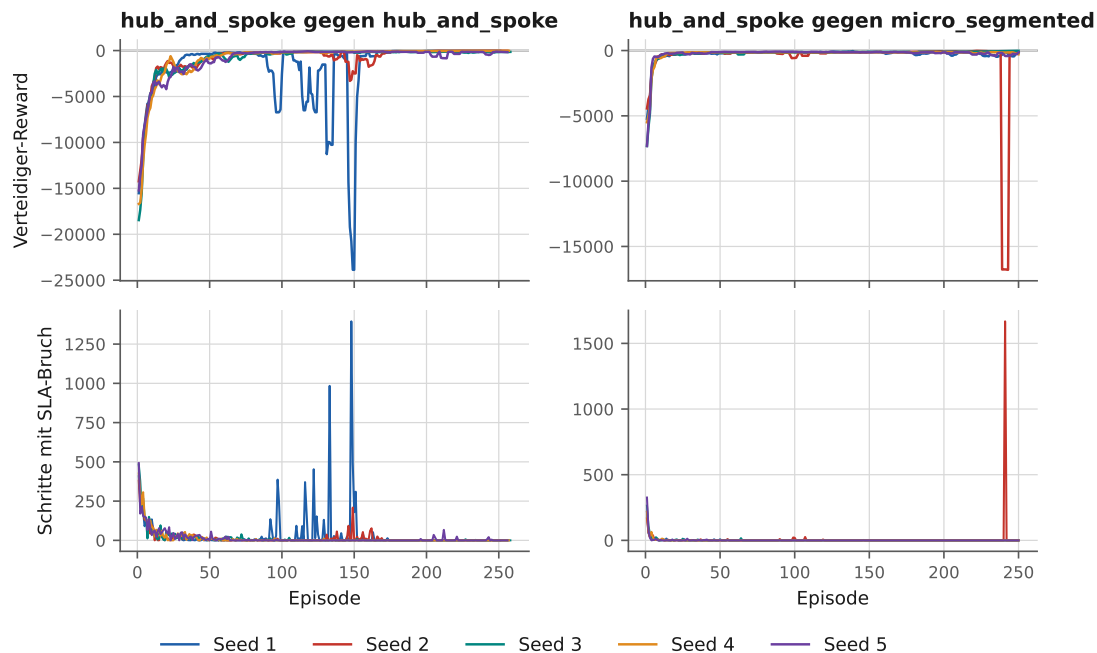


Abbildung 4.3: Verteidiger-Reward und Schritte mit SLA-Bruch für die beiden auffälligen Matchups, je Seed einzeln aufgetragen und nicht gemittelt. Der Reward ist über fünf Episoden geglättet, die Bruchschritte sind es nicht.

Abbildung 4.3 stellt beides nebeneinander. Die Ausschläge der Reward-Kurve und die Spitzen der Bruchschritte liegen an denselben Episoden, und sie beschränken sich auf wenige Läufe: links auf zwei, von denen einer deutlich stärker ausschlägt, rechts auf einen einzigen. Kurze Brüche treten auch in den übrigen Seeds vereinzelt auf; Episoden mit mehreren hundert Bruchschritten bleiben jedoch auf diese wenigen Läufe beschränkt. Im gesamten Training gibt es nur zwölf Episoden mit mehr als 500 Bruchschritten, und acht davon liegen in den ersten beiden Episoden ihres Laufs.

Eine Erklärung liefert der Vergleich innerhalb des Laufs mit der schwersten Episode, also des einen betroffenen Seeds gegen den Micro-Segmentation-Angreifer; als ruhig gelten dabei seine bruchfreien Episoden ab Episode 51, nach der Anfangsphase. In der Episode mit 1668 Bruchschritten bleibt die Zahl der Reimages mit 117 gegenüber 112 im Median praktisch unverändert. Die Zahl der Sperren steigt dagegen von 7 auf 36, und keine einzige von ihnen wird wieder aufgehoben, gegenüber im Median 61 Freigaben in einer ruhigen Episode. Nicht das Neuaufrichten bricht also die Verfügbarkeit, sondern Sperren, die über den größten Teil der Episode Bestand haben; eine gesetzte Sperre wirkt bis zu ihrer Aufhebung fort. Eine erreichte Policy ist demnach nicht dauerhaft stabil, und das Konvergenzkriterium aus Abschnitt 3.7 erfasst solche späten Rückfälle nicht.

Woher der Umschwung in der Policy rührt, lässt sich aus den erhobenen Daten nicht eindeutig bestimmen. Zweierlei ist dazu festzuhalten. Erstens ist es kein allgemeines Verhalten des Verfahrens, sondern eines einzelner Läufe: Von fünf Seeds sind jeweils einer oder zwei betroffen, die übrigen zeigen keine solchen Ausschläge in der Zahl der Bruchschritte. Genau diese Streuung zwischen sonst gleichen Läufen beschreiben Henderson et al. als eigenständige Schwierigkeit von Policy-Gradient-Verfahren; sie zeigen, dass allein die Wahl des Zufallsstartwerts zu deutlich verschiedenen Ergebnisverteilungen führen kann und ein Mittel über wenige Läufe deshalb keine belastbaren Schlüsse erlaubt [23]. Für die vorliegende Arbeit begrenzt das lediglich die Deutung: Die Einbrüche in den Mittelkurven sind keine Aussage über die Topologie, sondern die sichtbare Spur einzelner Seeds.

Zweitens ist der naheliegende Verdacht, eine einzelne Ausreißerepisode habe die Policy unmittelbar verschoben, mit dem Verfahren nicht vereinbar. Genau das verhindert die Begrenzung aus Gleichung (2.7): Sie deckelt, wie weit sich die Policy je Aktualisierung von der bisherigen entfernen darf, unabhängig davon, wie groß der Vorteil ausfällt.

Diese Grenze gilt allerdings nur für die Policy selbst. Der Critic darf seine Prognose nach einer solchen Episode beliebig stark anpassen, und da sich beide in der eingesetzten Umsetzung die vorderen Schichten des Netzes teilen, wirkt seine Korrektur mittelbar auch auf den Actor – an dessen eigener Begrenzung vorbei. Derselbe Mechanismus trägt die rasche Rückkehr: Sobald die Episoden wieder normal ausfallen, fallen die Schätzungen des Critic deutlich negativer aus als das tatsächliche Geschehen. Jede gewöhnliche Episode erscheint dadurch besser als erwartet, das zuletzt gezeigte Verhalten wird verstärkt, und die Policy findet auf das alte Niveau zurück. Was den Ausschlag überhaupt auslöst, erklären diese Überlegungen nicht.

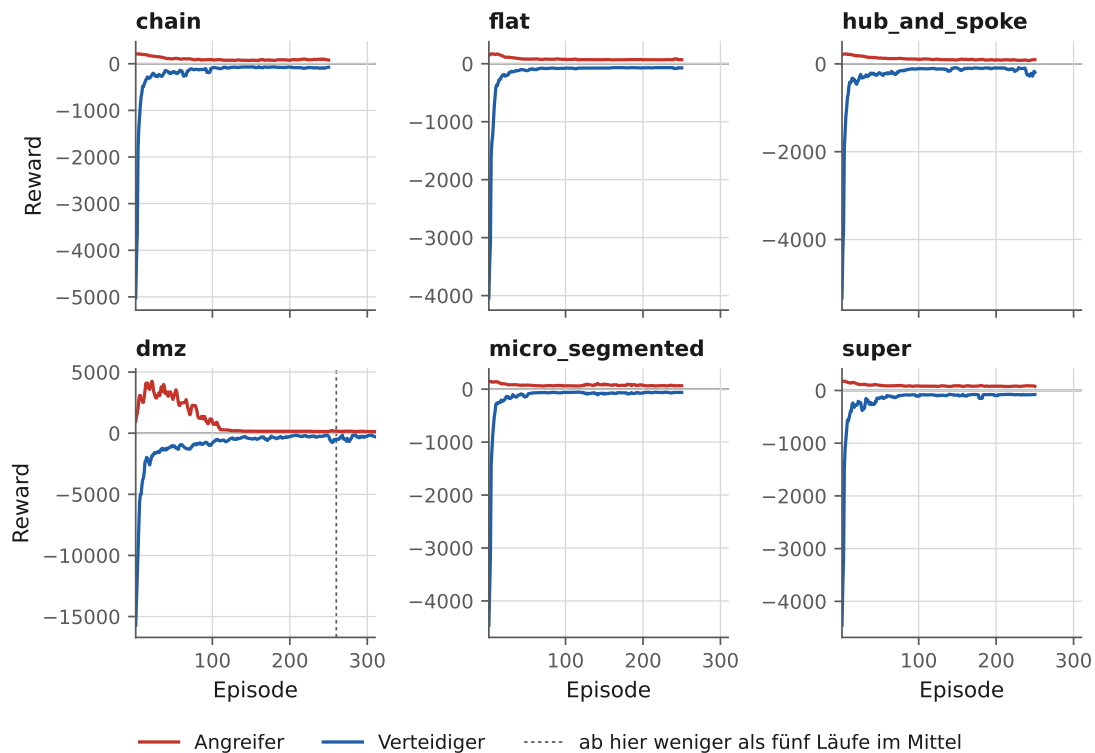


Abbildung 4.4: Verteidiger auf *dmz* gegen alle sechs Angreifer.

*dmz* (Abbildung 4.4) zeigt dasselbe Muster, aber deutlich langsamer. Die Mittelkurve des Angreifers steigt zunächst noch, bis auf 4257 in Episode 22, und geht erst danach zurück: auf 2277 in Episode 75, 946 in Episode 100 und 282 in Episode 110; von dort an ändert sich wenig mehr. Die Mittelkurve des Verteidigers verbessert sich über denselben Zeitraum stetig und liegt in Episode 50 bereits bei  $-1030$ . Da der Angreifer eingefroren ist, lernt er selbst nichts dazu; eine gerichtete Veränderung seiner Kurve geht deshalb auf den Verteidiger zurück, während kurzfristige Schwankungen der Streuung zwischen den Läufen zuzuschreiben sind. Erst nach rund 25 Episoden beginnt dieser, ihn tatsächlich zu bremsen, und dafür braucht er dann etwa 80 weitere. In demselben Zeitraum bricht auch die Crown-Jewel-Quote dieses Matchups ein (Abbildung 4.5); beide Größen bewegen sich also gemeinsam.

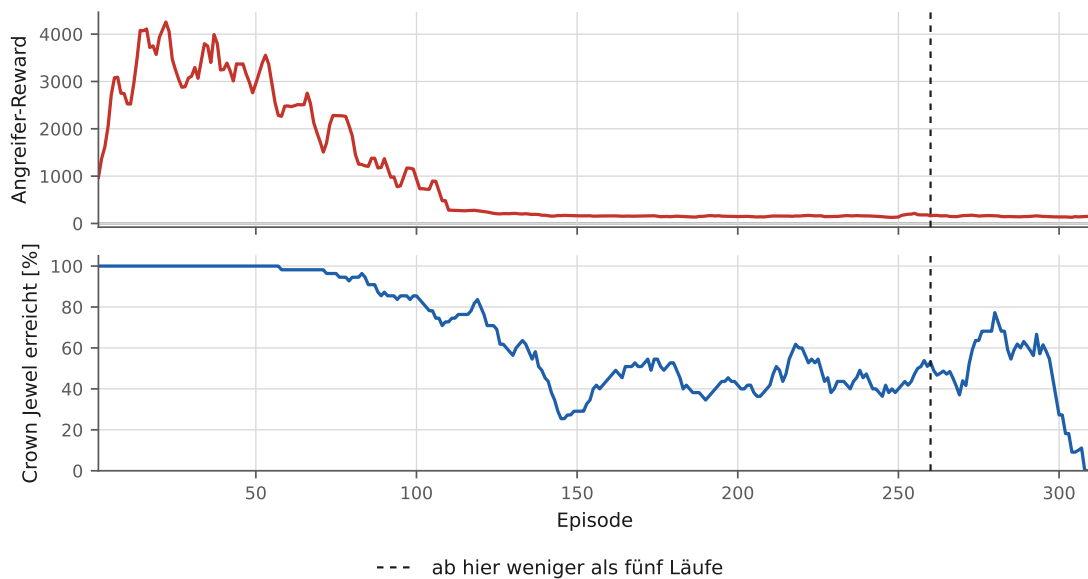


Abbildung 4.5: Angreifer-Reward und Crown-Jewel-Quote im Matchup *dmz* gegen *dmz*.

Bis Episode 55 fällt das Crown Jewel in jeder einzelnen Episode. Danach sinkt die Quote parallel zum Reward, auf 85 Prozent in Episode 100 und auf ihren Tiefstwert von 26 Prozent in Episode 145. Der Rückgang des Rewards ist also kein bloßer Punktverlust, sondern bedeutet, dass der Verteidiger den Angreifer in einem wachsenden Teil der Episoden vom wertvollsten System fernhält.

Danach pendelt die Quote zwischen 26 und 62 Prozent, ohne weiter zu fallen; die noch höheren Werte am rechten Rand beruhen bereits auf weniger als fünf Läufen. Das ist jedoch keine gemeinsame Bewegung aller Läufe, sondern deren Streuung: Über die Episoden 150 bis 250 erreichen die fünf Seeds Quoten von 26, 31, 37, 59 und 70 Prozent, jeder für sich weitgehend stabil. Die gemittelte Kurve bildet damit eine Mischung aus fünf verschiedenen, aber jeweils gefestigten Policies ab, nicht einen Rückfall. Die Evaluation bestätigt das Niveau später mit 36 Prozent (Tabelle 4.5).

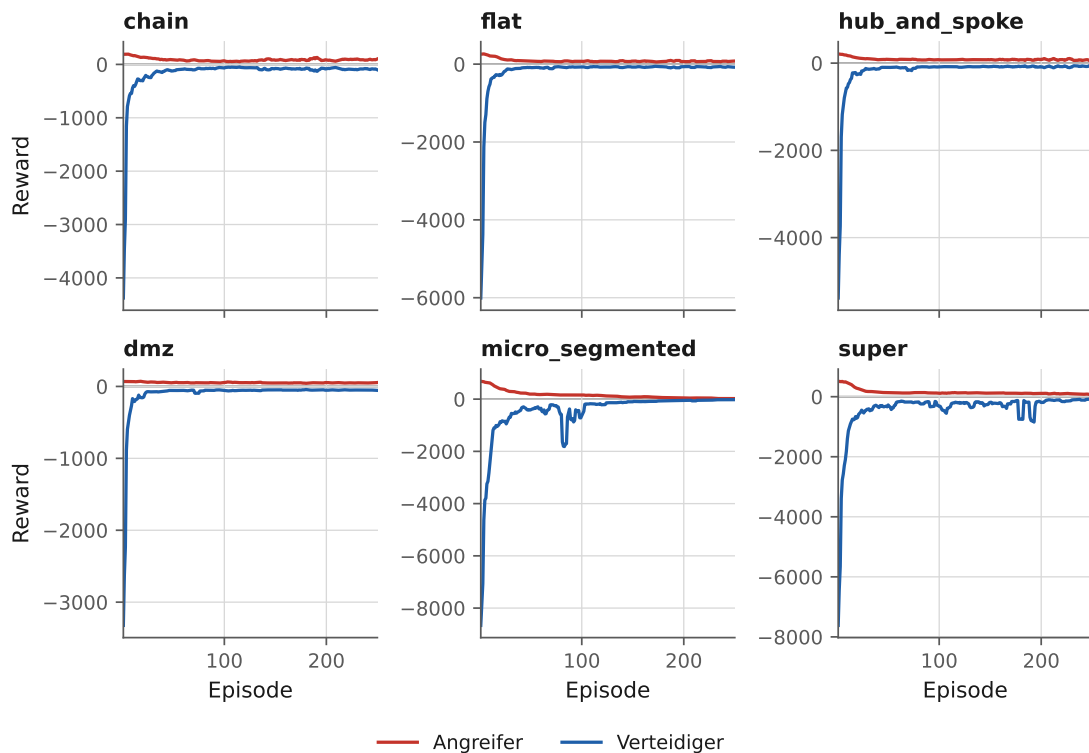


Abbildung 4.6: Verteidiger auf *micro\_segmented* gegen alle sechs Angreifer.

In *micro\_segmented* (Abbildung 4.6) verlaufen beide Kurven im Teilbild des eigenen Angreifers über das gesamte Training aufeinander zu und enden bei den Niveaus +20 und -20. Anders als in *dmz* beginnt der Rückgang sofort: Die Mittelkurve fällt auf 363 in Episode 25 und 193 in Episode 50, danach langsamer weiter. Von den vier Matchups auf der Diagonalen ist dies das einzige, in dem der Angreifer am Ende praktisch nichts mehr erwirtschaftet. Innerhalb der Abbildung ähnelt dem Teilbild des eigenen Angreifers einzig das des Super-Angreifers: Dessen Mittelkurve liegt in Episode 25 mit 208 noch weit über den vier übrigen Teilbildern, die dort 58 bis 127 zeigen, und die Verteidigerkurve beginnt gegen ihn tiefer und holt langsamer auf – in Episode 25 steht sie bei -494, in den vier übrigen Teilbildern zwischen -72 und -228. Das fügt sich in das Bild, denn der Super-Angreifer ist der einzige weitere, der diese Topologie im Training gesehen hat, als letzte Station seines gestuften Trainings; Abschnitt 4.7 nimmt das wieder auf. Die Anfangswerte fallen zugleich am mildesten aus: Das tiefste Startniveau liegt hier bei -3711 gegenüber -8381 in *flat* (Tabelle 4.2). Das passt zu den nur 145 erreichbaren Punkten dieser Topologie.

Über alle vier Abbildungen hinweg zeigt sich damit dasselbe Muster: Das Teilbild des eigenen Angreifers ist stets dasjenige mit dem tiefsten Startwert und dem längsten Weg.

Ein Verteidiger tut sich gegen den Gegner am schwersten, der seine eigene Struktur kennt. Abschnitt 4.7 geht dem nach.

Tabelle 4.2 stellt Start- und Endniveau des Verteidigers gegenüber. Am Start trägt die Diagonale in jeder Zeile mit Abstand den tiefsten Wert; bemerkenswert ist, dass ihre Werte am Ende nicht mehr herausstechen: In *micro\_segmented* ist sie sogar der beste Wert der Zeile, in *hub\_and\_spoke* liegt sie im Mittelfeld. Der anfängliche Rückstand gegen den passgenauen Angreifer ist bis zum Trainingsende also aufgeholt. Die Ausnahme ist erneut *flat* gegen *flat* mit  $-710$ , dem mit Abstand schlechtesten Wert der Tabelle; alle übrigen 23 Zellen liegen zwischen  $-20$  und  $-224$ . Der zweitschlechteste Wert,  $-224$  gegen den Hub-and-Spoke-Angreifer in *dmz*, wird dabei von einem einzelnen Seed bei  $-839$  getragen; die übrigen vier Läufe dieser Zelle enden zwischen  $-58$  und  $-86$ .

Tabelle 4.2: Start- und Endniveau des Verteidiger-Rewards. Werte näher an null sind besser.

| Verteidigt  | Chain                    | Flat                     | H & S                    | DMZ                      | Micro                    | Super                    |
|-------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|
| Flat        | $-1118 \rightarrow -182$ | $-8381 \rightarrow -710$ | $-1694 \rightarrow -193$ | $-1209 \rightarrow -168$ | $-1318 \rightarrow -188$ | $-2349 \rightarrow -197$ |
| Hub & Spoke | $-983 \rightarrow -109$  | $-951 \rightarrow -64$   | $-7910 \rightarrow -78$  | $-789 \rightarrow -81$   | $-946 \rightarrow -117$  | $-1600 \rightarrow -72$  |
| DMZ         | $-1159 \rightarrow -77$  | $-977 \rightarrow -67$   | $-1193 \rightarrow -224$ | $-7265 \rightarrow -157$ | $-978 \rightarrow -62$   | $-866 \rightarrow -76$   |
| Micro-Seg.  | $-735 \rightarrow -76$   | $-1156 \rightarrow -74$  | $-904 \rightarrow -49$   | $-434 \rightarrow -53$   | $-3711 \rightarrow -20$  | $-2799 \rightarrow -83$  |

Diese Zahlen sind allerdings nur innerhalb einer Zeile vergleichbar, nicht zwischen den Zeilen. Der Verteidiger-Reward ist über die Grundkopplung an den des Angreifers gebunden, und der erreichbare Punktevorrat unterscheidet sich je Topologie: In *micro\_segmented* sind vom Einstiegs-knoten aus nur 145 der 235 Punkte überhaupt erreichbar (Abschnitt 3.3.1). Ein Teil des besseren Rewards geht dort also allein darauf zurück, dass weniger zu holen war; welcher Anteil das ist, lässt sich der Zahl nicht entnehmen. Die Aussagen zur Wirksamkeit stützen sich deshalb in Abschnitt 4.6 auf Größen, die von der Reward-Skala unabhängig sind.

## 4.2. Crown-Jewel-Quote im Trainingsverlauf

Die Reward-Kurven laufen in allen 24 Matchups früh auf ein Niveau zu, auf dem sie anschließend verharren. Daraus ließe sich schließen, dass auch die Verteidigung ab

diesem Punkt nicht mehr besser wird. Die Crown-Jewel-Quote widerspricht dem. Sie ist zugleich die Größe, an der die Fragestellung unmittelbar hängt, denn sie sagt, ob der Angreifer das wertvollste System erreicht.

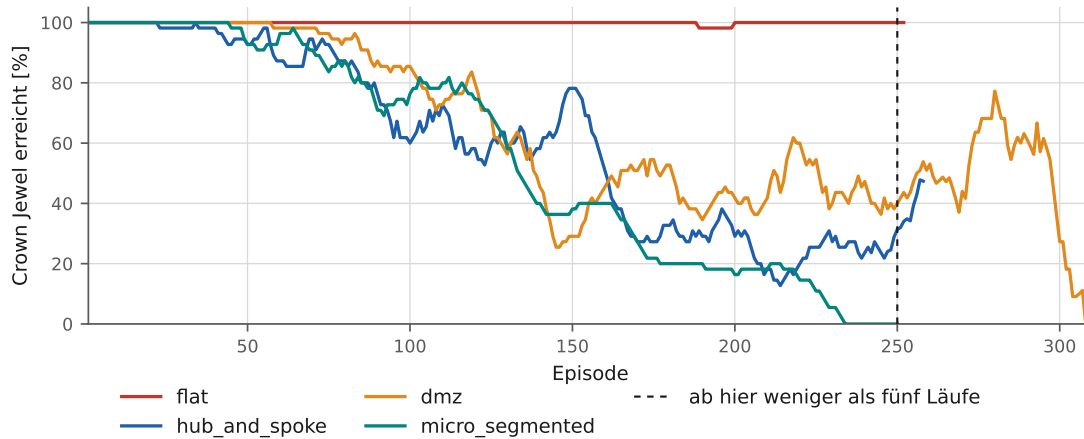


Abbildung 4.7: Anteil der Episoden mit erreichtem Crown Jewel für die vier Matchups auf der Diagonalen. Die Quote ist über ein Fenster von elf Episoden und alle fünf Seeds gebildet, also über 55 Episoden je Punkt.

Abbildung 4.7 trennt die vier Topologien deutlicher als jede andere Größe dieses Kapitels. Zu Beginn erreicht der Angreifer das Ziel überall in jeder Episode. Danach gehen die Verläufe auseinander:

- In *flat* verhindert der Verteidiger über das gesamte Training keinen einzigen Zugriff auf das Crown Jewel, von einer einzigen Episode abgesehen.
- In *micro\_segmented* fällt sie stetig und liegt kurz nach Episode 230 bei null; über die letzten zwanzig Episoden aller fünf Seeds erreicht der Angreifer das Ziel in keiner einzigen.
- *hub\_and\_spoke* und *dmz* liegen dazwischen und kommen auf 28 beziehungsweise 42 Prozent in den letzten zwanzig Episoden.

Bemerkenswert ist der Zeitpunkt. In *micro\_segmented* sinkt die Quote zwischen Episode 150 und 230 von 38 auf 5 Prozent, während der Reward in diesem Abschnitt nur noch von etwa  $-100$  auf  $-20$  steigt. Am Reward gemessen geschieht dort fast nichts mehr, an der Crown-Jewel-Quote gemessen das Entscheidende. Der Grund liegt in der Reward-Konstruktion: Ein Zugriff, der verhindert wird, schlägt sich nur mit dem Wert des Zielknotens von 100 Punkten nieder, weil dem Angreifer nichts genommen, sondern nur nichts gegeben wird (Abschnitt 3.1.1).

In *dmz* erreicht der Angreifer das Crown Jewel auch nach dem Einpendeln regelmäßig,

ohne dass sich das im Reward niederschlägt (Abbildung 4.5, bei den Lernkurven). Der Grund liegt in der Kopplung aus Verlust und Rückeroberung (Abschnitt 3.6). Über die Episoden 150 bis 250 verliert der Angreifer in den Episoden mit erreichtem Crown Jewel im Median 2035 Punkte durch neu aufgesetzte Knoten, gegenüber 1552 in den übrigen. Er nimmt das Ziel also, verliert es wieder und nimmt es erneut, und jede dieser Runden gleicht sich aus. Am Ende trennt die Episoden mit erreichtem Crown Jewel von denen ohne ein Unterschied von 175 zu 126 Punkten – weniger als die 100 Punkte, die das gehaltene Crown Jewel allein wert wäre.

Beide Größen messen demnach Verschiedenes: Die Crown-Jewel-Quote sagt, ob der Angreifer das Ziel überhaupt erreicht, der Reward, ob er es halten kann. Sie bewegen sich dennoch in dieselbe Richtung. Ordnet man die fünf Seeds nach ihrer Quote, so steigt der Reward-Median mit ihr, von 123 bei 26 Prozent bis 188 bei 70 Prozent. Erst aus beiden zusammen ergibt sich ein Bild, wie in Abschnitt 3.8 angelegt.

### 4.3. Gehaltene Knoten im Trainingsverlauf

Ob das Crown Jewel gefallen ist, beantwortet noch nicht, wie weit der Angreifer insgesamt vorgedrungen ist. Ein Netz, in dem er acht Knoten hält, und eines, in dem er auf dem Einstiegsknoten feststeht, stehen in dieser Quote gleich, sofern das Ziel in beiden Fällen unangetastet blieb. Dafür dient die dritte Zielgröße aus Abschnitt 3.8, die größte Zahl gleichzeitig kompromittierter Knoten je Episode. Der Einstiegsknoten zählt dabei mit: Ein Wert von 1,0 bedeutet, dass der Angreifer über ihn nicht hinausgekommen ist.

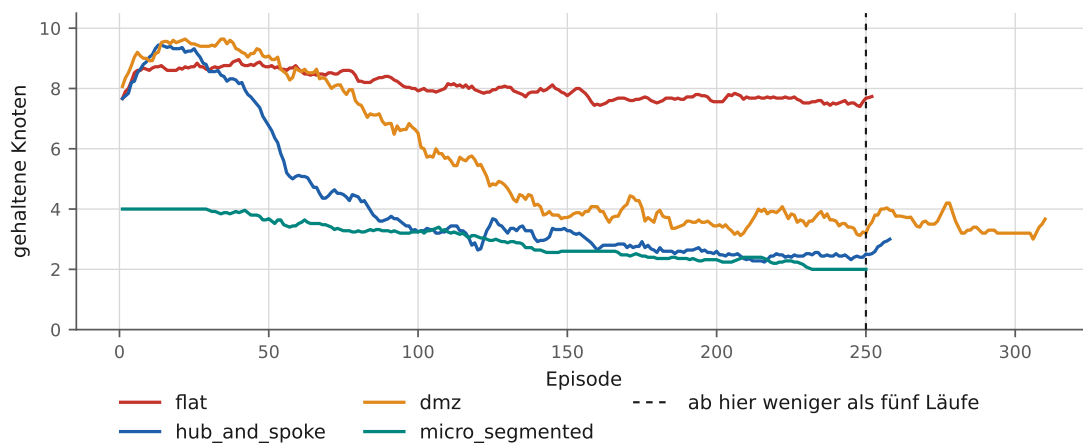


Abbildung 4.8: Größte Zahl gleichzeitig gehaltener Knoten je Episode für die vier Matchups auf der Diagonalen, gemittelt über fünf Seeds und geglättet über fünf Episoden. Von zehn Knoten im Netz.

In *dmz* und *hub\_and\_spoke* steigt die Mittelkurve zunächst noch an und erreicht 9,6 beziehungsweise 9,4. Der Angreifer hält dort in den meisten Episoden das gesamte Netz. Erst danach setzt der Rückgang ein. *micro\_segmented* beginnt dagegen bei 4,0 und hält damit ebenfalls alles, was von dort aus überhaupt erreichbar ist (Abschnitt 3.3.1).

Die Abbildung zeigt nur die Diagonale, alle 24 Matchups zeigt Abschnitt A.4. Zieht man sie alle heran, so liegen 21 von ihnen am Ende des Trainings zwischen 1,8 und 2,5. Der Verteidiger drückt den Angreifer also im Regelfall auf ein bis zwei eroberte Rechner zurück. Drei Zellen weichen davon ab, und zwei davon sind in der Abbildung zu sehen: *dmz* gegen *dmz* mit 3,4 und *flat* gegen *flat* mit 7,5, dazu *flat* gegen den Super-Angreifer mit 3,0.

Auffällig ist davon nur *flat* gegen *flat*: Auch dort hält der Angreifer anfangs fast das gesamte Netz, doch anders als in *dmz* und *hub\_and\_spoke* setzt der Rückgang nie ein – die Kurve bleibt über das gesamte Training nahe ihrem Höchstwert, der Angreifer behält dauerhaft rund drei Viertel des Netzes.

## 4.4. Konvergenz

Das in Abschnitt 3.7 beschriebene Kriterium wird von allen 120 Läufen erfüllt, im Median in Episode 42. Der früheste überhaupt mögliche Wert ist Episode 39, der früheste gemessene Episode 40. Die Mediane je verteidigter Topologie liegen zwischen 41 und 42,5 und unterscheiden sich damit nicht sonderlich voneinander. Sämtliche Läufe gelten also nahezu unmittelbar an der unteren Grenze des Kriteriums als konvergiert, obwohl sie bis zur Episode 250 laufen.

Dieser Befund ist kein Ergebnis über Topologien, sondern eines über das Kriterium. Es misst die Restunruhe der letzten Episoden im Verhältnis zur bisherigen Reward-Spanne des Laufs. Genau diese Spanne wird durch die ersten Episoden sehr groß, die je nach Lauf im fünfstelligen Bereich liegen, und jede spätere Verbesserung wirkt daran gemessen klein. Ein Lauf gilt deshalb bereits als ruhig, während er sich tatsächlich noch verbessert.

*dmz* gegen *dmz* zeigt das exemplarisch: Die Spanne wird von der ersten Episode bestimmt, die je nach Seed zwischen rund –15 000 und –31 000 liegt; fünf Prozent davon sind 750 bis 1550 Punkte. Zu dem Zeitpunkt, an dem das Kriterium zutrifft, im Median in Episode 49, steht die Mittelkurve erst bei rund –1000; bis zum Trainingsende verbessert sich der Reward weiter, auf Endniveaus zwischen –133 und –196 je Seed.

Diese Verbesserung verteilt sich über zweihundert Episoden und bleibt damit je Fenster unter der Schwelle. Das Kriterium trennt das Ende der Unruhe nicht vom Ende der Verbesserung.

Eine Spur des in Abschnitt 4.1 beschriebenen Musters bleibt dennoch erkennbar. In drei der vier Zeilen trägt die Diagonale den höchsten Wert, in *micro\_segmented* teilt sie ihn mit der Zelle gegen den Super-Angreifer: *dmz* gegen *dmz* konvergiert im Median erst in Episode 49, *hub\_and\_spoke* gegen sich selbst in 47, *micro\_segmented* in 46 und *flat* in 44, gegenüber 40 bis 46 in allen übrigen Zellen (Tabelle 4.3, jeweils der Wert vor dem Schrägstrich). Die Richtung stimmt also mit den übrigen Größen überein, die Spreizung ist aber so gering, dass sich darauf keine Aussage stützen lässt.

Dass die Topologien sich nicht trennen, könnte an der Wahl der Schwelle liegen. Das Kriterium besteht aus zwei voneinander unabhängigen Bedingungen, die Verschiedenes messen: Die Streuung erfasst das Rauschen der Kurve, der Trend erfasst, ob die Verbesserung aufgehört hat. Beide erhalten in Abschnitt 3.7 denselben Wert von 0,05 der Spanne, was eine Konvention ist und keine Notwendigkeit.

Nachgerechnet zeigt sich, dass davon nur eine der beiden Bedingungen wirksam ist. Die Streuung ist in fast allen 120 Läufen von dem Moment an erfüllt, ab dem das Kriterium überhaupt geprüft wird; scheitert eine Prüfung vor der Konvergenz, dann in aller Regel am Trend. Was den Zeitpunkt bestimmt, ist also allein der Trendtest, und dessen Toleranz von 5 Prozent der Gesamtspanne ist weit gefasst.

Als Gegenprobe wurde die Schwelle des Trends deshalb auf 0,01 gesenkt, die der Streuung blieb bei 0,05. Am Kriterium selbst ändert das nichts, nur an einer seiner beiden Zahlen.

Tabelle 4.3: Konvergenzepisode je Matchup als Median der fünf einzeln geprüften Seeds; vor dem Schrägstrich beim Trendtest mit Schwelle 0,05, dahinter mit 0,01. Hervorgehoben ist die Diagonale.

| Verteidigt  | Chain   | Flat           | H & S          | DMZ            | Micro          | Super   |
|-------------|---------|----------------|----------------|----------------|----------------|---------|
| Flat        | 41 / 58 | <b>44 / 72</b> | 43 / 55        | 41 / 59        | 43 / 58        | 42 / 76 |
| Hub & Spoke | 41 / 50 | 41 / 55        | <b>47 / 88</b> | 40 / 51        | 41 / 50        | 44 / 53 |
| DMZ         | 41 / 49 | 42 / 53        | 41 / 59        | <b>49 / 95</b> | 41 / 59        | 42 / 68 |
| Micro-Seg.  | 40 / 57 | 42 / 49        | 41 / 50        | 40 / 52        | <b>46 / 66</b> | 46 / 60 |

Auch mit dem strengeren Trendtest erfüllen alle 120 Läufe das Kriterium; der Median

wandert von Episode 42 auf 58,5. Die Topologien trennen sich weiterhin nicht: Ihre Mediane liegen zwischen 53 und 62, die Quartile der 30 Läufe je Topologie reichen von 55 bis 77 in *flat* und von 50 bis 62 in *micro\_segmented* und überlappen damit fast vollständig.

Sichtbar wird dagegen etwas anderes. Betrachtet man nicht die Zeilen gegeneinander, sondern die Zellen innerhalb einer Zeile, so tritt die Diagonale hervor (Tabelle 4.3, Werte nach dem Schrägstrich). Ihre Mediane liegen zwischen Episode 66 und 95, die der übrigen 20 Zellen zwischen 49 und 76. In drei der vier Zeilen ist die Diagonale die Zelle mit der spätesten Konvergenz, in *flat* die zweitspäteste hinter der Zelle gegen den Super-Angreifer. Beim Kriterium aus Abschnitt 3.7 war dieser Unterschied kaum erkennbar. Der Befund fügt sich damit in die übrigen Größen ein, die in Abschnitt 4.7 zusammengeführt werden.

Angegeben ist hier der Median der fünf Seeds einer Zelle, nicht ihr Mittel. Einzelne Läufe konvergieren sehr viel später als die übrigen vier ihrer Zelle, etwa *flat* gegen den Hub-and-Spoke-Angreifer mit 48, 49, 55, 63 und 164. Das Mittel dieser Zelle läge bei 76 und damit über der Diagonalen, obwohl vier ihrer fünf Läufe zu den schnellsten der Zeile gehören.

Für die Frage nach der Topologie bleibt es beim Ergebnis: Bei fünf Seeds je Zelle ist ein Einfluss auf die Lerngeschwindigkeit nicht nachweisbar. Ausgeschlossen ist er damit nicht, die Daten tragen den Nachweis nur nicht.

Als Maß der Lerneffizienz ist der Konvergenzzeitpunkt in dieser Form deshalb nur eingeschränkt geeignet. Dass die Läufe ihr volles Schrittbudget erhielten und nicht beim Erfüllen des Kriteriums abgebrochen wurden (Abschnitt 3.7), erweist sich hier als nützlich: Beide Varianten ließen sich nachträglich auf vollständige Kurven anwenden, und der Vergleich in Tabelle 4.3 wurde dadurch überhaupt erst möglich. Wäre wie im Referenzlauf abgebrochen worden, stünde am Ende eine einzelne Zahl, deren Zustandekommen sich nicht mehr prüfen ließe.

Der Vergleich der Topologien stützt sich in dieser Arbeit deshalb auf das erreichte Niveau (Abschnitt 4.1 bis Abschnitt 4.3) und auf die Wirksamkeit der fertig trainierten Policy (Abschnitt 4.6). Abschnitt 4.9 schlägt ein Maß vor, das ohne die Normierung auf die Gesamtspanne auskommt.

## 4.5. Aktionsverhalten des Verteidigers

Über alle 120 Läufe gemittelt sinkt der Anteil der Sperren von 16 Prozent in den ersten 25 Episoden auf 2 Prozent ab Episode 101. Der Verteidiger gibt das Sperren also überwiegend auf, jedoch nicht vollständig: In 87,8 Prozent aller Episoden ist zu jedem Zeitpunkt mindestens ein Knoten aktiv abgeschildert. Zurück geht die Häufigkeit, mit der Sperren umgesetzt werden, nicht ihr Bestand.

Zugleich steigt der Anteil ungültiger Aktionen von 38 auf 62 Prozent. Eine ungültige Aktion kostet nichts und überspringt den Zug (Abschnitt 3.1.1); sie ist damit der billigste Weg, nichts zu tun. Der Aktionsraum des Verteidigers enthält keine ausdrückliche Aktion dafür. Dasselbe gilt für Freigaben auf Knoten, die ohnehin nicht gesperrt sind: Sie sind folgenlos. Freigaben machen über alle Läufe hinweg 36,8 Prozent aller Aktionen aus und stehen damit nach den ungültigen Aktionen an zweiter Stelle. Beide Anteile sind deshalb nicht als Verhalten zu lesen, sondern zum größten Teil als Untätigkeit; einzelne Freigaben mögen berechtigt sein, aus den erhobenen Zählungen lässt sich das jedoch nicht trennen. Aussagekräftig bleiben Sperren und Reimages sowie ihr Verhältnis zueinander.

Abbildung 4.9 zeigt beide Aktionsarten im Verlauf. Die Bewegung geht in allen vier Topologien in dieselbe Richtung, aber unterschiedlich weit.

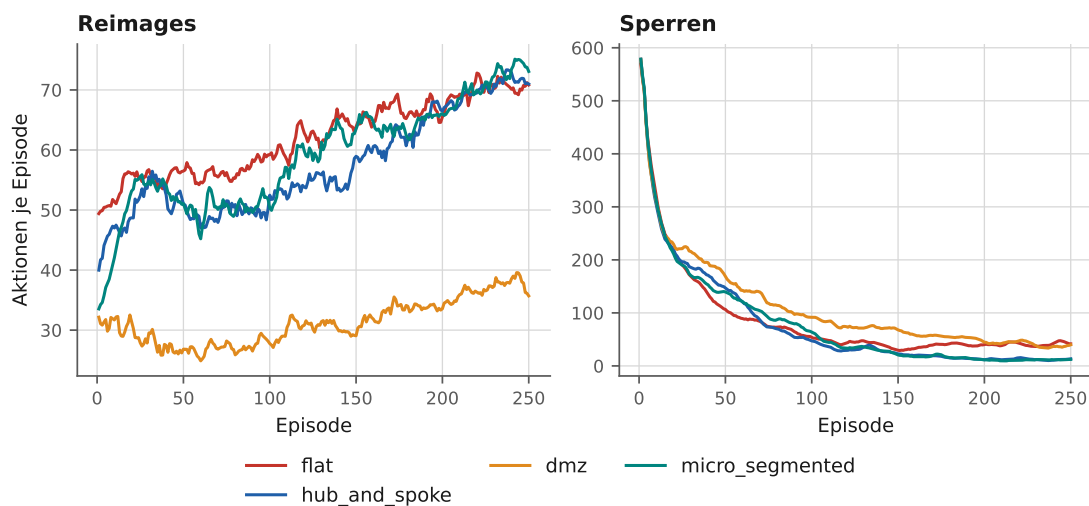


Abbildung 4.9: Reimages und Sperren je Episode, gemittelt über die sechs Angreifer und fünf Seeds einer Topologie und geglättet über fünf Episoden. Die Kurven enden beim kürzesten der 30 Läufe, weil sich die Zusammensetzung des Mittels danach ändern würde.

Die Sperren gehen überall zurück, von mehreren hundert je Episode zu Beginn auf 11

bis 45 am Ende. Der Rückgang ist aber ungleich: *hub\_and\_spoke* und *micro\_segmented* landen bei 11 beziehungsweise 12, *dmz* bei 37 und *flat* bei 45. Umgekehrt steigen die Reimages von 33 bis 51 auf 72 bis 75, mit *dmz* als Ausnahme, dessen Zahl mit 36 bei etwa der Hälfte der übrigen liegt.

Der Verteidiger verlagert also sein Gewicht vom Sperren auf das Neuaufsetzen, und zwar unabhängig von der Topologie. Bemerkenswert ist, dass ausgerechnet *flat* am Ende am häufigsten sperrt. Die naheliegende Deutung, der Verteidiger gebe dort das Sperren auf, trifft also nicht zu.

Wie gut diese Sperren gesetzt sind, zeigt der Abwehrbonus (Abbildung 4.10). Er ist so gebaut, dass eine Sperre gegen eine bestehende Bedrohung positiv zählt und eine Sperre ohne Bedrohung negativ (Gleichung 3.1); sein Vorzeichen zeigt damit an, ob die Sperren eines Laufs überwiegend gegen eine bestehende Bedrohung standen.

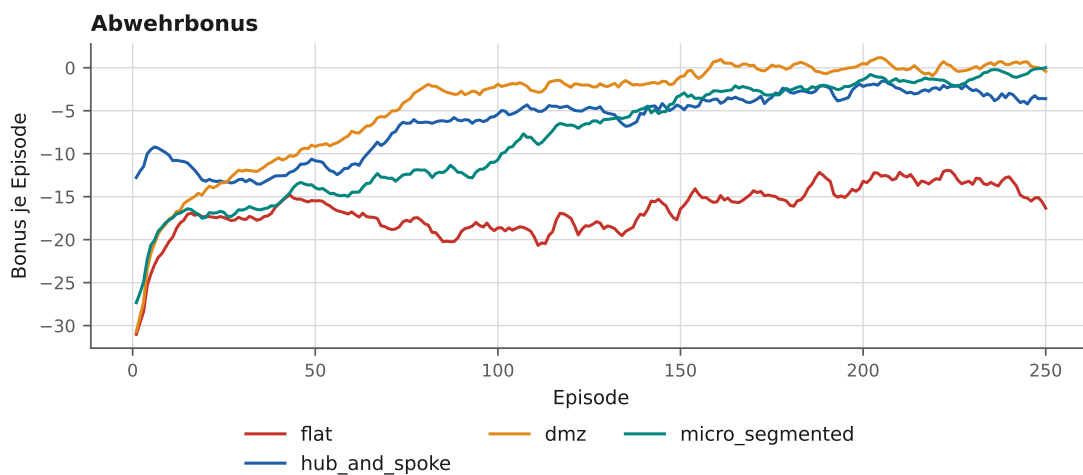


Abbildung 4.10: Abwehrbonus je Episode, gebildet wie in Abbildung 4.9. Positive Werte bedeuten, dass die gesetzten Sperren überwiegend gegen eine bestehende Bedrohung standen.

Alle vier Topologien beginnen deutlich im Negativen, weil ein untrainierter Verteidiger wahllos sperrt. Der anschließende Anstieg gegen null ist für sich genommen aber noch kein Zeichen besseren Sperrrens, denn der Bonus folgt zunächst schlicht dem Sperrvolumen aus Abbildung 4.9: Wo kaum noch gesperrt wird, fällt auch kaum noch Bonus an. *hub\_and\_spoke* und *micro\_segmented* erreichen die Nähe der Null mit Endniveaus von  $-3,6$  und  $-0,3$  auf genau diesem Weg. Sie legen das Sperren fast vollständig ab, und die wenigen verbliebenen Sperren stehen unverändert überwiegend nicht gegen eine bestehende Bedrohung.

Anders *dmz*, dessen Endniveau mit  $+0,6$  als einziges im Positiven liegt. Dort pendelt sich das Sperrvolumen bei rund 30 bis 40 Sperren je Episode ein, und diese treffen

zunehmend: Ab Episode 150 fällt in gut der Hälfte der Episoden ein positiver Bonus an, gegen drei der sechs Angreifer ist die Bilanz im Mittel sogar positiv (Abschnitt A.6). Es ist zugleich die Topologie, in der sich das Sperren auszahlt: Vergleicht man je Matchup die stärker mit den schwächer gesperrten Episoden nach Episode 150, so steht der Verteidiger in den stärker gesperrten im Median 13 Punkte besser da und der Angreifer 11 Punkte schlechter. Die Zonenübergänge dieser Topologie sind Engpässe; einer Sperre dort kann der Angreifer nicht ausweichen. Das erklärt zugleich die Ausnahmerolle beim Neuaufsetzen aus Abbildung 4.9: Wer den Angreifer an den Engpässen abfängt, hat weniger neu aufzusetzen.

*flat* bleibt im Endniveau bei  $-14,2$  stehen. Mit dem Ausreißer aus Tabelle 4.2 hat das allerdings nichts zu tun: Ausgerechnet gegen den eigenen Angreifer sperrt der Verteidiger ab Episode 150 im Median gar nicht mehr, sein Bonus liegt bei null (Abschnitt A.6). Der Ausreißer ist schlicht die Grundkopplung: Der eigene Angreifer erwirtschaftet am Ende 664 Punkte (Tabelle 4.1), und der Verteidiger spiegelt sie. Die Belastung stammt aus den übrigen fünf Matchups, in denen der Verteidiger das Sperren nicht ablegt, obwohl es sich dort nach Gleichung (3.1) kaum lohnen kann: Jeder Knoten in *flat* hat neun Vorgänger, und weil der Angreifer in diesen Matchups auf zwei bis drei Knoten zurückgedrängt ist, sind fast alle davon unkompromittiert – eine stehende Sperre zählt damit beinahe mit dem vollen negativen Beitrag, und genau das zeigt der anhaltend negative Bonus. Anders als in *dmz* zahlt sich das nicht aus: In den stärker gesperrten Episoden steht der Verteidiger im Median 16 Punkte schlechter, während der Angreifer kaum gebremst wird – die Zahl seiner gehaltenen Knoten ändert sich nicht, denn im vollvermaschten Netz weicht er einer Sperre schlicht auf einen anderen Knoten aus. Der Hebel des Ratenverhältnisses aus Abschnitt 3.6.1 greift hier nicht, denn er setzt voraus, dass die Sperre einen Knoten schützt, den der Angreifer andernfalls hielte. Warum die Policy das Verhalten dennoch beibehält, lässt sich aus den Daten nicht bestimmen; der negative Abwehrbonus von rund 18 Punkten je Episode ist klein gegen die Streuung des Episoden-Rewards, und entsprechend schwach ist das Signal, das er dagegen setzt. Beide Vergleiche sind zudem Beobachtungen innerhalb der gelernten Policies, keine kontrollierten Eingriffe.

## 4.6. Wirksamkeit der trainierten Verteidiger

In der Evaluation (Abschnitt 3.9) gibt es keinen Verlauf mehr: Je Zelle wird erst der Median über die 25 Episoden eines Seeds gebildet, dann der Median über die fünf Seeds; Quoten fassen alle 125 Episoden einer Zelle zusammen. Die Stufe ohne Verteidiger

liefert den Bezugspunkt für den Restanteil, der zufällige Verteidiger trennt den Beitrag des Trainings vom zufälligen Handeln.

Der Restanteil liegt für das trainierte Modell zwischen 0,4 und 12,1 Prozent und damit in *jeder* der 24 Zellen unter dem des Zufallsverteidigers, der zwischen 5 und 37 Prozent erreicht (Tabelle 4.4). Das Training bewirkt also durchgängig mehr als zufälliges Handeln, und der Abstand ist deutlich: Im Referenzlauf hatte der zufällige Verteidiger noch nahezu gleichauf mit dem trainierten Modell gelegen (Abschnitt 3.1.3); nach dem Umbau trennt beide in jeder Zelle ein klarer Abstand, am größten in *micro\_segmented*: Dort liegt der Restanteil des trainierten Modells im Mittel 16,4 Prozentpunkte unter dem des Zufallsverteidigers. Einschränkungen dieser Schlussfolgerung behandelt Abschnitt 4.9.

Tabelle 4.4: Restanteil des Angreifer-Rewards gegen den trainierten Verteidiger, in Prozent des ungeschützten Netzes; dieses selbst liegt definitionsgemäß bei 100. In Klammern der Wert des Zufallsverteidigers. Niedriger ist besser.

| Verteidigt  | Chain    | Flat      | H & S    | DMZ       | Micro    | Super    |
|-------------|----------|-----------|----------|-----------|----------|----------|
| Flat        | 8,8 (14) | 12,1 (19) | 9,0 (16) | 7,6 (12)  | 8,7 (15) | 3,0 (9)  |
| Hub & Spoke | 5,5 (11) | 6,4 (15)  | 0,5 (10) | 3,9 (6)   | 9,8 (19) | 0,4 (5)  |
| DMZ         | 4,1 (9)  | 3,1 (7)   | 3,8 (10) | 2,4 (10)  | 4,7 (9)  | 3,6 (7)  |
| Micro-Seg.  | 4,3 (11) | 4,9 (16)  | 3,0 (22) | 10,2 (12) | 1,2 (37) | 3,6 (27) |

Die Crown-Jewel-Quote bestätigt das Bild und schärft es (Tabelle 4.5). Gegen das trainierte Modell erreicht der Angreifer das Crown Jewel in nahezu allen Zellen deutlich seltener als gegen den Zufallsverteidiger. In *micro\_segmented* fällt die Quote bei drei von sechs Angreifern auf exakt null – beim eigenen Angreifer, der gegen den Zufallsverteidiger noch jede Episode gewann; der Chain- und der Flat-Angreifer kommen in je einer einzigen der 125 Episoden einer Zelle durch. Das andere Extrem ist erneut *flat* gegen den eigenen Angreifer mit 99,2 Prozent: Hier erreicht der Angreifer das Crown Jewel praktisch in jeder Episode, obwohl ein trainierter Verteidiger im Netz steht.

Tabelle 4.5: Anteil der Episoden mit erreichtem Datenbankserver gegen den trainierten Verteidiger, in Prozent. Die kleine Zeile darunter nennt Zufallsverteidiger und ungeschütztes Netz.

| Verteidigt  | Chain  | Flat    | H & S   | DMZ    | Micro   | Super   |
|-------------|--------|---------|---------|--------|---------|---------|
| Flat        | 51,2   | 99,2    | 53,6    | 25,6   | 49,6    | 84,0    |
| Zufall/ohne | 92/92  | 100/100 | 94/100  | 55/66  | 90/95   | 100/100 |
| Hub & Spoke | 8,8    | 4,0     | 27,2    | 2,4    | 20,0    | 2,4     |
| Zufall/ohne | 46/68  | 50/77   | 100/100 | 14/94  | 54/69   | 57/90   |
| DMZ         | 8,0    | 0,8     | 1,6     | 36,0   | 0,8     | 1,6     |
| Zufall/ohne | 46/78  | 43/98   | 58/95   | 99/100 | 31/98   | 25/99   |
| Micro-Seg.  | 0,8    | 0,8     | 0,0     | 0,0    | 0,0     | 15,2    |
| Zufall/ohne | 39/100 | 52/100  | 6/56    | 0/44   | 100/100 | 100/100 |

Die Zahl der gehaltenen Knoten weist in dieselbe Richtung (Tabelle 4.6). Gegen das trainierte Modell hält der Angreifer im Median 2 von zehn Knoten, den Einstiegsknoten mitgezählt, also faktisch einen einzigen eroberten. Ohne Verteidiger sind es je nach Zelle 3 bis 10, gegen den Zufallsagenten 2 bis 8. Ausnahmen beim trainierten Modell sind wiederum *flat* gegen *flat* mit 7 Knoten sowie *flat* gegen den Super-Angreifer und *dmz* gegen *dmz* mit je 3.

Tabelle 4.6: Größte Zahl gleichzeitig gehaltener Knoten gegen den trainierten Verteidiger, als Median einer Zelle; die kleine Zeile darunter nennt Zufallsverteidiger und ungeschütztes Netz. Von zehn Knoten, der Einstiegsknoten zählt mit.

| Verteidigt    | Chain | Flat | H & S | DMZ  | Micro | Super |
|---------------|-------|------|-------|------|-------|-------|
| Flat          | 2     | 7    | 2     | 2    | 2     | 3     |
| Zufall / ohne | 3/7   | 7/10 | 3/7   | 3/9  | 3/8   | 4/10  |
| Hub & Spoke   | 2     | 2    | 2     | 2    | 2     | 2     |
| Zufall / ohne | 3/8   | 3/8  | 8/10  | 3/9  | 3/3   | 3/10  |
| DMZ           | 2     | 2    | 2     | 3    | 2     | 2     |
| Zufall / ohne | 3/8   | 3/8  | 3/8   | 8/10 | 3/5   | 3/9   |
| Micro-Seg.    | 2     | 2    | 2     | 2    | 2     | 2     |
| Zufall / ohne | 3/4   | 3/4  | 3/4   | 2/3  | 4/4   | 4/4   |

Beide Größen zusammen zeigen, dass die drei segmentierten Topologien einander im Ergebnis ähnlicher sind, als der Reward vermuten ließ, und dass *flat* klar abfällt. Bemerkenswert ist dabei, dass Restanteil und Crown-Jewel-Quote nicht dieselbe Rangfolge liefern. *hub\_and\_spoke* gegen den eigenen Angreifer hat mit 0,5 Prozent einen der niedrigsten Restanteile der Tabelle, zugleich aber eine Crown-Jewel-Quote von 27,2 Prozent. Der Restanteil ist ein Verhältnis zum ungeschützten Netz; wo dort sehr viel zu holen war, sieht eine Verteidigung relativ gut aus, obwohl das Crown Jewel in mehr als jeder vierten Episode fällt. Für eine Aussage über das tatsächliche Risiko ist deshalb die absolute Größe maßgeblich.

Schließlich die Gegenrechnung. Tabelle 4.7 stellt den Aufwand beider Verteidiger gegenüber. Der Zufallsagent setzt im Median rund 650 Sperren je Episode, also bei etwa jedem dritten der 2000 Schritte, und verletzt dadurch in 99,7 bis 100 Prozent der Episoden das SLA; sein Reward liegt entsprechend um das Vierzig- bis Hundertsechzigfache unter dem des trainierten Modells. Dieses sperrt fast nie und bricht das SLA nur selten. Der Unterschied liegt damit nicht in der Menge der Eingriffe, sondern in ihrer Auswahl.

Tabelle 4.7: Aufwand des trainierten Verteidigers je Evaluationsepisode, in Klammern der Zufallsverteidiger. Mediane über alle Seeds und Angreifer.

| Verteidigt  | Sperren  | Reimages     | SLA-Bruch      | Reward       |
|-------------|----------|--------------|----------------|--------------|
| Flat        | 10 (650) | 29,0 (21,0)  | 14,9 % (100 %) | −191 (−8016) |
| Hub & Spoke | 1 (650)  | 103,0 (34,0) | 0,8 % (100 %)  | −91 (−8356)  |
| DMZ         | 19 (654) | 27,0 (16,0)  | 4,5 % (99,7 %) | −78 (−6934)  |
| Micro-Seg.  | 0 (648)  | 103,5 (34,0) | 0,1 % (100 %)  | −53 (−8497)  |

Auffällig ist die Zeile *micro\_segmented*: null Sperren im Median, dafür 103,5 Reimages und damit dreimal so viele wie beim Zufallsagenten. Der wirksamste Verteidiger dieser Untersuchung sperrt also gar nicht, sondern setzt konsequent neu auf. Dasselbe Muster zeigt *hub\_and\_spoke*. Abschnitt 4.8 geht der Frage nach, ob das eine Eigenschaft der Topologien ist oder eine des Reward-Entwurfs.

## 4.7. Einfluss der Angreiferherkunft

Die Topologie, auf der ein Angreifer trainiert wurde, beeinflusst das Ergebnis sichtbar. In drei der vier verteidigten Topologien erreicht derjenige Angreifer die höchste Crown-Jewel-Quote, der auf ebendieser Topologie entstanden ist: *flat* mit 99,2 Prozent, *hub\_and\_spoke* mit 27,2 und *dmz* mit 36,0. Wie deutlich der Abstand zur zweithöchsten Quote derselben Zeile ausfällt, ist allerdings sehr verschieden (Tabelle 4.5): In *dmz* folgt der Chain-Angreifer erst bei 8,0 Prozent, in *hub\_and\_spoke* der Micro-Segmentation-Angreifer bei 20,0, und in *flat* liegt der Super-Angreifer bei 84,0.

Der Vorteil ist also kein allgemeiner Fähigkeitsvorsprung, sondern Passgenauigkeit: Der Angreifer kennt die Wege dieser Struktur, weil er auf ihr gelernt hat. Umgekehrt bricht seine Leistung ein, sobald er auf eine fremde und stärker segmentierte Struktur trifft.

Die Ausnahme ist *micro\_segmented* selbst. Dort scheitert der eigene Angreifer vollständig (0,0 Prozent), während der Super-Angreifer als einziger mit 15,2 Prozent durchkommt. Die Vielfalt seiner Trainingsstationen vor der letzten auf *micro\_segmented* verschafft ihm hier offenbar einen Vorteil gegenüber dem Spezialisten: Wo eine Struktur nur wenige

und enge Wege lässt, scheint breites Wissen mehr zu helfen als spezialisiertes.

In einzelnen Konstellationen kann breites Wissen für einen Verteidiger also gefährlicher sein als der Spezialist. Für drei der vier Topologien gilt jedoch das Gegenteil: Dort liefert der eigens für die verteidigte Topologie trainierte Angreifer die besseren Ergebnisse, und der Generalist bleibt teils weit zurück – gegen die Verteidiger von *hub\_and\_spoke* und *dmz* etwa mit 2,4 und 1,6 Prozent gegenüber 27,2 und 36,0 Prozent der Spezialisten (Tabelle 4.5). Auch die Ausnahme trägt zudem einen Vorbehalt: Das gestufte Training des Super-Angreifers endete auf *micro\_segmented* (Abschnitt 3.4); seine Stärke gerade dort kann deshalb ebenso Aktualität wie Breite sein – die Policy wurde zuletzt auf dieser Topologie angepasst, und früher Gelerntes wird dabei teilweise überschrieben. Die vorliegenden Daten lassen beide Deutungen zu; keine von ihnen lässt sich ausschließen.

Für die Fragestellung folgt daraus zweierlei. Erstens ist die Angreiferherkunft eine Einflussgröße eigener Art und darf nicht über alle Angreifer hinweg gemittelt werden, wie in Abschnitt 3.9 begründet. Zweitens beschreibt die Diagonale den ungünstigsten Fall, den ein Verteidiger zu erwarten hat, nämlich einen Gegner, der genau diese Topologie kennt.

## 4.8. Einordnung des Reward-Designs

Der auffälligste Befund dieses Kapitels ist, dass die wirksamsten Verteidiger kaum sperren. Diese Beobachtung lässt zwei Deutungen zu; anhand der erhobenen Daten lässt sich zwischen ihnen nicht abschließend entscheiden.

Die erste Deutung ist inhaltlich: In einem Netz, in dem ohnehin nur wenige Wege bestehen, ist das Neuaufsetzen das schärfere Mittel, weil es den Angreifer entfernt, statt ihn nur aufzuhalten. Dafür spricht, dass gerade die Verteidiger der beiden Topologien mit den wenigsten Kanten am häufigsten neu aufsetzen.

Die zweite Deutung liegt im Reward-Entwurf. Der Abwehrbonus ist mit  $0,00025 \cdot \text{Wert}(Z)$  je Schritt bewusst klein gehalten (Abschnitt 3.6.1), damit das Neuaufsetzen stets lohnender bleibt; das Verhältnis der Raten beträgt 20. Ein Reimage bringt dagegen sofort den vollen Knotenwert. Prävention ist damit zwar sichtbar, aber deutlich schwächer vergütet als Wiederherstellung, und der Agent folgt dieser Gewichtung.

Beide Deutungen sind mit den Daten vereinbar. Zu trennen wären sie nur durch Läufe mit systematisch veränderter Abwehrbonusrate, die in dieser Arbeit nicht durchgeführt wurden. Festzuhalten bleibt, dass die in Abschnitt 3.5.3 beschriebene Einschließungs-

strategie in keinem Lauf auftrat: Der Abwehrbonus genügte nicht, um eine mehrstufige Sperrkette aufzubauen.

Ob sich damit auch die Diagnose des Referenzlaufs bestätigt, nach der Prävention im Reward unsichtbar blieb (Abschnitt 3.1.3), lässt sich dagegen nicht sagen: Die beiden Läufe sind dafür zu verschieden (Abschnitt 4.9). Eine Kontrolle des Entwurfs liefert dafür die Obergrenze aus der Grundkopplung: Der Abwehrbonus ist so klein bemessen, dass er sich nicht zu positivem Reward farmen lassen soll (Abschnitt 3.6.1) – und tatsächlich bleibt der Verteidiger-Reward in jeder einzelnen Episode von Training und Evaluation negativ, der beste gemessene Einzelwert liegt bei  $-12,8$ .

## 4.9. Limitationen

Das Konvergenzkriterium unterscheidet die Topologien nicht. Es ist eigens für diese Arbeit entworfen und nicht durch Literatur gedeckt (Abschnitt 3.7); seine Schwäche liegt in der Normierung auf die Gesamtspanne des Laufs, die von den extremen Startwerten der Anfangsphase bestimmt wird. Ein strengerer Trendtest verschiebt den Zeitpunkt, ändert an dieser Bezugsgröße aber nichts (Abschnitt 4.4).

Ein Maß ohne diese Schwäche ließe sich so bilden: Man bestimmt je Lauf den Median seiner ersten und seiner letzten zehn Episoden und bestimmt, ab welcher Episode er 90 Prozent der Strecke dazwischen erreicht hat und zehn Episoden lang hält. Bezugspunkt ist damit die Verbesserung des Laufs selbst, nicht seine Gesamtspanne. Gesucht wird erst ab Episode 11, weil kein Lauf als konvergiert gelten kann, bevor sein eigener Bezugspunkt feststeht; der frühestmögliche Wert liegt damit bei Episode 20.

Angewendet auf die vorliegenden Daten erreichen alle 120 Läufe diese Marke, im Median in Episode 43. Die Mediane je Topologie lauten 50 für *flat*, 45 für *dmz*, 42 für *hub\_and\_spoke* und 39 für *micro\_segmented* und ordnen die Topologien damit genauso wie Reward, Crown-Jewel-Quote und gehaltene Knoten. Nachweisbar ist auch diese Ordnung nicht, die Quartile reichen von 38 bis 68 in *flat* und von 31 bis 55 in *micro\_segmented*. Der Befund aus Abschnitt 4.4 hängt also weder an der Schwelle noch an der Konstruktion des Maßes, sondern ebenso an der Zahl der Läufe. Weitere naheliegende Schritte wären ein Ausschluss der Anfangsphase und vor allem mehr Seeds je Zelle; eine Prüfung dieser Varianten liegt jedoch außerhalb des Umfangs dieser Arbeit.

Jede Zelle beruht auf fünf Seeds. Henderson et al. zeigen, dass allein der Zufallsstartwert

bei Policy-Gradient-Verfahren zu deutlich verschiedenen Ergebnisverteilungen führen kann, und raten deshalb zu mehr Läufen und zur Angabe von Konfidenzbereichen [23]. Fünf Seeds sind das untere Ende des dort empfohlenen Wertebereichs. Abgemildert wird das an zwei Stellen: Es werden alle Läufe berichtet und keine ausgewählt, und die Evaluation stützt sich auf Mediane, die ein einzelner Ausreißer nicht verschiebt. Für die Lernkurven gilt das nicht, dort schlägt ein einzelner Seed in das Mittel durch, wie Abschnitt 4.1 zeigt. Die berichteten Unterschiede zwischen den Topologien sind allerdings so groß, dass sie sich nicht durch Streuung zwischen Seeds erklären lassen; kleinere Unterschiede, etwa zwischen *dmz* und *micro\_segmented*, wären erst mit einer größeren Zahl von Läufen aussagekräftig.

Der Angreifer ist während des Verteidigertrainings eingefroren (Abschnitt 3.4). Der Verteidiger kann sich deshalb auf ein festes Verhalten einstellen, und die gemessene Wirksamkeit gilt zunächst nur gegen diesen einen Gegner. Die Auswertung je Matchup mildert diesen Umstand, da sechs verschiedene Angreifer geprüft werden, hebt ihn aber nicht vollständig auf. Das relativiert insbesondere den Befund, dass der passgenaue Angreifer am Trainingsende neutralisiert ist: Er gelingt einem Verteidiger, der sich ungestört auf genau dieses eine Verhalten einstellen konnte. Gegen einen mitlernenden Angreifer stünde diese Möglichkeit nicht offen, und der Einfluss der Angreiferherkunft könnte entsprechend größer ausfallen.

Auch die Reihenfolge, in der der Super-Angreifer die fünf Topologien durchlief, wurde nicht variiert. Sie endete auf *micro\_segmented*, und Abschnitt 4.7 zeigt, dass sich seine Stärke dort nicht von einem bloßen Aktualitätsvorteil der letzten Station trennen lässt.

Der Hub in *hub\_and\_spoke* ist ein übernehmbarer Knoten (Abschnitt 3.3.1). In einem realen Netz wäre die Vermittlungsstelle häufig ein Gerät ohne Nutzerdienste. Die Ergebnisse dieser Topologie sind deshalb nicht ohne Weiteres auf eine Sterntopologie mit gehärtetem Zentrum zu übertragen.

Zwei Entwurfsentscheidungen am Aktionsraum wurden nicht validiert. Der Dienststopp wurde entfernt, weil die knotenweite Sperre dieselbe Wirkung erzielt; ein Lauf mit beiden Aktionen nebeneinander wurde nicht gespielt (Abschnitt 3.5.2). Ebenso wurde die Sperre von portweise auf knotenweise umgestellt, ohne beide Varianten zu vergleichen; an dieser Entscheidung hängt auch die Deutung der Sperrwirkung in Abschnitt 4.5, denn eine portweise Sperre träfe in *flat* nicht alle neun Zugänge zugleich. Ungeprüft blieb schließlich, ob der Verteidiger in *flat* ohne sein dauerhaftes, überwiegend negativ verbuchtes Sperren besser abschnitte (Abschnitt 4.5); auch dies würde eigene Läufe erfordern.

Eine Lücke betrifft schließlich die Erhebung selbst. Die Aktionsstatistik zählt Freigaben

allein nach Aktionstyp und hält nicht fest, ob der betroffene Knoten zum Zeitpunkt der Freigabe überhaupt gesperrt war. Berechtigte Freigaben lassen sich deshalb nachträglich nicht von folgenlosen trennen, was die Deutung ihres Anteils in Abschnitt 4.5 begrenzt. Der Sperrzustand liegt zur Laufzeit vor; ihn mitzuschreiben wäre der naheliegende Weg, diese Lücke zu schließen.

Aus dem Vergleich mit dem Referenzlauf lassen sich nur eingeschränkt Ursachen ableiten. Zwischen beiden Läufen wurden mehrere Änderungen gleichzeitig vorgenommen, nämlich der Abwehrbonus, der Aktionsraum, die Maskierung, drei Fehlerkorrekturen – und der Wegfall des Abbruchkriteriums, das die Referenzläufe schon nach wenigen Dutzend Episoden beendete (Abschnitt 3.1.2). Welcher Anteil der Verbesserung auf welche Änderung entfällt, lässt sich daraus nicht bestimmen; insbesondere kann die Nähe von trainiertem und zufälligem Verteidiger im Referenzlauf auch an dessen kurzem Training gelegen haben, nicht allein an der im Reward unsichtbaren Prävention.

Schließlich bleibt der Aufbau eine Simulation. CBS bildet Angriffe als Folge diskreter Schritte auf einem gerichteten Graphen ab, ohne Zeitverhalten, ohne Erkennungsunsicherheit und mit einem Verteidiger, der den Zustand seines Netzes vollständig kennt. Die Ergebnisse sind Aussagen über dieses Modell. Ob sich die beobachtete Rangfolge der Topologien auf reale Netze überträgt, ist damit nicht gezeigt.

# 5

## Fazit und Ausblick

Dieses Kapitel fasst die Ergebnisse entlang der drei Teilfragen zusammen und leitet daraus eine Empfehlung für die Praxis ab. Der anschließende Ausblick benennt die Ansatzpunkte, an denen weitere Arbeiten anknüpfen können.

### 5.1. Fazit

Gegenstand dieser Arbeit war die Frage, wie sich die Netzwerktopologie auf RL-trainierte Verteidigungsagenten auswirkt (Abschnitt 1.2). Grundlage waren 120 Trainingsläufe über 24 Matchups aus vier verteidigten Topologien und sechs Angreifern sowie eine Evaluation mit 9000 Episoden gegen drei Vergleichsstufen (Kapitel 4). Für die drei Teilfragen ergibt sich ein unterschiedliches Bild.

Ein Einfluss der Topologie auf die *Lerngeschwindigkeit* ließ sich nicht nachweisen. Alle 120 Läufe erfüllen das Konvergenzkriterium, im Median in Episode 42, und die Mediane der vier Topologien liegen mit 41 bis 42,5 so dicht beieinander, dass sich darauf keine Aussage stützen lässt (Abschnitt 4.4). Das gilt auch für das nachträglich entworfene Alternativmaß; bei fünf Seeds je Zelle bleibt ein etwaiger Unterschied unterhalb der Nachweisgrenze (Abschnitt 4.9). Sichtbar wurde stattdessen eine Eigenschaft der Matchups: Gegen den Angreifer, der auf der verteidigten Topologie trainiert wurde, dauert es fast durchweg am längsten, bis ein Lauf konvergiert.

Deutlich trennt die Topologie dagegen das *erreichte Ergebnis*. Das Training wirkt zu-

nächst überall: Der Restanteil des trainierten Modells liegt in jeder der 24 Zellen unter dem des Zufallsverteidigers (Abschnitt 4.6). Wie viel dem Angreifer dennoch gelingt, bestimmt maßgeblich die Struktur: Mit zunehmender Segmentierung sinkt die Crown-Jewel-Quote, vom nahezu ungebremsten Zugriff im vollvermaschten *flat* über die beiden teilsegmentierten Topologien, die sich untereinander kaum trennen, bis zu *micro\_segmented*, wo der Zugriff fast vollständig unterbunden ist. Die Auswertung des Aktionsverhaltens liefert dazu den Mechanismus (Abschnitt 4.5): Wo die Struktur Engpässe bietet, werden Sperren wirksam; im vollvermaschten Netz weicht der Angreifer jeder Sperre aus, und auch dem trainierten Verteidiger bleibt kein greifendes Mittel. Die wirksamsten Verteidiger dieser Untersuchung sperren kaum, sondern setzen kompromittierte Knoten konsequent neu auf (Abschnitt 4.8). Dieser Befund beruht allerdings auf einer Vereinfachung der Simulation: Der Verteidiger kennt dort den Zustand jedes Knotens exakt und kann gezielt neu aufsetzen. In einem realen Netz ist gerade das ungewiss; dort wäre vermutlich die Sperre die naheliegende erste Maßnahme und das Wiederherstellen aus einem Backup die zweite.

Neben der verteidigten Topologie prägt auch die *Herkunft des Angreifers* das Ergebnis (Abschnitt 4.7). Der auf der verteidigten Topologie trainierte Angreifer beginnt um den Faktor 1,3 bis 4 stärker als jeder andere und beschreibt damit den ungünstigsten Fall, den ein Verteidiger zu erwarten hat. Der über alle Topologien gestuft trainierte Generalist ist den Spezialisten fast überall unterlegen; nur in *micro\_segmented* kommt er als einziger noch durch, wobei offen bleibt, ob dafür die Breite seines Trainings oder dessen letzte Station verantwortlich ist.

Neben diesen inhaltlichen Antworten steht ein methodisches Ergebnis. Der Reward allein erwies sich als unzureichendes Maß für Verteidigungsleistung: In *micro\_segmented* fällt die Crown-Jewel-Quote zwischen Episode 150 und 230 von 38 auf 5 Prozent, während sich der Reward kaum noch bewegt (Abschnitt 4.2) – erst die Zielgrößen Crown-Jewel-Quote und gehaltene Knoten machen sichtbar, was die Verteidigung tatsächlich leistet. Auch das eigens entworfene Konvergenzkriterium trennte die Topologien nicht, weil seine Normierung von den extremen Startwerten dominiert wird; beides ist für nachfolgende Arbeiten mit CBS unmittelbar verwertbar.

Für die Praxis lässt sich daraus – im Rahmen des Modells und mit den Einschränkungen aus Abschnitt 4.9 – eine Richtung ableiten: möglichst konsequente Segmentierung. Sie wirkt in dieser Untersuchung doppelt. Zum einen begrenzt die Struktur selbst, was ein Angreifer überhaupt erreichen kann, noch bevor ein Verteidiger handelt: In *micro\_segmented* bleiben ihm nur 145 der 235 Punkte, und den DMZ-Angreifer hielt dort bereits der Zufallsverteidiger in jeder Episode vom Crown Jewel fern (Tabelle 4.5). Zum anderen ist Segmentierung die Voraussetzung dafür, dass aktive Verteidigung

überhaupt greift: Erst Engpässe machen Sperren treffsicher, während im flachen Netz selbst der trainierte Agent das wertvollste System praktisch nie schützen konnte. Einem Unternehmen, das automatisierte Abwehr einsetzen will, wäre demnach zu raten, die Topologie nicht als gegebene Randbedingung zu behandeln, sondern als ersten Teil der Verteidigung – die Struktur dämmt den Angreifer ein, bevor der erste Agent eine Aktion wählt.

## 5.2. Ausblick

Die größte Einzeleinschränkung dieser Arbeit ist der eingefrorene Angreifer: Die gemessene Wirksamkeit gilt gegen ein festes Verhalten, auf das sich der Verteidiger ungestört einspielen konnte (Abschnitt 4.9). Zwei Erweiterungen lägen nahe: zunächst Angreifer, die gegen einen zufällig handelnden Verteidiger trainiert wurden und Gegenwehr damit überhaupt kennen, und darauf aufbauend ein Versuch, in dem beide Agenten von null an gleichzeitig lernen. Erst dort zeigt sich, ob die Neutralisierung des passgenauen Angreifers Bestand hat oder ob der Einfluss der Angreiferherkunft gegen einen ausweichenden Gegner wächst.

Am gestuften Training des Generalisten blieb unentschieden, ob seine Stärke auf *micro\_segmented* Breite oder Aktualität ist. Läufe mit permutierter Stationenfolge würden beides trennen; ergänzend ließe sich eine Effektivitätsmatrix eingefrorener Angreifer gegen eingefrorene Verteidiger aller Stationen aufnehmen. Ebenso stehen die in Abschnitt 4.9 benannten Entwurfsprüfungen aus: eine systematisch variierte Abwehrbonusrate, der Vergleich port- gegen knotenweiser Sperren, ein Lauf ohne Sperren in *flat* sowie eine ausdrückliche Nichtstun-Aktion beziehungsweise ein Action-Masking. Ob die ungültigen Aktionen, zuletzt 62 Prozent aller Verteidigeraktionen, tatsächlich gewollte Untätigkeit sind oder der Agent schlicht nicht gelernt hat, welche Aktionen gültig sind, ließe sich erst damit unterscheiden.

Die statistische Basis ließe sich mit mehr Seeds je Zelle und Konfidenzbereichen verbreitern, wie von Henderson et al. empfohlen [23]; kleinere Unterschiede zwischen den segmentierten Topologien wären erst damit aussagekräftig. Für die Lerngeschwindigkeit fehlt zudem weiterhin ein Maß, das späte Verbesserungen von früher Ruhe trennt; die in Abschnitt 4.9 vorgeschlagene 90-Prozent-Marke wäre ein Ausgangspunkt.

Schließlich bleibt die Übertragung. CBS abstrahiert von Zeitverhalten, Erkennungsunsicherheit und unvollständiger Sicht des Verteidigers; die hier gefundene Rangfolge der Topologien ist eine Aussage über dieses Modell. Größere und heterogenere Netze, eine

Sterntopologie mit gehärtetem, nicht übernehmbarem Zentrum sowie Umgebungen mit Teilbeobachtbarkeit würden zeigen, ob der doppelte Nutzen der Segmentierung auch dort trägt, wo reale Netze weniger ideal geschnitten sind.

# Literaturverzeichnis

- [1] CrowdStrike. *2026 Global Threat Report*. 2026. URL: <https://www.crowdstrike.com/en-us/global-threat-report/> (besucht am 28.08.2026).
- [2] IBM Corporation. *X-Force Threat Intelligence Index 2026*. 2026. URL: <https://www.ibm.com/reports/threat-intelligence> (besucht am 28.08.2026).
- [3] Thanh Thi Nguyen und Vijay Janapa Reddi. “Deep Reinforcement Learning for Cyber Security”. In: *IEEE Transactions on Neural Networks and Learning Systems* 34.8 (Aug. 2023), S. 3779–3795. ISSN: 2162-2388. DOI: [10.1109/TNNLS.2021.3121870](https://doi.org/10.1109/TNNLS.2021.3121870). URL: <https://doi.org/10.1109/TNNLS.2021.3121870>.
- [4] Microsoft Defender Research Team. *CyberBattleSim*. <https://github.com/microsoft/CyberBattleSim>. Zugriff: 21.08.2026. 2021.
- [5] Maxwell Standen u. a. *CybORG: A Gym for the Development of Autonomous Cyber Agents*. IJCAI-21 1st International Workshop on Adaptive Cyber Defense. 2021. arXiv: [2108.09118](https://arxiv.org/abs/2108.09118) [cs.CR]. URL: <https://arxiv.org/abs/2108.09118>.
- [6] Thomas Kunz u. a. “A Multiagent CyberBattleSim for RL Cyber Operation Agents”. In: *2022 International Conference on Computational Science and Computational Intelligence (CSCI)*. 2022, S. 897–903. DOI: [10.1109/CSCI58124.2022.00161](https://doi.org/10.1109/CSCI58124.2022.00161).
- [7] Scott Rose u. a. *Zero Trust Architecture*. NIST Special Publication 800-207. Gaithersburg, MD: National Institute of Standards und Technology, Aug. 2020. DOI: [10.6028/NIST.SP.800-207](https://doi.org/10.6028/NIST.SP.800-207).
- [8] Richard S. Sutton und Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2. Aufl. Adaptive Computation and Machine Learning. Cambridge, MA: The MIT Press, 2018. ISBN: 9780262039246. URL: <http://incompleteideas.net/book/the-book-2nd.html> (besucht am 22.08.2026).
- [9] Marvin Minsky. “Steps toward Artificial Intelligence”. In: *Proceedings of the IRE* 49.1 (1961), S. 8–30. DOI: [10.1109/JRPROC.1961.287775](https://doi.org/10.1109/JRPROC.1961.287775).
- [10] Volodymyr Mnih u. a. “Human-level control through deep reinforcement learning”. In: *Nature* 518.7540 (Feb. 2015), S. 529–533. ISSN: 1476-4687. DOI: [10.1038/nature14236](https://doi.org/10.1038/nature14236). URL: <https://doi.org/10.1038/nature14236>.
- [11] John Schulman u. a. “Proximal Policy Optimization Algorithms”. In: *CoRR* abs/1707.06347 (2017). arXiv: [1707.06347](https://arxiv.org/abs/1707.06347). URL: <http://arxiv.org/abs/1707.06347>.

- [12] John Schulman u. a. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. 2015. arXiv: 1506.02438 [cs.LG]. URL: <https://arxiv.org/abs/1506.02438>.
- [13] Neil de la Fuente und Daniel A. Vidal Guerra. *A Comparative Study of Deep Reinforcement Learning Models: DQN vs PPO vs A2C*. 2024. arXiv: 2407.14151 [cs.LG]. URL: <https://arxiv.org/abs/2407.14151>.
- [14] Rohit Gangupantulu u. a. *Crown Jewels Analysis using Reinforcement Learning with Attack Graphs*. 2021. arXiv: 2108.09358 [cs.CR]. URL: <https://arxiv.org/abs/2108.09358>.
- [15] The MITRE Corporation. *MITRE ATT&CK: Lateral Movement, Taktik TA0008 (Enterprise)*. Version v19. Erstellt am 17. Oktober 2018, zuletzt geändert am 11. August 2025. 2025. URL: <https://attack.mitre.org/versions/v19/tactics/TA0008/> (besucht am 12. 08. 2026).
- [16] The MITRE Corporation. *MITRE ATT&CK: OS Credential Dumping: NTDS, Technik T1003.003 (Enterprise)*. Version v19. Erstellt am 11. Februar 2020, zuletzt geändert am 12. Mai 2026. 2026. URL: <https://attack.mitre.org/versions/v19/techniques/T1003/003/> (besucht am 13. 08. 2026).
- [17] Karen Scarfone und Paul Hoffman. *Guidelines on Firewalls and Firewall Policy*. NIST Special Publication 800-41 Rev. 1. Gaithersburg, MD: National Institute of Standards und Technology, Sep. 2009. DOI: [10.6028/NIST.SP.800-41r1](https://doi.org/10.6028/NIST.SP.800-41r1).
- [18] Microsoft. *Hub-spoke network topology in Azure*. Azure Architecture Center, Referenzarchitektur; Seitenstand 10. April 2026; Hauptautoren Jose Moreno, Alejandra Palacios, Adam Torkar. 2026. URL: <https://learn.microsoft.com/en-us/azure/architecture/networking/architecture/hub-spoke> (besucht am 17. 08. 2026).
- [19] Antonin Raffin u. a. "Stable-Baselines3: Reliable Reinforcement Learning Implementations". In: *Journal of Machine Learning Research* 22.268 (2021), S. 1–8. URL: <http://jmlr.org/papers/v22/20-1364.html>.
- [20] Li Li, Raed Fayad und Adrian Taylor. *CyGIL: A Cyber Gym for Training Autonomous Agents over Emulated Network Systems*. IJCAI-21 1st International Workshop on Adaptive Cyber Defense. 2021. arXiv: 2109.03331 [cs.CR]. URL: <https://arxiv.org/abs/2109.03331>.
- [21] Li Li u. a. *Unified Emulation-Simulation Training Environment for Autonomous Cyber Agents*. 2023. arXiv: 2304.01244 [cs.LG]. URL: <https://arxiv.org/abs/2304.01244>.

- 
- [22] Shengyi Huang und Santiago Ontañón. “A Closer Look at Invalid Action Masking in Policy Gradient Algorithms”. In: *The International FLAIRS Conference Proceedings* 35 (Mai 2022). ISSN: 2334-0762. DOI: [10.32473/flairs.v35i.130584](https://doi.org/10.32473/flairs.v35i.130584). URL: <https://journals.flvc.org/FLAIRS/article/view/130584>.
- [23] Peter Henderson u. a. *Deep Reinforcement Learning that Matters*. 2017. arXiv: [1709.06560](https://arxiv.org/abs/1709.06560) [cs.LG]. URL: <https://arxiv.org/abs/1709.06560>.

# A

## Anhang

Der Anhang dokumentiert die Einstellungen der Trainingsläufe und zeigt jene Verläufe vollständig, die Kapitel 4 nur ausschnittsweise wiedergibt. Die Abbildungen decken alle 24 Matchups ab, getrennt nach verteidigter Topologie: Crown-Jewel-Quote, gehaltene Knoten, Schritte mit SLA-Bruch, Abwehrbonus und Konvergenzmarken.

### A.1. Quellcode und Daten

Der Quellcode dieser Arbeit, die Rohdaten beider Läufe und die Skripte, die sämtliche Abbildungen und Kennzahlen daraus erzeugen, sind dauerhaft archiviert unter [10.5281/zenodo.22207721](https://zenodo.org/record/22207721) und werden unter <https://github.com/QuasiMika/rl-defender-network-topology> weitergeführt.

Enthalten sind die Trainings- und Evaluationsprotokolle als CSV-Dateien, die eingefrorenen Angreifermodelle, der Topologie-Generator sowie Skripte, mit denen sich jede im Text genannte Zahl unabhängig gegen die Rohdaten nachrechnen lässt. Die trainierten Verteidigermodelle sind wegen ihres Umfangs nicht enthalten; für die Auswertung werden sie nicht benötigt.

Das Mehragenten-Rahmenwerk MARLon ist nicht Teil des Archivs, da es ohne Lizenz veröffentlicht wurde. Ein beiliegendes Skript lädt es beim Urheber auf dem Stand, gegen den entwickelt wurde, und spielt die Anpassungen dieser Arbeit als Patch ein.

## A.2. Hyperparameter

Die Agenten werden mit den Voreinstellungen von SB3 trainiert; im Programmcode wird kein Hyperparameter gesetzt. Tabelle A.1 führt die Werte auf, die daraus folgen.

Diese Zurückhaltung ist beabsichtigt. Würden die Parameter angepasst, so geschähe das notwendigerweise an einer bestimmten Topologie, und die dort gefundenen Werte bevorzugten diese gegenüber den übrigen. Da der Vergleich der Topologien der Gegenstand der Arbeit ist, wäre damit genau die unabhängige Variable verunreinigt. Einheitliche Voreinstellungen halten diesen Einfluss konstant.

Tabelle A.1: Hyperparameter des Trainings. Alle Werte entsprechen den Voreinstellungen von SB3 in der eingesetzten Fassung; abweichend gesetzt wird allein die Zahl der Zeitschritte.

| Parameter                        | Wert    | Bedeutung                                                                               |
|----------------------------------|---------|-----------------------------------------------------------------------------------------|
| <code>learning_rate</code>       | 0,0003  | Schrittweite $\alpha$ in Gleichung (2.4)                                                |
| <code>n_steps</code>             | 2048    | Länge eines Rollouts, also gesammelte Schritte je Lernphase                             |
| <code>batch_size</code>          | 64      | Größe eines Minibatches                                                                 |
| <code>n_epochs</code>            | 10      | Durchgänge über denselben Rollout                                                       |
| <code>gamma</code>               | 0,99    | Diskontfaktor $\gamma$ aus Gleichung (2.1)                                              |
| <code>gae_lambda</code>          | 0,95    | Gewichtung zwischen Critic-Schätzung und beobachteten Rewards bei der Vorteilsschätzung |
| <code>clip_range</code>          | 0,2     | Breite des Bandes $\varepsilon$ in Gleichung (2.7)                                      |
| <code>normalize_advantage</code> | wahr    | Vorteile werden je Minibatch auf Mittelwert null und Streuung eins gebracht             |
| <code>ent_coef</code>            | 0,0     | Gewicht des Entropie-Anteils, der zusätzliche Exploration begünstigen würde             |
| <code>vf_coef</code>             | 0,5     | Gewicht des Critic-Verlusts gegenüber dem des Actors                                    |
| <code>max_grad_norm</code>       | 0,5     | Obergrenze für die Länge des Gradienten je Schritt                                      |
| Zeitschritte je Lauf             | 500.000 | eigens gesetzt, siehe Abschnitt 3.4                                                     |

### A.3. Crown-Jewel-Quote je Topologie

Die folgenden vier Abbildungen zeigen den Anteil der Episoden mit erreichtem Crown Jewel für alle 24 Matchups. Abschnitt 4.2 stellt daraus nur die vier Matchups auf der Diagonalen dar. Gebildet ist die Quote wie dort über ein Fenster von elf Episoden und alle fünf Seeds; die Skala ist in allen Feldern dieselbe. Fett überschrieben ist jeweils das Feld des Angreifers, der auf der verteidigten Topologie trainiert wurde.

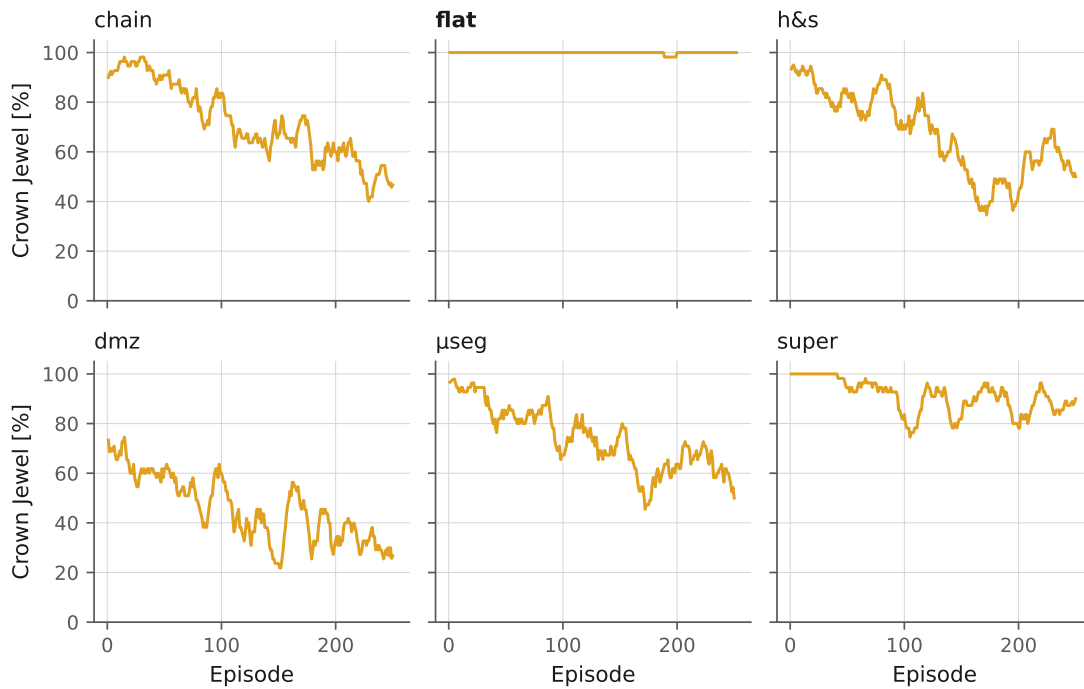
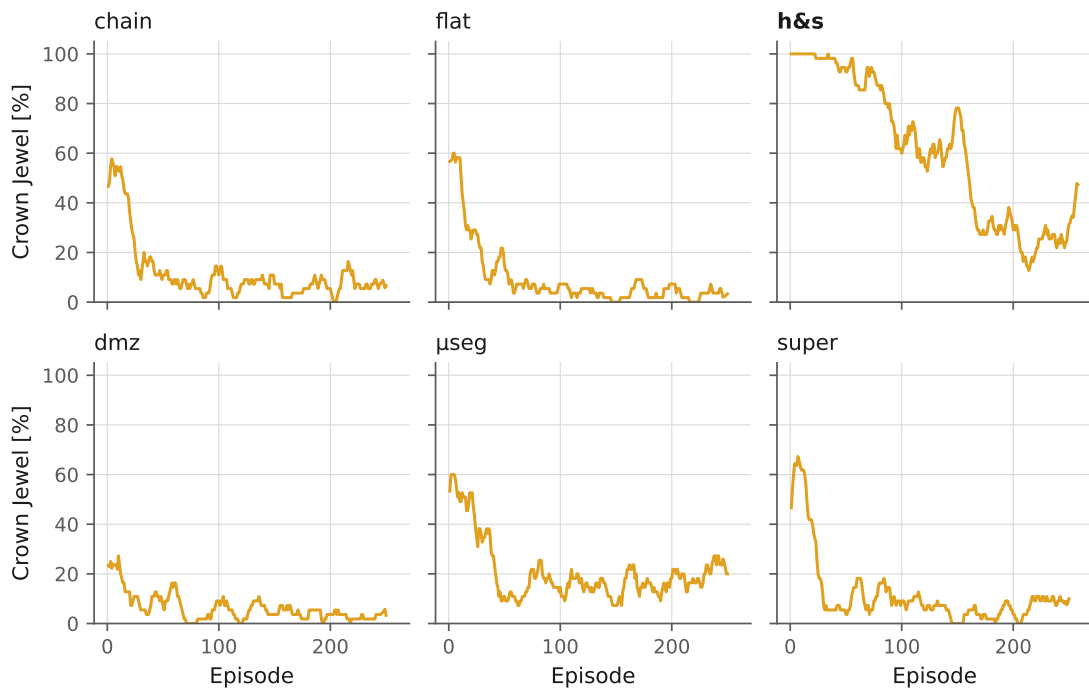
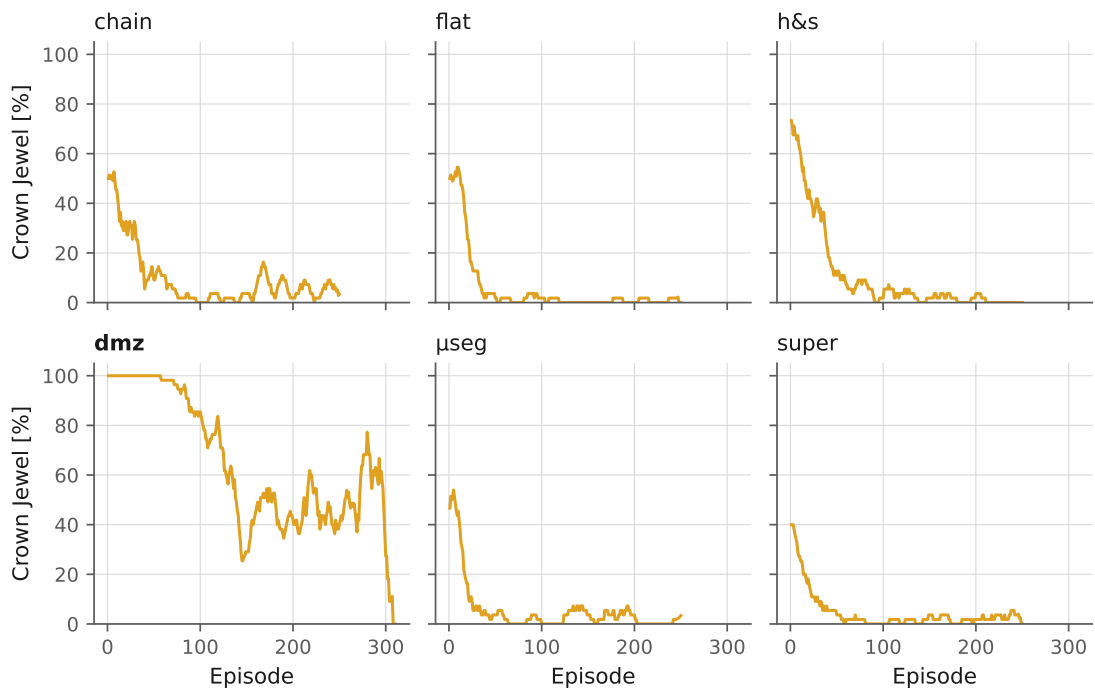


Abbildung A.1: Crown-Jewel-Quote der Verteidiger auf *flat* gegen alle sechs Angreifer.

Abbildung A.2: Crown-Jewel-Quote der Verteidiger auf *hub\_and\_spoke* gegen alle sechs Angreifer.Abbildung A.3: Crown-Jewel-Quote der Verteidiger auf *dmz* gegen alle sechs Angreifer.

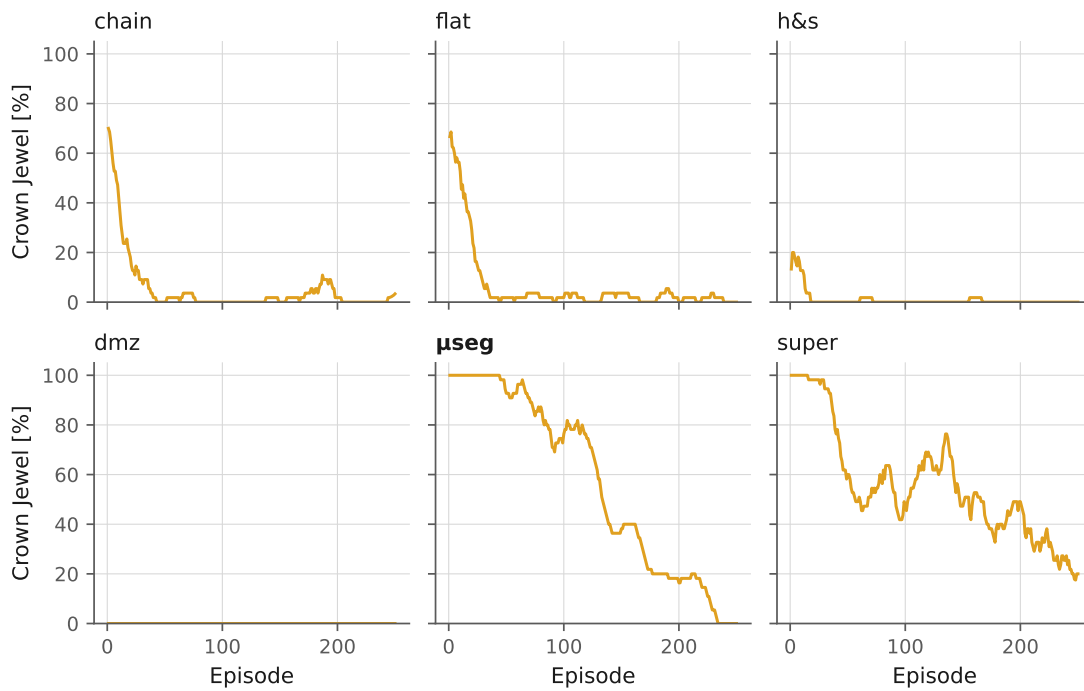
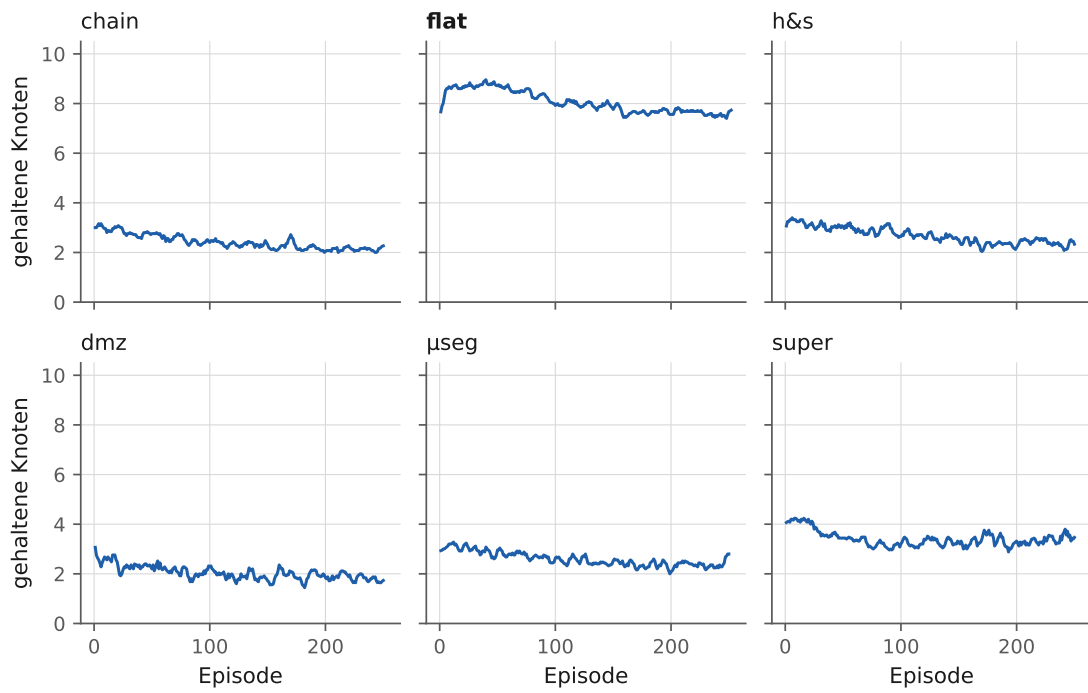
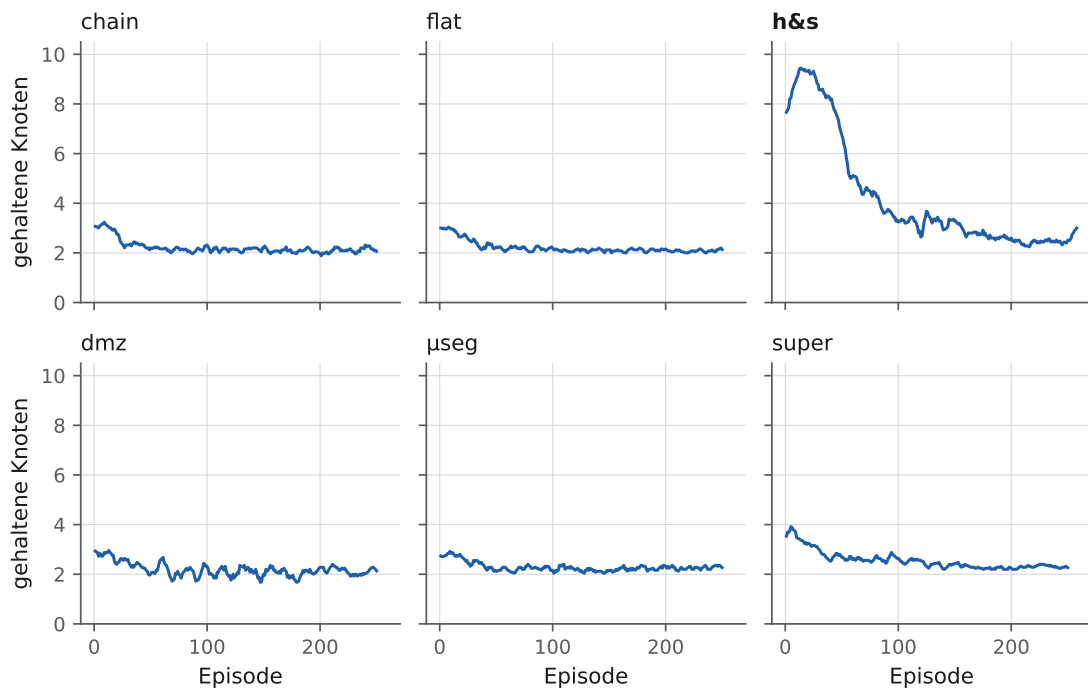
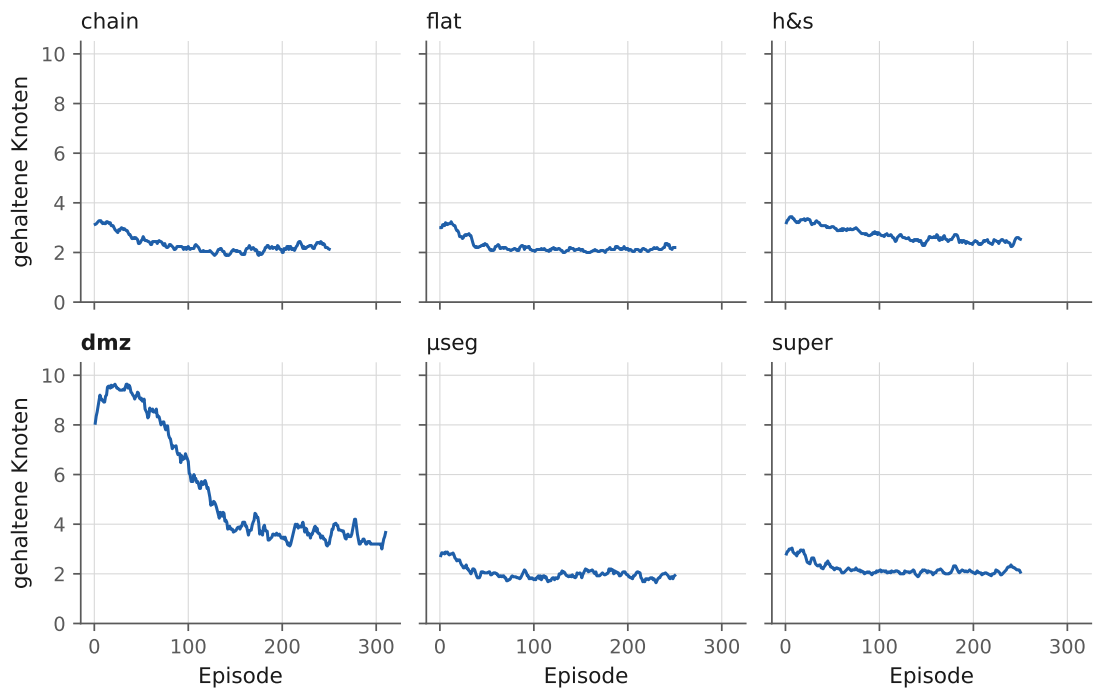
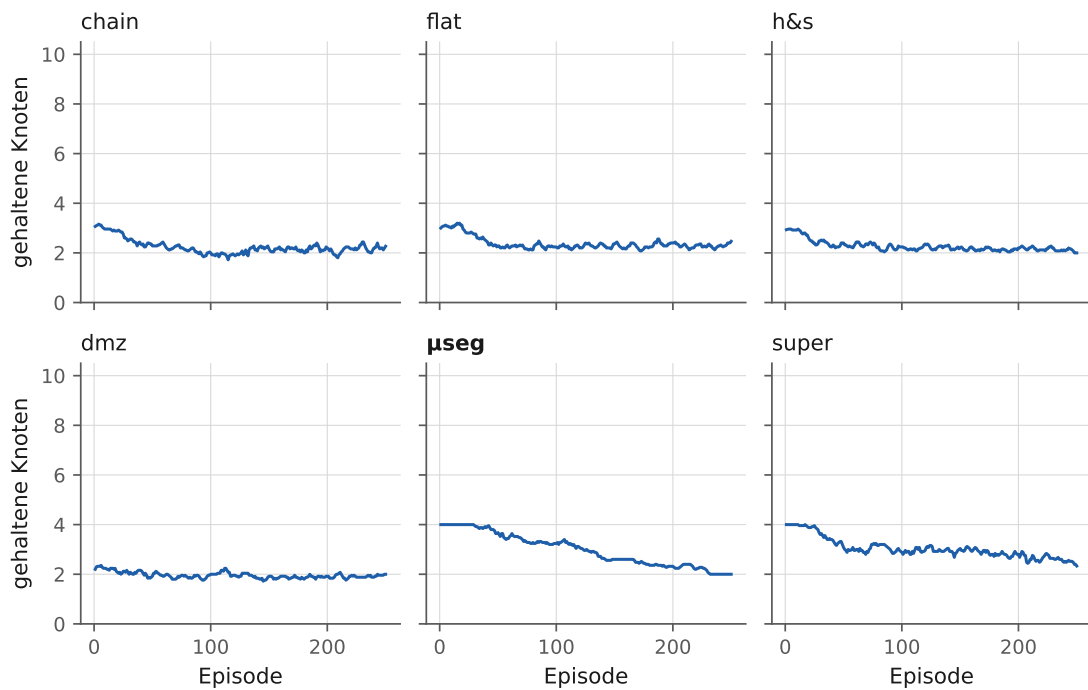


Abbildung A.4: Crown-Jewel-Quote der Verteidiger auf *micro\_segmented* gegen alle sechs Angreifer.

## A.4. Gehaltene Knoten je Topologie

Die folgenden vier Abbildungen zeigen die größte Zahl gleichzeitig gehaltener Knoten je Episode für alle 24 Matchups, gemittelt über die fünf Seeds und geglättet über fünf Episoden. Abschnitt 4.3 stellt daraus nur die vier Matchups auf der Diagonalen dar. Die Skala reicht in allen Feldern von null bis zehn Knoten. Fett überschrieben ist jeweils das Feld des Angreifers, der auf der verteidigten Topologie trainiert wurde.

Abbildung A.5: Gehaltene Knoten der Verteidiger auf *flat* gegen alle sechs Angreifer.Abbildung A.6: Gehaltene Knoten der Verteidiger auf *hub\_and\_spoke* gegen alle sechs Angreifer.

Abbildung A.7: Gehaltene Knoten der Verteidiger auf *dmz* gegen alle sechs Angreifer.Abbildung A.8: Gehaltene Knoten der Verteidiger auf *micro\_segmented* gegen alle sechs Angreifer.

## A.5. SLA-Brüche je Topologie

Die folgenden vier Abbildungen zeigen, in wie vielen Schritten je Episode die Verfügbarkeit unter der Schwelle lag, gemittelt über die fünf Seeds. Jedes Feld trägt eine eigene Skala, weil die Werte weit auseinanderliegen;  $\Sigma$  nennt die Summe aller Bruchschritte des Matchups über alle fünf Seeds. Abbildung 4.3 zeigt zwei dieser Fälle nach Seeds getrennt.

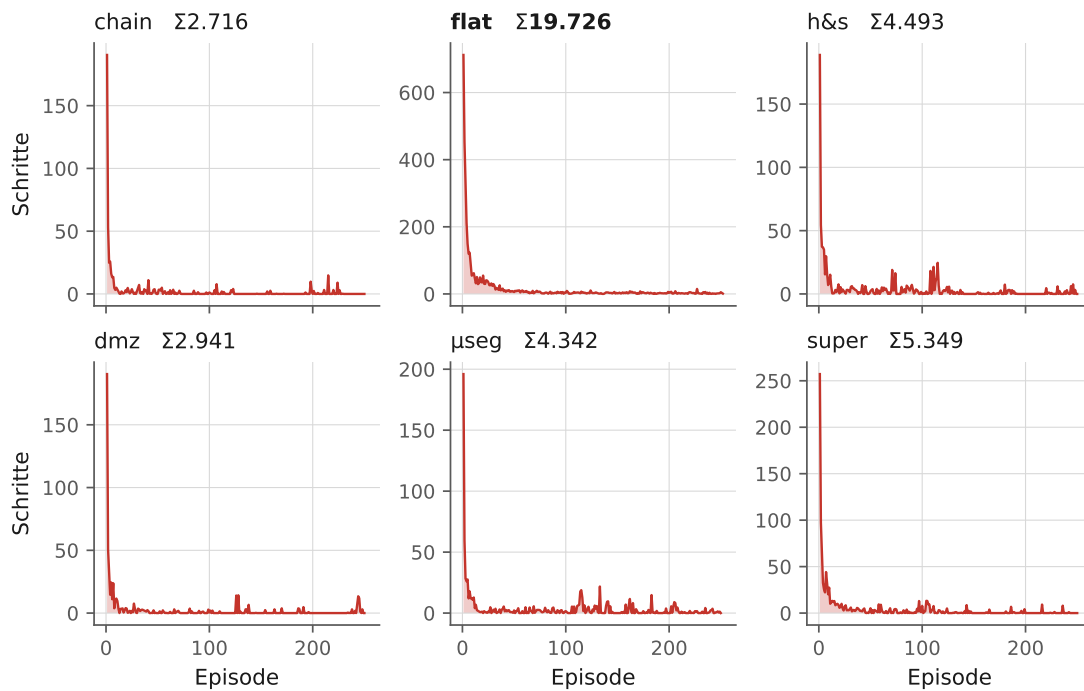
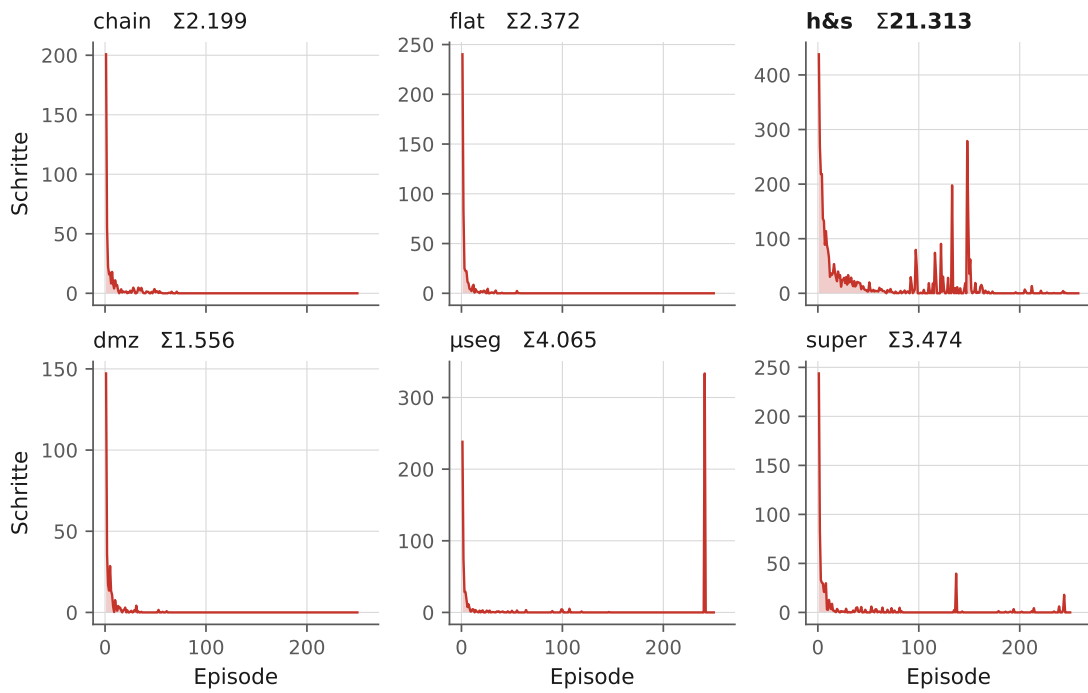
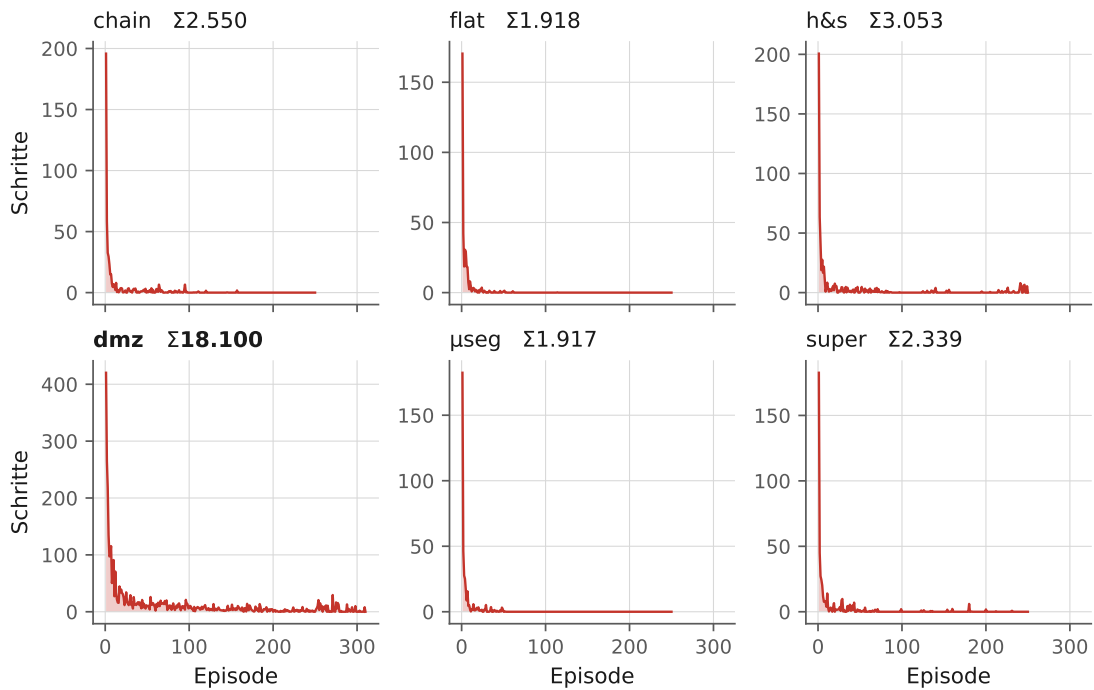


Abbildung A.9: Schritte mit SLA-Bruch der Verteidiger auf *flat* gegen alle sechs Angreifer.

Abbildung A.10: Schritte mit SLA-Bruch der Verteidiger auf *hub\_and\_spoke* gegen alle sechs Angreifer.Abbildung A.11: Schritte mit SLA-Bruch der Verteidiger auf *dmz* gegen alle sechs Angreifer.

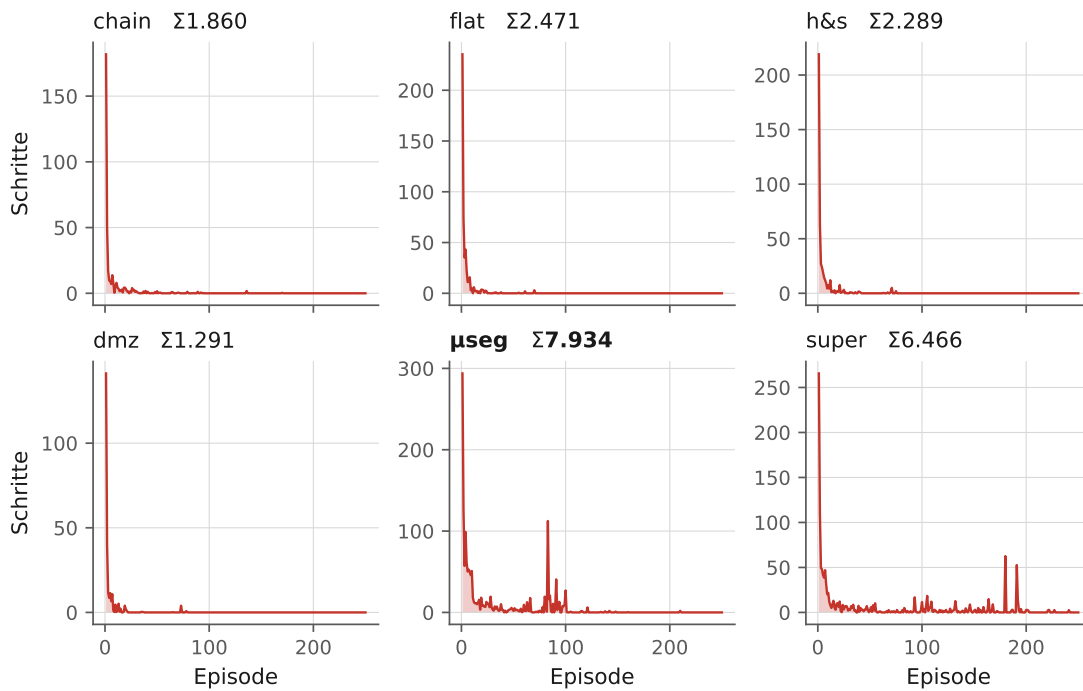
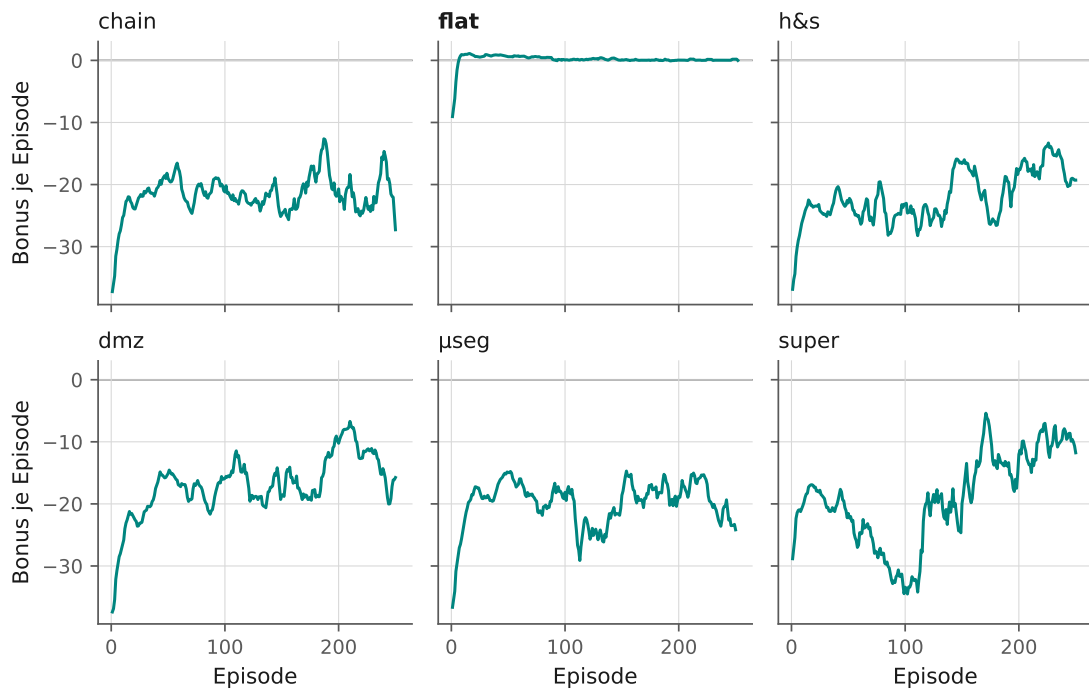
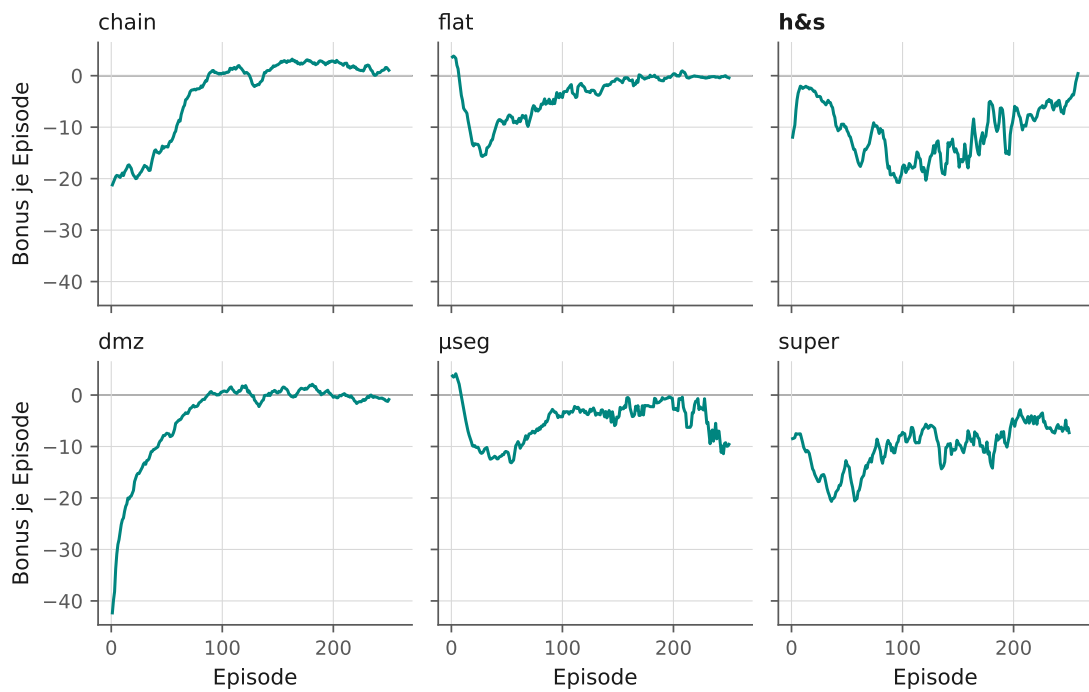
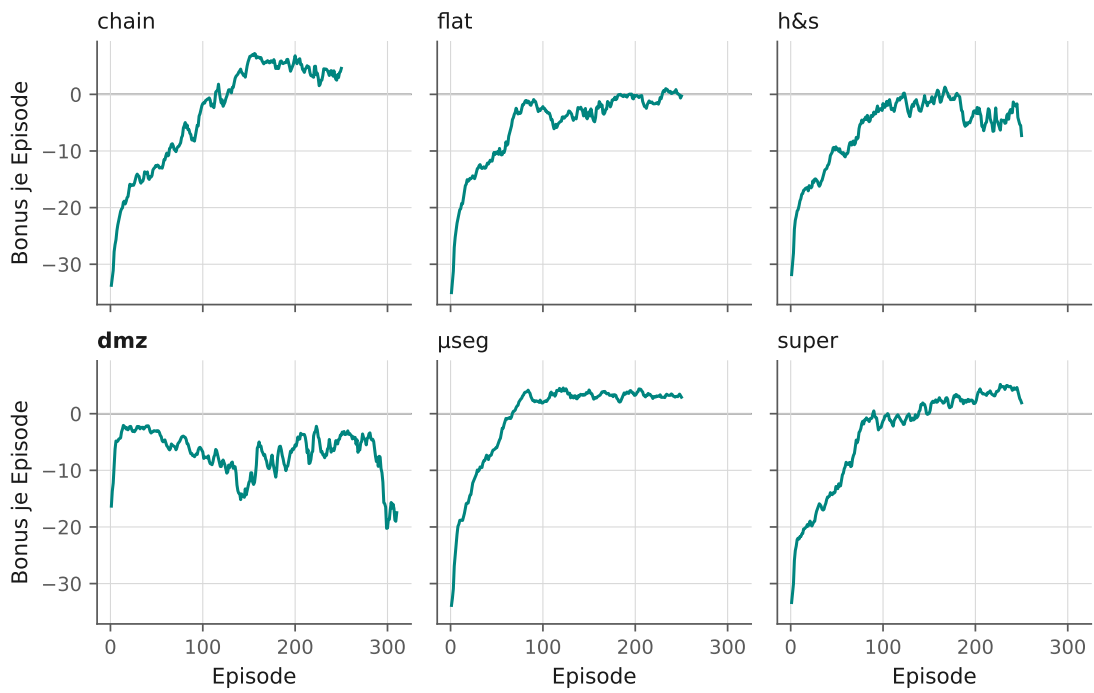
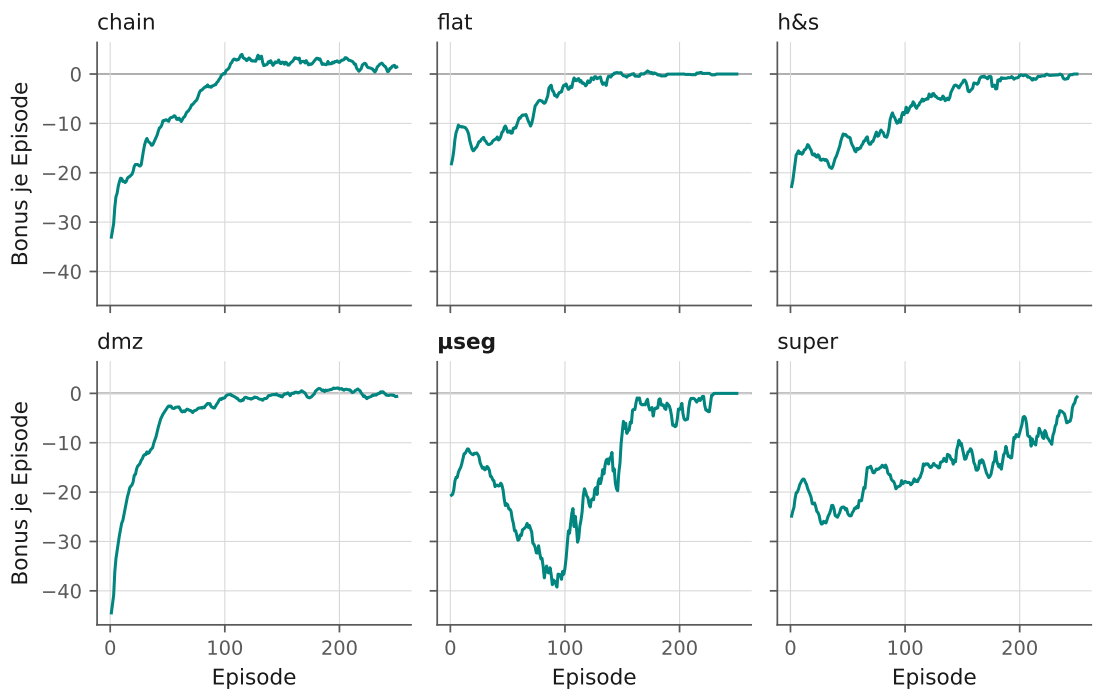


Abbildung A.12: Schritte mit SLA-Bruch der Verteidiger auf *micro\_segmented* gegen alle sechs Angreifer.

## A.6. Abwehrbonus je Topologie

Die folgenden vier Abbildungen zeigen den Abwehrbonus je Episode für alle 24 Matchups, gemittelt über die fünf Seeds und geglättet über fünf Episoden; Abschnitt 4.5 zeigt daraus nur das Mittel über die sechs Angreifer je Topologie. Die Skala ist in allen Feldern dieselbe, die Nulllinie ist eingezeichnet. Fett überschrieben ist jeweils das Feld des Angreifers, der auf der verteidigten Topologie trainiert wurde.

Abbildung A.13: Abwehrbonus der Verteidiger auf *flat* gegen alle sechs Angreifer.Abbildung A.14: Abwehrbonus der Verteidiger auf *hub\_and\_spoke* gegen alle sechs Angreifer.

Abbildung A.15: Abwehrbonus der Verteidiger auf *dmz* gegen alle sechs Angreifer.Abbildung A.16: Abwehrbonus der Verteidiger auf *micro\_segmented* gegen alle sechs Angreifer.

## A.7. Konvergenzmarken je Topologie

Die folgenden vier Abbildungen zeigen für alle 24 Matchups, an welcher Stelle des Verlaufs die beiden in Abschnitt 4.4 verglichenen Maße greifen. Gezeichnet ist der Verteidiger-Reward, gemittelt über die fünf Seeds und geglättet über fünf Episoden. Die drei Senkrechten markieren das Kriterium aus Abschnitt 3.7, dasselbe Kriterium mit dem strengeren Trendtest aus Abschnitt 4.4 und die 90-Prozent-Marke aus Abschnitt 4.9, jeweils gemittelt über die fünf Seeds. Fett überschrieben ist das Feld des Angreifers, der auf der verteidigten Topologie trainiert wurde.

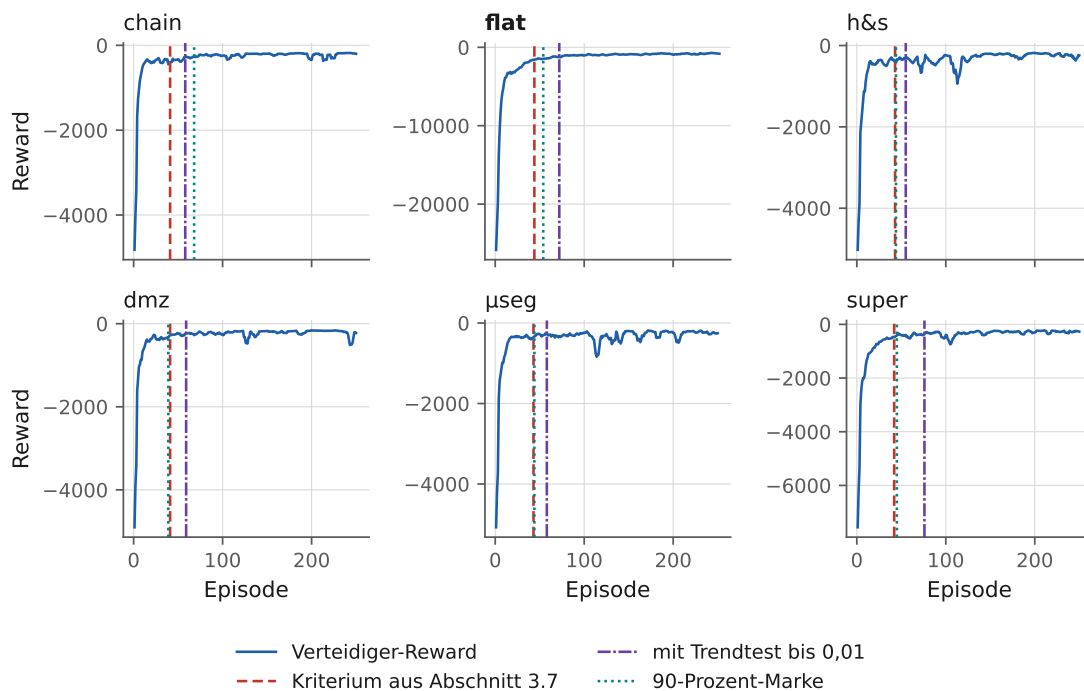
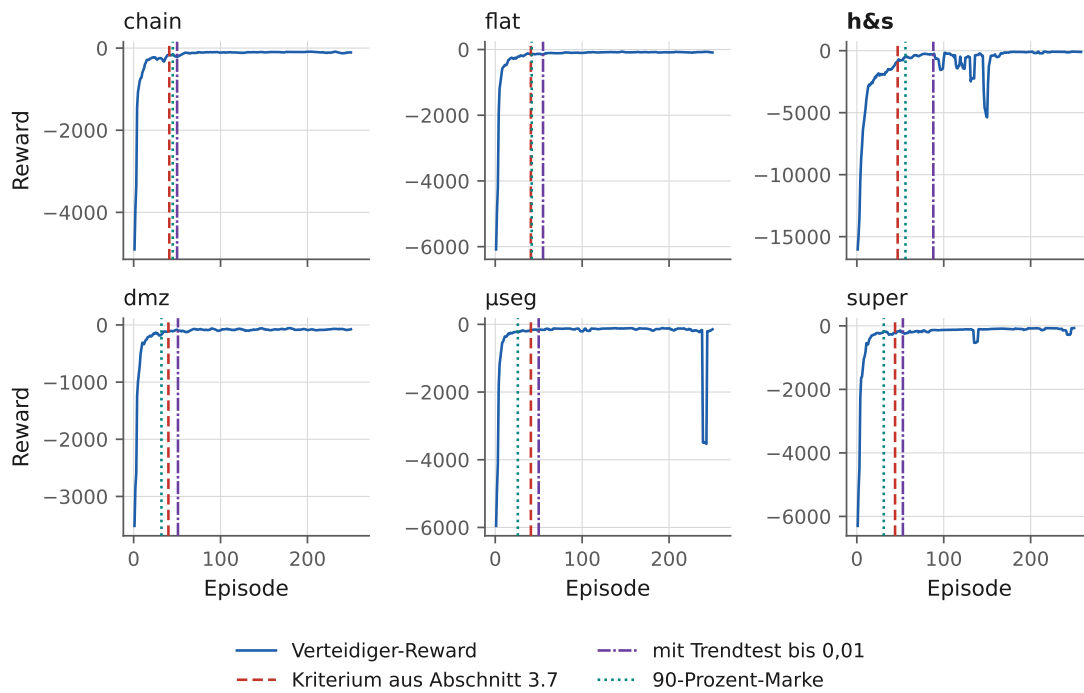
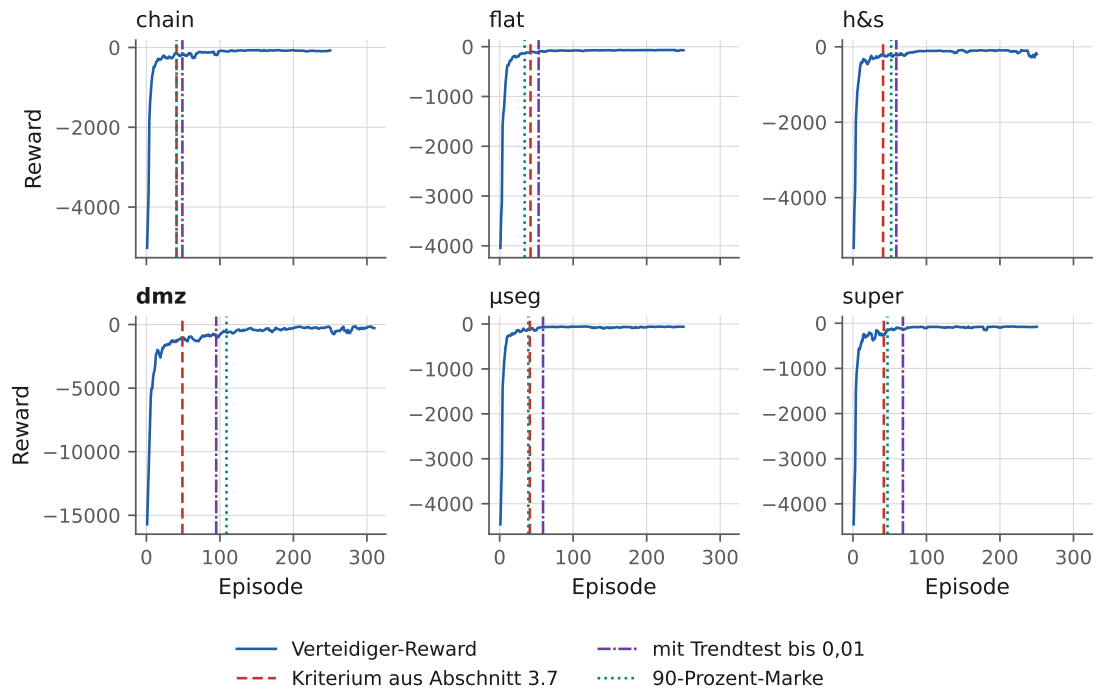


Abbildung A.17: Konvergenzmarken der Verteidiger auf *flat* gegen alle sechs Angreifer.

Abbildung A.18: Konvergenzmarken der Verteidiger auf *hub\_and\_spoke* gegen alle sechs Angreifer.Abbildung A.19: Konvergenzmarken der Verteidiger auf *dmz* gegen alle sechs Angreifer.

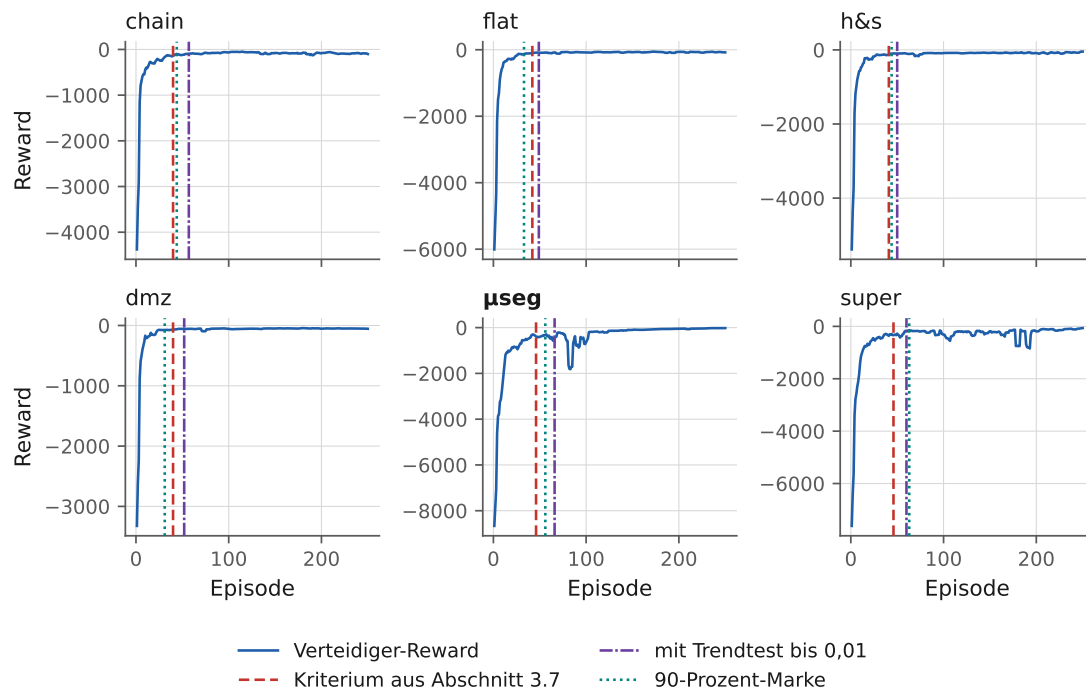


Abbildung A.20: Konvergenzmarken der Verteidiger auf *micro\_segmented* gegen alle sechs Angreifer.

# Eidesstattliche Erklärung

Ich versichere an Eides statt, dass ich die vorliegende Bachelorarbeit selbstständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Schriften entnommen wurden, sind als solche kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Beim Anfertigen dieser Arbeit wurden KI-gestützte Werkzeuge eingesetzt: Claude Code zur Unterstützung bei der Programmierung und bei der sprachlichen Überarbeitung des Textes, Gemini Notebook zur Unterstützung bei der Literaturrecherche. Sämtliche auf diesem Weg entstandenen Texte und Programmteile habe ich geprüft und überarbeitet. Die Fragestellung, der Versuchsaufbau sowie die Auswertung und Einordnung der Ergebnisse verantworte ich; alle verwendeten Quellen sind im Literaturverzeichnis nachgewiesen und wurden im Original geprüft.



Ort, Datum



Unterschrift